

Grupo 4 - Amigo Perto

Nome: Eric Senne Roma, Vitor Gomes e Antônio Almeida

Etapa 1 Semana 1	5
Perfis de clientes	5
Diagrama de Contexto	7
Tabelas - Requisitos Funcionais e Não Funcionais	9
Requisitos Funcionais	9
Requisitos de Sistema	9
Requisitos de Software	11
Requisitos Não Funcionais	12
TRL	13
Etapa 1 Semana 2	14
Requisitos Consolidados	14
Requisitos Funcionais	14
Requisitos de Sistema	14
Requisitos de Software	16
Requisitos Não Funcionais	18
Diagrama de Blocos	21
Diagrama de Hardware	21
Diagrama de Software	22
Diagrama de camadas	23
Interfaces e Dependências principais	28
Etapa 1 Semana 3	33
Plano de Abstração (HAL)	33
Arquitetura de Software	33
Camada HAL (Hardware Abstraction Layer)	39
HAL do BLE	39
HAL do Buzzer	41
HAL da Bateria	41
HAL do Botão	42
Comparação microcontrolador	42
Justificativas técnicas e econômicas	43
Diagrama consolidado do sistema	44
Etapa 2 Semana 1	46
Pesquisa dos conhecimentos necessários para o app (back-end e front-end)	46
1. O que saber sobre Android BLE	46
2. O que saber sobre nRF52840 / nRF54L15	48
3. Arquitetura recomendada (Android + nRF)	50
4. Estrutura do projeto Android/Kotlin	50
5. Exemplo de código Android BLE (Kotlin)	50
6. Exemplo no nRF52840 (Zephyr / NCS)	52
7. O que pesquisar:	52

ROADMAP COMPLETO – 3 SEMANAS	55
Estudo do curso “Bluetooth Low Energy Fundamentals” da Nordic	61
Bluetooth Low Energy (BLE)	61
Modo Advertising	66
Modo Conexão	71
GATT e ATT – visão geral	73
Segurança no BLE	76
Funções de BLE do SDK que serão usadas no projeto	80
Etapa 2 Semana 2	82
Cronograma	82
1) Início da implementação do App Android	82
2) Teste do Buzzer	83
3) Teste BLE Advertising	83
4) Teste BLE Conexão	84
Início da implementação do App Android	84
Teste BLE Advertising	87
Teste BLE Conexão	1
Etapa 2 Semana 3	1
Relatório de Resultados de Testes Integrados.	1
4. Cenário de Testes e Resultados:	1
5. Análise e Conclusão	1
Revisão e justificativa do TRL atualizado:	1
Plano de ação para otimização e integração completa na Etapa 3.	1
Diagrama consolidado do Sistema Integrado até o Momento	1
Etapa 3 Semana 1	1
Descrição das Integração	1
1. Arquitetura do Sistema	1
2. Módulos Implementados	1
2.1 HAL (Hardware Abstraction Layer)	1
2.2 GATT Services	1
3. Fluxo de Dados	1
Inicialização	1
Controle Remoto (BLE → Buzzer)	1
Leitura de Bateria (BLE → Battery)	1
Eventos BLE (Stack → Main)	1
4. Tabela de Comunicação entre Módulos	1
5. Dependências e Ordem de Inicialização	1
6. Módulo Pendente	1
HAL Button (include/hal/button.h + src/hal/button.c)	1
Otimização de Energia	1
Etapa 3 Semana 2	1
Refatoração (R1.2)	1
Melhorias no Sistema de Bateria	1
Atualização do Serviço de Buzzer (18 kHz)	1

Ajustes de Integração	1
Teste de Consumo Energético	1
1. Objetivo	1
2. Metodologia	1
3. Resultados	1
4. Checklist de Funcionalidades Validadas	1
5. Conclusão do Teste	1
Otimização de Consumo de Energia (R1.3)	1
Etapa 3 Semana 3	1
Avaliação de Métodos de Estimativa de Distância via RSSI BLE	1
1. Introdução e Contexto	1
2. Metodologia e Explicação dos Códigos	1
Calibração (calibra_measured_power.py)	1
Coleta e Filtragem	1
ble_site_survey_gui.py	1
ble_site_survey_gui_filter.py	1
3. Resultados e Análise Comparativa	1
4. Conclusão	1
Falha na Indicação da Carga da Bateria	1
Desenvolvimento da placa de circuito impresso	1
Etapa 3 Semana 4	1
1. Estado do Sistema — Baseline “Beta Final”	1
2. Firmware — Arquitetura e Decisões Técnicas	1
3. Aplicativo	1
4. Testes	1
5. Métricas da Etapa 3	1
6. Demonstração Interna — 05/jan/2026	1
7. Melhorias Prioritárias	1
8. TRL Esperado	1
9. Registro da Versão Beta	1
Etapa 4 Semana 1	1
1. Comparação Requisitos x Estado Atual	
Itens marcados como Deprecated (DEP) são considerados fora do escopo atual.	1
Requisitos de Sistema (SYS)	1
Requisitos Não Funcionais (SYS-NF)	1
Requisitos de Software (SW)	1
2. Ajustes necessários	1
3. TRL	1
Etapa 4 Semana 2	1
Relatório de Teste Unitário — Módulo BLE	1
Visão Geral	1
1. Objetivo	1
2. Ambiente de Teste	1

3. Escopo dos Testes	1
4. Metodologia	1
5. Resultados	1
Motivos dos casos parciais	1
6. Próximos Passos	1
Etapla 4 Semana 3	1
1. Aplicativo (Antônio)	1
a. Diagrama de Camadas	1
b. Diagrama de Blocos	1
c. Diagrama de Contexto	1
d. Manual de uso do aplicativo	1
a. Diagrama de Camadas	1
Entendendo o Contexto:	1
d. Manual de uso do aplicativo	1
A Experiência do Usuário	1
Tela Inicial: O Ponto de Partida	1
Painel de Controle: A Interação Real	1
2. Firmware (Eric)	1
Diagrama de Camadas do Firmware:	1
1) Aplicação (Firmware)	1
2) GATT (Serviços BLE)	1
3) HAL (Hardware Abstraction Layer)	1
4) Zephyr (RTOS + drivers + stack BLE)	1
5) Config (Kconfig + Devicetree)	1
6) Hardware	1
Diagrama em Blocos do Firmware:	1
1) Cliente BLE (Celular / WebApp)	1
• Atua como Central BLE (cliente).	1
• Estabelece conexão BLE e acessa o GATT Server do dispositivo.	1
• Envia comandos para controle (ex.: buzzer) e lê telemetria (ex.: bateria).	1
• Implementação no projeto (Web Bluetooth): BluetoothService em bluetooth.service.ts.	1
2) Interface BLE no dispositivo (Zephyr BLE Stack)	1
• Implementa GAP/Advertising para descoberta do dispositivo (broadcast do nome + UUIDs).	1
• Implementa GATT Server com serviços e características:	1
○ Battery Service padrão 0x180F e Battery Level 0x2A19.	1
○ Serviço custom do buzzer com escrita (0/1) para ligar/desligar.	1
• Camada HAL responsável pelo fluxo de advertising e callbacks:	1
○ hal_ble_init em ble.c.	1
○ Tipos/callbacks em hal_ble_adv_params_t e callbacks em ble.h.	1
• Configuração principal do BLE em prj.conf (ex.: CONFIG_BT, CONFIG_BT_PERIPHERAL, CONFIG_BT_DEVICE_NAME).	1
3) MCU / SoC (XIAO nRF52840)	1
• É o núcleo do sistema: executa Zephyr RTOS e o firmware.	1

• Conecta-se aos periféricos via:	1
◦ GPIO (LEDs e sinais do carregador/medição),	1
◦ PWM (buzzer),	1
◦ ADC (tensão da bateria).	1
• Aplicação principal e “estado do sistema” em main no arquivo main.c.	1
4) LEDs de status (GPIO)	1
• Finalidade: feedback visual de estado BLE com baixo consumo (duty-cycle baixo).	1
• Pinos (conforme diagrama):	1
◦ LED Verde: P0.30	1
◦ LED Azul: P0.06	1
• Controle via timers do Zephyr:	1
◦ led_azul_timer_handler em main.c.	1
◦ (timer verde análogo) em main.c.	1
• Mapeamento/alias via Devicetree overlay: xiao_ble.overlay.	1
5) Buzzer (PWM)	1
• Finalidade: gerar alerta sonoro.	1
• Interface: saída PWM do nRF52840.	1
• Pino (conforme diagrama): PWM0 / P0.28	1
• Driver/controle no HAL:	1
◦ Frequência alvo: 18 kHz (período ~55.5 µs).	1
◦ Modo intermitente: 20 ms ON / 80 ms OFF (economia).	1
◦ Implementação: hal_buzzer_init, hal_buzzer_set em buzzer.c.	1
6) Bloco de bateria (LiPo) + Gerenciamento/Carregador (BQ24011 ou equivalente)	1
• Bateria LiPo 3.7 V conectada ao bloco de carga/monitoramento (BAT+ / BAT-).	1
• O carregador/monitoramento expõe sinais digitais e permite habilitar leitura/ajustar corrente de carga.	1
Sinais digitais (GPIO) — conforme diagrama	1
• charging-enable (P0.17)	1
◦ Entra no nRF52840 (status de carga, típico “active-low”).	1
◦ HAL agenda processamento fora de ISR via workqueue: charging_callback_handler em battery.c.	1
• read-enable (P0.14)	1
◦ Controla a chave/transistor do divisor de VBAT para reduzir consumo quando não mede.	1
◦ Configurado via nó DT e usado no HAL: battery.c.	1
• charge-speed (P0.13)	1
◦ Seleciona perfil de carga (ex.: lenta/rápida).	1
◦ Controlado pelo HAL de bateria: battery.c.	1
Medição analógica (ADC) — conforme diagrama	1
• adc-channel = AIN7 (no nRF52840 costuma mapear para P0.31).	1
• Leitura via SAADC + filtros (trimmed mean, IIR, rejeição de spikes).	1
• HAL de bateria:	1
◦ Definições via Devicetree (ex.: adc_channel_id, resolução, número de amostras): battery.c.	1

○ Amostragem periódica em workqueue: sample_periodic_handler em battery.c.	1
Configuração via Devicetree	1
● Binding do nó de bateria: xiao-ble-battery.yaml.	1
● Overlay com nó/alias: xiao_ble.overlay.	1
7) Fluxo de dados (setas principais do diagrama)	1
● Celular/WebApp ⇌ BLE (GAP): descoberta e conexão (advertising/conexão).	1
● Celular/WebApp ⇌ GATT Server:	1
○ Lê bateria (Battery Level + telemetria custom se existir no serviço).	1
○ Escreve comando no serviço custom do buzzer (0/1).	1
● Firmware (App) coordena:	1
○ Inicialização do BLE e serviços GATT em main (main.c).	1
○ Indicação de estado via LEDs (GPIO/timers) em main.c.	1
● GATT → HAL:	1
○ Controle do buzzer aciona HAL PWM em buzzer.c.	1
○ Leitura/telemetria de bateria consulta HAL ADC/filtros em battery.c.	1
● HAL → Hardware:	1
○ GPIO/PWM/ADC mapeados por xiao_ble.overlay e habilitados por prj.conf.	1
Manual do Firmware	1
2. Requisitos de Hardware	1
3. Identificação do Dispositivo BLE	1
4. Procedimento de Inicialização	1
5. LEDs de Status	1
6. Conexão Bluetooth	1
7. Serviços BLE Disponíveis	1
7.1 Battery Service (UUID padrão: 0x180F)	1
7.2 Buzzer Service (Customizado)	1
8. Acionamento e Comportamento do Buzzer	1
Características técnicas do buzzer:	1
Segurança	1
9. Monitoramento da Bateria	1
Detalhes de medição:	1
10. Fluxo Básico de Uso	1
11. Dicas de Utilização	1
12. Solução de Problemas	1
O dispositivo não aparece para conexão	1
Conexão instável (conecta e desconecta)	1
Buzzer não emite som	1
13. Configurações Técnicas Importantes	1
5. Hardware (Vitor)	1
a. Diagrama de Blocos	1
b. Limitações	1
c. Instalação e uso do protótipo	1
Etapas 4 Semanas 4	1
Manual do Protótipo	1

1. Introdução	1
2. Visão Geral do Sistema	1
3. Diagrama de Blocos e Descrição do Hardware	1
4. Funcionamento do Sistema	1
5. Limitações do Protótipo	1
6. Instalação e Montagem do Protótipo	1
7. Uso do Protótipo	1

Etapa 1 Semana 1

Perfis de clientes

1 - Tutores de cães domésticos (usuários finais principais)

- **Perfil:** Pessoas que têm cães como animais de estimação, especialmente em **áreas urbanas** ou **semiurbanas**.
 - **Necessidades:**
 - Garantir a **segurança** do animal (evitar fugas e perdas).
 - Acompanhar **localização em tempo real**.
 - **Educar/adestrar** o cão com métodos não agressivos (alerta sonoro).
 - **Benefícios:**
 - Tranquilidade e praticidade para o tutor.
 - Redução de risco de perda ou acidentes.
 - Melhor controle do comportamento e movimentação.
-

2 - Tutores em áreas rurais ou com grandes espaços

- **Perfil:** Pessoas que moram em chácaras, sítios ou fazendas, onde o cão tem liberdade para circular.
 - **Necessidades:**
 - Delimitar **áreas seguras** de circulação.
 - Evitar que o cão entre em regiões perigosas (estradas, plantações, áreas com animais silvestres).
 - Encontrar o animal facilmente em terrenos amplos.
 - **Benefícios:**
 - Controle à distância, mesmo em áreas extensas.
 - Segurança para o animal e para o ambiente.
-

3 - Adestradores profissionais

- **Perfil:** Profissionais de comportamento animal que utilizam ferramentas tecnológicas para treinamento.
- **Necessidades:**
 - Ferramentas de **reforço positivo e estímulos controlados**.
 - Registro de comportamento e movimentação.
 - Tecnologia que facilite o treinamento à distância.

- **Benefícios:**
 - Aprimora a **precisão e consistência** no adestramento.
 - Pode servir como **equipamento auxiliar moderno** em sessões de treinamento.
-

4 - Clínicas veterinárias e pet shops

- **Perfil:** Estabelecimentos que oferecem **hospedagem, passeios ou day care** para cães.
 - **Necessidades:**
 - Monitorar a **localização de vários animais** em tempo real.
 - Evitar que algum cão fuja durante passeios ou hospedagens.
 - **Benefícios:**
 - Aumento da **segurança** operacional.
 - Diferencial competitivo no serviço oferecido ao cliente.
-

5 - ONGs e protetores de animais

- **Perfil:** Organizações e pessoas que cuidam de múltiplos cães, frequentemente soltos em áreas abertas.
 - **Necessidades:**
 - Controlar deslocamento e comportamento de vários animais.
 - Reduzir perdas e acidentes.
 - Acompanhar rotinas e movimentação sem grande infraestrutura.
 - **Benefícios:**
 - Melhora a **gestão de segurança e bem-estar animal**.
 - Permite uma resposta rápida em caso de fuga.
-

6 - Tutores idosos ou com mobilidade reduzida

- **Perfil:** Pessoas que têm dificuldade em correr ou buscar o cão em caso de fuga.
- **Necessidades:**
 - Sistema de **rastreamento fácil de usar**.
 - Alerta automático se o cão se afasta.
- **Benefícios:**
 - **Independência e tranquilidade**.
 - Reduz o esforço físico em emergências.

Diagrama de Contexto

“Amigo Perto-BLE” se relaciona com atores externos e principais componentes.
(não detalha a arquitetura interna, apenas **define fronteiras (domínios), entradas e saídas.**

Atores Externos

1. Tutor / Usuário (App móvel)

- Interage via aplicativo (BLE e interface gráfica).
- Define raio, perfis e comandos (“Encontrar”, “Modo Perdido”).

2. Coleira Inteligente

- Dispositivo embarcado com MCU e rádio BLE.
- Mede RSSI, estima distância e aciona buzzer.

3. Ambiente físico

- Afeta RSSI (interferências, obstáculos, distância).

4. Servidor / Nuvem (opcional futuro)

- Sincronização, firmware update ou histórico de uso (fora do escopo inicial, mas possível evolução).

Sistema em foco: “Coleira Inteligente BLE”

Composto de dois módulos lógicos principais:

- **Módulo Coleira (HW embarcado)**

- MCU + BLE + buzzer + sensores.

- **Módulo App Móvel (SW no celular)**

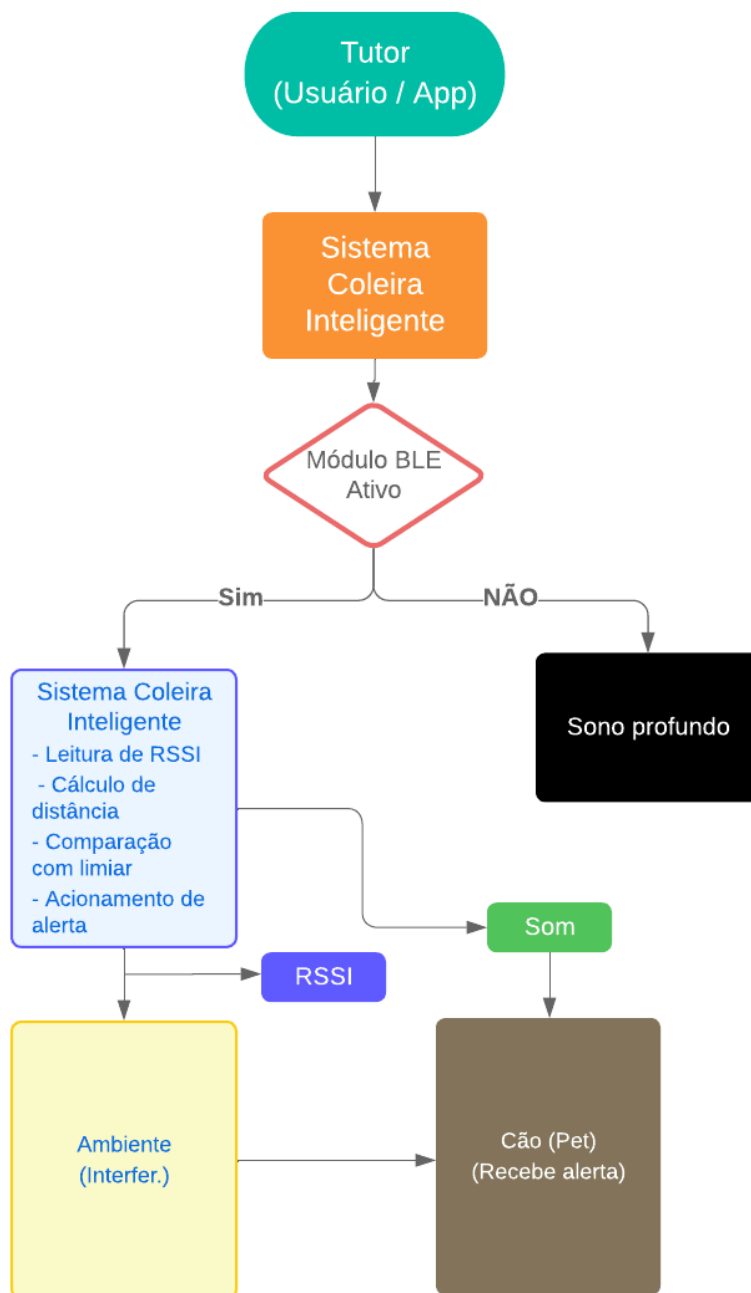
- Interface, configuração e controle via BLE.

Origem	Destino
App → Coleira	Parâmetros de configuração (raio, perfil, comandos)
Coleira → App	RSSI, status de conexão, eventos de alerta
Coleira → Cão	Sinal sonoro de alerta
Ambiente → Coleira	Atenuação do sinal BLE (RSSI)
Tutor ↔ App	Interação de configuração, visualização e controle
(opcional) App ↔ Servidor	Backup, logs, atualizações

Título:

Monitoramento de Proximidade BLE para Cães

Elementos e conexões:



Tabelas - Requisitos Funcionais e Não Funcionais

Requisitos Funcionais

Requisitos de Sistema

ID	Título	Descrição resumida
----	--------	--------------------

SYS-01	Tecnologia de comunicação	Usar BLE 5.x ou superior para comunicação entre coleira e dispositivo do tutor.
SYS-02	Monitoramento de RSSI	Monitorar RSSI continuamente para detectar variações de proximidade.
SYS-03	Estimativa de distância	Estimar distância a partir do RSSI usando modelo de propagação calibrado (ex.: Log-Distance) com filtragem para reduzir ruído.
SYS-04	Comparação com limiar	Comparar distância estimada com limiar configurável e gerenciar estado do sistema (seguro/afastado).
SYS-05	Alerta Físico Local	Acionar alerta físico (beep) quando limiar for excedido.
SYS-06	Estabilidade (Histerese)	Aplicar histerese para evitar alertas repetitivos próximos ao limiar.
SYS-07	Conexão Segura e Exclusiva	Conectar apenas ao dispositivo do tutor previamente emparelhado, ignorando outros sinais BLE.
SYS-08	Configuração via BLE	Permitir ajuste remoto de parâmetros essenciais via comunicação BLE.
SYS-09	Interface do Usuário (App)	Disponibilizar aplicativo móvel funcional para configuração mínima e monitoramento do estado da coleira via BLE.
SYS-10	Dome virtual de proximidade	Criar zona de proximidade configurável, alterar estado do sistema e acionar alerta quando limite for ultrapassado.

Requisitos de Software

ID	Título	Descrição resumida
SW-01	Leitura periódica de RSSI	Realizar leituras periódicas do RSSI para proximidade em tempo real.
SW-02	Algoritmo de Estimativa de Distância	Implementar algoritmo de conversão de RSSI em distância estimada com filtros para reduzir variações e ruídos.
SW-03	Comparação com limiar	Comparar continuamente distância estimada com limiar configurado, gerando eventos internos para alertas.
SW-04	Controle de alerta físico	Gerenciar acionamento do buzzer conforme eventos de ultrapassagem ou retorno ao limite.
SW-05	Implementação de histerese	Aplicar histerese para evitar alertas repetitivos.
SW-06	Autenticação e filtragem de dispositivos	Autenticar dispositivo do tutor durante emparelhamento e ignorar sinais BLE não autorizados.
SW-07	Configuração via BLE	Expor características BLE para ajuste remoto de parâmetros essenciais da coleira.
SW-08	Implementação Interface do Usuário (App)	Disponibilizar app funcional para configuração mínima, monitoramento de proximidade e alertas, podendo ser simplificado.

SW-09	Implementação do Dome Virtual de Proximidade	Implementar lógica do dome virtual: manter raio configurável, comparar distância, alterar estado e acionar alerta físico.
--------------	--	---

Requisitos Não Funcionais

ID	Título	Descrição resumida
SYS-NF01	Fonte de energia compacta	O sistema deve utilizar uma fonte de alimentação de dimensões reduzidas.
SYS-NF02	Eficiência energética	Minimizar consumo para operação contínua com fonte restrita.
SYS-NF03	Compacidade e leveza	Design compacto e leve para uso permanente na coleira.
SYS-NF04	Sinalização segura	Alerta perceptível sem causar incômodo ou risco ao PET.
SYS-NF05	Robustez BLE	Comunicação BLE estável e resiliente a interferências.
SYS-NF06	Evolução de TRL	Avançar de protótipo para hardware dedicado com testes documentados.
SYS-NF07	PCB dedicada	Empregar PCB própria, otimizada para espaço, peso e RF.

SYS-NF08	Ergonomia e segurança	Materiais e acabamentos seguros e confortáveis para o PET.
SYS-NF09	Confiabilidade ambiental	Operar em condições externas típicas (temperatura, umidade, respingos).
SYS-NF10	Usabilidade do app	App simples, intuitivo e fácil de usar, mesmo com interface mínima via BLE.
SYS-NF11	Plataforma dedicada	Não usar BitDogLab; adotar microcontrolador + rádio BLE em PCB própria.

TRL

Nível atual

TRL 2 - Estamos na etapa de pesquisa aplicada: já definimos a aplicação, estamos estudando o hardware e o perfil do cliente, e elaborando os requisitos iniciais.

Nível a ser atingido

TRL 5 - A expectativa é de termos ao final do projeto um protótipo que apresente os requisitos iniciais testados em um ambiente controlado.

Etapa 1 Semana 2

Requisitos Consolidados

Requisitos Funcionais

Requisitos de Sistema

[SYS-01] Tecnologia de comunicação

O sistema deve utilizar Bluetooth Low Energy (BLE) versão 5.x ou superior como protocolo de comunicação entre a coleira e o dispositivo do tutor.

Rationale: O BLE é essencial para atingir os objetivos de autonomia, alcance, baixo consumo de energia e interoperabilidade com os dispositivos móveis atuais.

[SYS-02] Monitoramento de RSSI

O sistema deve monitorar continuamente a intensidade do sinal BLE (RSSI) entre a coleira e o dispositivo do tutor, com taxa de amostragem suficiente para detectar variações de proximidade.

Rationale: O monitoramento contínuo do RSSI é essencial para identificar mudanças de distância e sustentar a lógica de estimativa e alerta.

[SYS-03] Estimativa de distância

O sistema deve estimar a distância entre os dispositivos a partir do valor do RSSI, utilizando um modelo de propagação de sinal calibrado e eficiente. Devem ser aplicadas técnicas de filtragem para reduzir ruídos e variações nas medições, assegurando maior estabilidade e precisão nos resultados.

Rationale: A estimativa de distância a partir do RSSI é essencial para converter a intensidade do sinal em uma métrica física (metros), viabilizando a definição de zonas de alcance e limites de segurança. O uso de um modelo de propagação calibrado e filtros adequados melhora significativamente a confiabilidade das estimativas, tornando o sistema mais robusto frente às variações do ambiente.

[SYS-04] Comparação com limiar configurado

O sistema deve comparar a distância estimada com um limiar de proximidade configurável e gerenciar o estado do sistema (seguro/afastado) conforme essa comparação.

Rationale: Essa lógica permite identificar quando o animal ultrapassa o raio seguro definido, funcionando como base para o acionamento de alertas.

[SYS-05] Alerta Físico Local

Quando o limiar for excedido, o sistema deve acionar um alerta físico sonoro (beep) na coleira, cessando automaticamente quando o animal retornar à zona segura.

Rationale: O alerta físico local é o principal mecanismo de aviso, permitindo resposta imediata e intuitiva, sem depender de conectividade adicional.

[SYS-06] Estabilidade (Histerese)

O sistema deve aplicar histerese no processo de decisão para evitar o acionamento repetitivo de alertas quando o animal oscila na região próxima ao limiar configurado.

Rationale: A histerese melhora a estabilidade e reduz falsos alarmes, evitando desconforto para o animal e irritação para o usuário.

[SYS-07] Conexão Segura e Exclusiva

O sistema deve estabelecer conexão BLE apenas com o dispositivo do tutor previamente emparelhado, ignorando sinais de outros dispositivos Bluetooth.

Rationale: Garante que a lógica de proximidade e alertas seja baseada somente no dispositivo autorizado, evitando interferências e problemas de segurança.

[SYS-08] Configuração via BLE

O sistema deve permitir ajuste remoto de parâmetros essenciais (como o raio seguro e intensidade do alerta) por meio da comunicação BLE com o dispositivo do tutor.

Rationale: A configuração sem fio aumenta a praticidade e evita ajustes físicos na coleira, simplificando o uso e a calibração.

[SYS-09] Interface do Usuário (App)

O sistema deve disponibilizar um aplicativo móvel que permita ao usuário configurar os parâmetros essenciais da coleira e acompanhar o estado de conexão e proximidade. O aplicativo deve se comunicar com o dispositivo por meio de BLE, podendo apresentar

interface simples ou funcionalidade mínima necessária para configuração e monitoramento.

Rationale: Um aplicativo móvel é fundamental para a usabilidade do sistema, pois permite ajustar parâmetros e visualizar informações sem necessidade de interação física direta com a coleira. Manter o aplicativo leve e modular garante flexibilidade para evolução futura sem comprometer o funcionamento básico.

[SYS-10] Dome virtual de proximidade

O sistema deve estabelecer um raio de proximidade configurável, definindo um “dome virtual” ao redor do dispositivo do tutor. Quando esse limite for ultrapassado, o sistema deve alterar seu estado e acionar o alerta local.

Rationale: Formaliza a zona de segurança como comportamento funcional do sistema, assegurando resposta determinística e previsível ao afastamento.

Requisitos de Software

[SW-01] Leitura periódica de RSSI

O software deve realizar leituras periódicas do RSSI durante a conexão BLE, com frequência mínima suficiente para permitir estimativa de proximidade em tempo real.

Rationale: Sem medições periódicas, não é possível calcular distância nem acionar alertas de forma responsiva.

[SW-02] Algoritmo de Estimativa de Distância

O software deve implementar um algoritmo para converter RSSI em distância estimada, aplicando filtragem adequada para suavizar variações de sinal.

Rationale: O RSSI é naturalmente instável; um modelo calibrado e filtragem são necessários para produzir estimativas consistentes.

[SW-03] Lógica de comparação com limiar

O software deve comparar continuamente a distância estimada com o limiar configurado, determinando o momento em que o limite é ultrapassado e gerando eventos internos para acionar alertas.

Rationale: Essa lógica é o núcleo funcional do sistema de proximidade, permitindo identificar o estado “dentro” ou “fora” da zona segura.

[SW-04] Controle de alerta físico

O software deve gerenciar o acionamento do buzzer conforme os eventos de ultrapassagem e retorno ao limite configurado.

Rationale: O alerta físico é a resposta imediata para indicar afastamento e ajudar na localização do PET. O controle preciso do alerta garante feedback imediato e reduz acionamentos desnecessários.

[SW-05] Implementação de histerese

O software deve aplicar histerese na lógica de comparação, prevenindo alertas intermitentes quando o RSSI varia próximo ao limiar.

Rationale: Reduz “chattering”. A histerese evita instabilidade e melhora a experiência do usuário e do PET.

[SW-06] Autenticação e Filtragem de Dispositivos

O software deve autenticar o dispositivo do tutor durante o emparelhamento BLE e ignorar sinais RSSI de dispositivos não autorizados.

Rationale: Evita interferências externas e mantém a confiabilidade e segurança do sistema.

[SW-07] Configuração via BLE

O software deve expor características BLE configuráveis que permitam ajustar remotamente os parâmetros principais, como o limiar de distância e intensidade do alerta.

Rationale: Permite calibração simples via interface BLE, sem necessidade de aplicativo complexo.

[SW-08] Interface do Usuário (App)

O software deve disponibilizar um aplicativo móvel funcional que permita ao usuário:

- Configurar parâmetros essenciais da coleira (como raio de proximidade e intensidade do alerta);
- Visualizar informações básicas de estado, incluindo conexão BLE, proximidade e alertas;
- Receber notificações e sinais de alerta do dispositivo.

Podendo ser implementado com interface gráfica mínima ou uma versão simplificada que execute as funções essenciais.

Rationale: O aplicativo móvel garante usabilidade e controle remoto do dispositivo, permitindo operação e monitoramento sem interação física direta. Manter o requisito flexível permite que a interface seja simples ou evolua no futuro, sem comprometer o funcionamento do principal do sistema.

[SW-09] Implementação do Dome Virtual de Proximidade

O software deve implementar a lógica de dome virtual, responsável por:

- Manter o raio de proximidade configurável ao redor do dispositivo do tutor;
- Comparar continuamente a distância estimada com o limite configurado;
- Alterar o estado do sistema (seguro / afastado) quando o limite é ultrapassado;
- Acionar o alerta físico local conforme o estado.

A implementação deve ser modular, permitindo ajustes futuros no cálculo da distância, limites ou comportamento do alerta, sem depender de interfaces adicionais ou modos avançados.

Rationale: O dome virtual formaliza a zona de segurança da coleira, garantindo que a transição de estado e o acionamento do alerta sejam determinísticos e previsíveis, assegurando a função principal de manter o animal dentro do perímetro seguro.

Requisitos Não Funcionais

[SYS-NF01] Fonte de alimentação compacta

O sistema deve utilizar uma fonte de alimentação de dimensões reduzidas (ex.: bateria tipo moeda, Li-Po de pequeno porte ou equivalente), garantindo operação contínua dentro das especificações de autonomia e funcionalidade definidas.

Rationale: Como o sistema será acoplado à coleira do animal, a fonte de energia deve ser pequena e leve, de modo a não causar desconforto ou restringir seus movimentos. A adoção de uma bateria compacta também reduz peso e facilita a manutenção, porém impõe limitações de capacidade e corrente, exigindo otimização do hardware e do firmware para baixo consumo e eficiência energética.

[SYS-NF02] Eficiência energética

A arquitetura do sistema deve minimizar consumo para viabilizar operação contínua com embalagem compacta e fonte de energia restrita.

Rationale: Baixo consumo é crítico para autonomia e viabilidade de uso diário.

[SYS-NF03] Compacidade e leveza

O sistema deve ser compacto e leve, adequado ao uso permanente na coleira.

Rationale: Dimensão e massa impactam diretamente no conforto, aceitação e segurança do animal.

[SYS-NF04] Sinalização embarcada perceptível e segura

O sistema de alerta (beep) deve ser claramente perceptível para o tutor, sem causar incômodo excessivo ou risco ao animal.

Rationale: A sinalização deve cumprir o objetivo de alerta sem comprometer o bem-estar do PET.

[SYS-NF05] Robustez da comunicação BLE

A comunicação BLE deve ser estável e resiliente a interferências, minimizando falsos alarmes por oscilações de link e perdas de conexão.

Rationale: Robustez do enlace é essencial para a confiabilidade do sistema de proximidade.

[SYS-NF06] Evolução de TRL

O desenvolvimento deve progredir de protótipo em placa para hardware dedicado, com

documentação e evidências de teste em cada fase, até atingir o nível de prontidão tecnológica definido pelo projeto. (TRL a ser atingida: 5)

Rationale: A evolução do TRL garante maturidade técnica, reduz riscos e prepara a solução para uso real.

[SYS-NF07] PCB dedicada ao formato de coleira

A solução deve empregar PCB própria, considerando restrições de espaço, peso e robustez, além de boas práticas de RF e manufatura.

Rationale: Uma PCB dedicada garante não apenas o desempenho ideal da antena e a integridade mecânica, mas também permite utilizar apenas os componentes necessários, resultando em um design mais compacto, leve e eficiente.

[SYS-NF08] Ergonomia e segurança animal

Materiais, acabamentos do sistema devem priorizar conforto e segurança, evitando arestas, aquecimento local e pontos de pressão.

Rationale: O bem-estar do PET é requisito básico para adoção do produto.

[SYS-NF09] Confiabilidade ambiental

O sistema deve operar de forma confiável em condições típicas externas (variações de temperatura, umidade, respingos).

Rationale: Uso em ambientes reais exige resistência a condições ambientais comuns.

[SYS-NF10] Usabilidade

O aplicativo móvel deve ser simples, intuitivo e de fácil operação, permitindo que o usuário configure os parâmetros essenciais da coleira e monitore seu estado sem dificuldades. A interface deve priorizar clareza e eficiência, independentemente de ter front-end gráfico completo ou implementação mínima via BLE.

Rationale: Uma interface de fácil utilização reduz erros operacionais e agiliza a configuração, mesmo que a aplicação seja funcionalmente mínima.

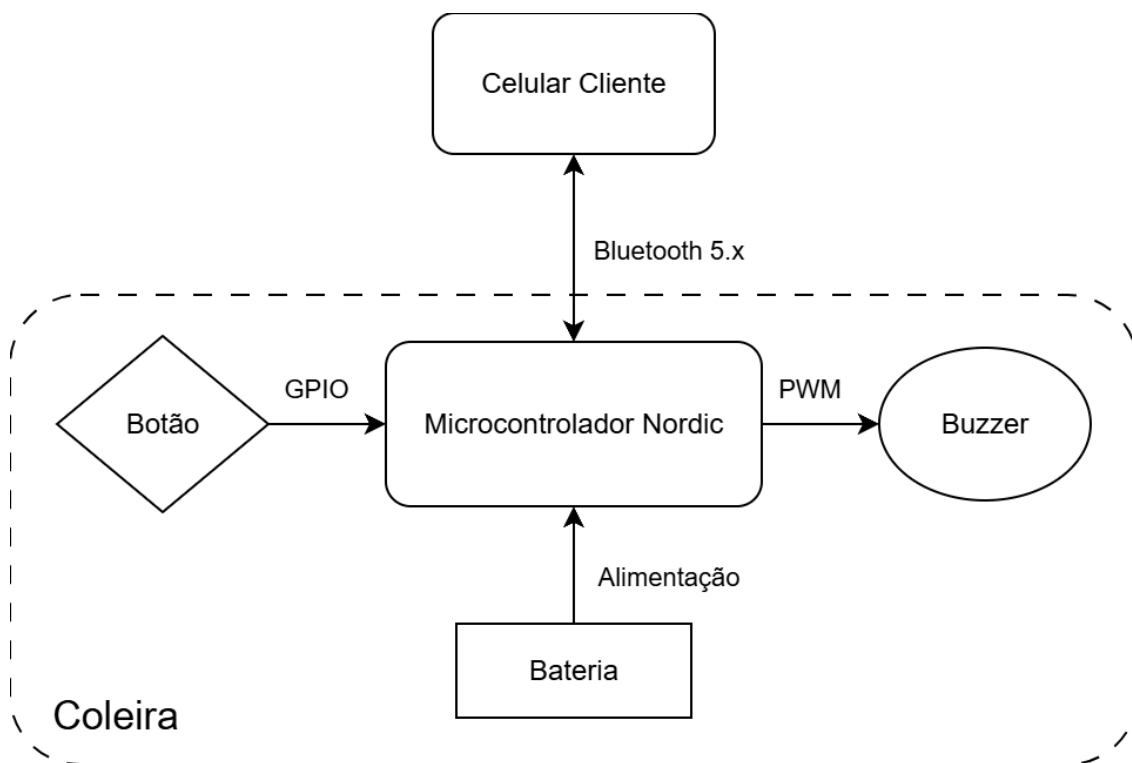
[SYS-NF11] Plataforma de hardware dedicada

O sistema não deve utilizar a placa de desenvolvimento BitDogLab, mas sim um microcontrolador e um rádio BLE adequados à aplicação, integrados em uma PCB própria.

Rationale: O uso de hardware dedicado garante otimização de consumo, redução de tamanho e peso, uma melhor qualidade do *Bluetooth*, além de maior confiabilidade e viabilidade atendendo às restrições do formato de coleira.

Diagrama de Blocos

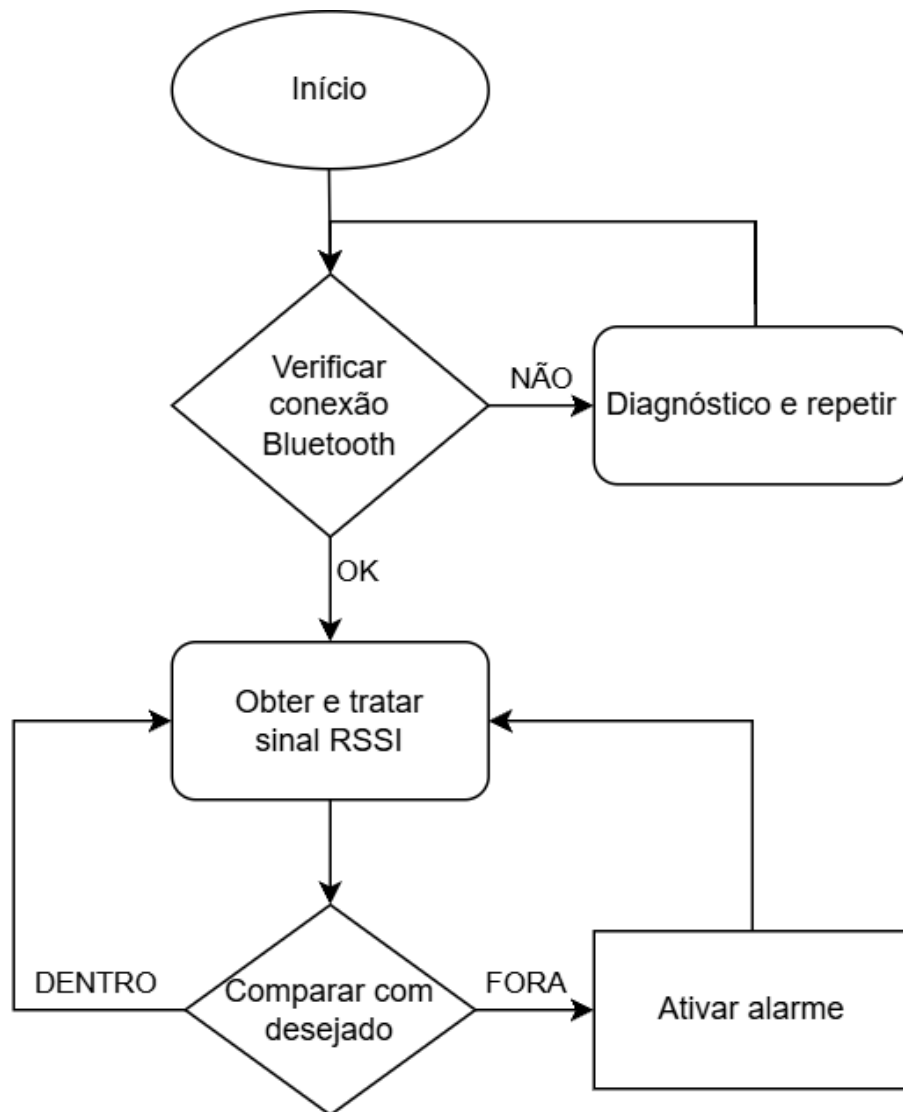
Diagrama de Hardware



- O celular do cliente tem papel fundamental no sistema, ele que se comunicará com o dispositivo que estará junto ao cachorro.
- O microcontrolador é a peça central do sistema. O número de modelo ainda será definido, mas sabemos que será de um fabricante da Nordic, referência em microcontroladores com Bluetooth. Ele será responsável por receber o sinal RSSI, gerenciar a conexão Bluetooth e acionar o alarme quando o limiar for ultrapassado.

- O botão servirá para ações do usuário, como parear o celular com a coleira e talvez ligar/desligar o dispositivo, ações a serem definidas.
- O buzzer servirá como alarme, tocará quando o cachorro estiver mais longe do que o dono determinou que ele deveria.
- A bateria servirá para alimentar toda a parte do sistema presente na coleira, o objetivo é usar uma bateria pequena e que consiga entregar boa longevidade para o sistema.

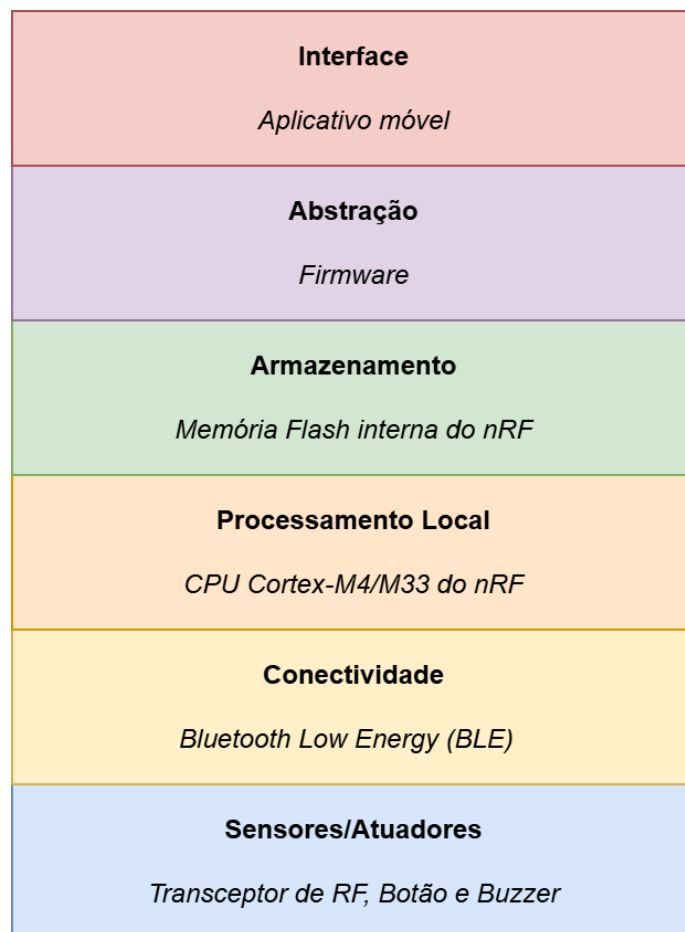
Diagrama de Software

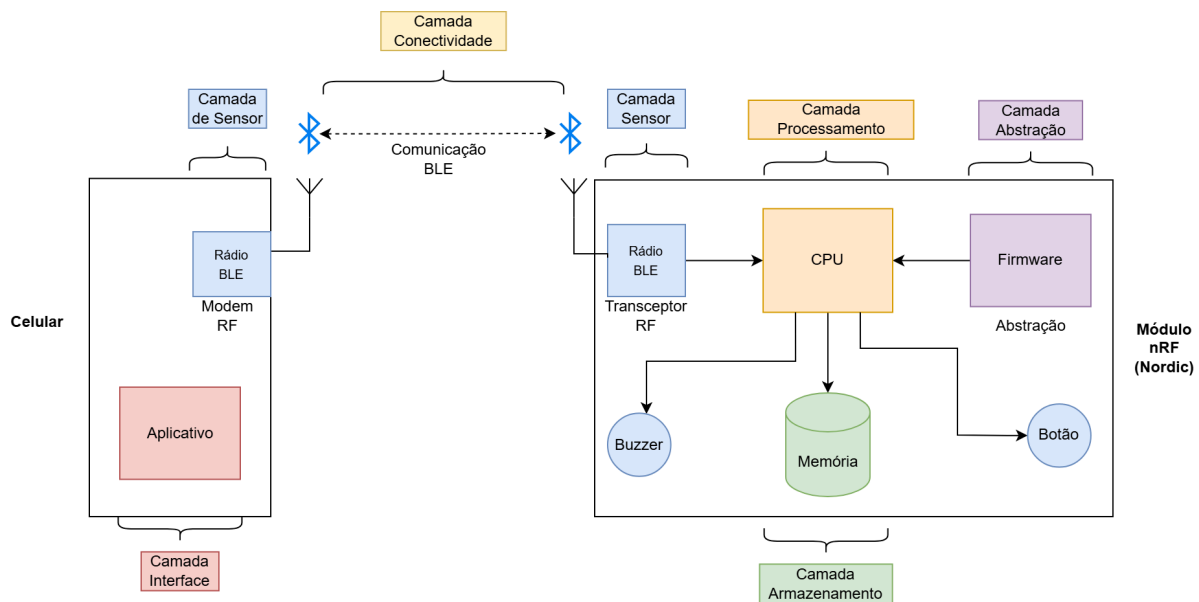


Ao iniciar o sistema, ele verificará se tem condições de conectar ao bluetooth da coleira, caso já exista algum pareamento anterior, caso seja a primeira vez, será necessário configurar. Caso consiga comunicar, segue o fluxo, do contrário, gera um diagnóstico de possíveis erros e tenta corrigi-los, após isso, tenta comunicar novamente e fica nesse loop até obter sucesso.

Com o pareamento concluído, o sinal RSSI obtido deve passar por uma série de tratamentos, como filtros e histerese para determinar qual distância está e quando alarmar. O alarme deve ser acionado quando tiver certeza de que a coleira está mais distante que o determinado pelo cliente, caso esteja dentro do dome setado, nada deve ser realizado. E assim se repete enquanto o sistema estiver ativo, verifica a conexão bluetooth, checa a distância e age.

Diagrama de camadas





1) Camada de Sensores/Atuadores

Função: Interação física com o ambiente (entrada e saída).

Componentes:

- **Buzzer (atuador)** → emite o beep de alerta quando o pet ultrapassa o limite de distância.
- **Botão (sensor)** → Servirá para ações do usuário
- **Transceptor de RF (sensor)** → O transceptor de RF (módulo responsável por TX/RX de pacotes BLE) possui internamente um circuito de detecção de potência de sinal. Esse circuito analógico mede a intensidade do sinal recebido (RSSI).

Responsável por:

- Executar a resposta física ao evento de afastamento e ações do usuário.
- Receber intensidade do sinal recebido via transceptor de RF.

Requisitos relacionados:

- [SYS-02]: Monitoramento de RSSI
- [SYS-05]: Alerta Físico Local
- [SW-01]: Leitura periódica de RSSI
- [SW-04]: Controle de alerta físico

2) Camada de Conectividade

Função: Comunicação sem fio entre a coleira e o smartphone do tutor.

Componentes:

- **Rádio BLE 5.x** embutido no SoC **nRF da Nordic** (ex.: nRF52832 / nRF52840 / nRF54L15).
- **Antena integrada** (PCB antenna ou chip antenna).

Responsável por:

- Transmitir/receber pacotes BLE.
- Medir e disponibilizar o **RSSI** (via hardware do transceptor).
- Garantir conexão segura (autenticação e emparelhamento).

Requisitos relacionados:

- [SYS-01]: Tecnologia de comunicação
 - [SYS-07]: Conexão Segura e Exclusiva
-

3) Camada de Processamento Local

Função: Processamento dos dados e lógica de decisão.

Componente principal: CPU Cortex-M4/M33 do nRF.

Tarefas do firmware:

- Leitura periódica do RSSI.
- Cálculo da distância estimada.
- Aplicação de filtros.
- Comparação da distância com o limiar configurado.
- Implementação de histerese.

- Acionamento e desligamento do buzzer.

Requisitos relacionados:

- [SYS-02]: Monitoramento de RSSI
 - [SYS-03]: Estimativa de distância
 - [SYS-04]: Comparação com limiar
 - [SYS-05]: Alerta Físico Local
 - [SYS-06]: Estabilidade (Histerese)
 - [SW-01]: Leitura periódica de RSSI
 - [SW-02]: Algoritmo de Estimativa de Distância
 - [SW-03]: Comparação com limiar
 - [SW-04]: Controle de alerta físico
 - [SW-05]: Implementação de histerese
-

4) Camada de Armazenamento

Função: Guardar parâmetros e dados essenciais.

Componentes: Memória Flash interna do nRF (ex.: 512 KB – 1 MB)

Dados armazenados:

- Limiar configurado (raio de proximidade).
- Intensidade do alerta.
- Dados de calibração do modelo de distância.
- Identificador do tutor emparelhado.

Requisitos relacionados:

- [SW-02]: Algoritmo de Estimativa de Distância
 - [SW-03]: Comparação com limiar
 - [SW-04]: Controle de alerta físico
 - [SW-05]: Implementação de histerese
 - [SW-06]: Autenticação e filtragem de dispositivos
-

5) Camada de Abstração

Função: interface entre hardware e software.

Componentes / Software:

- HAL (Hardware Abstraction Layer) da Nordic SDK.
- Drivers BLE, GPIO, PWM, Timer.
- Middleware BLE Stack (Zephyr BLE stack).

Responsável por:

- Esconder detalhes do hardware.
- Oferecer APIs seguras e padronizadas para as camadas superiores.
- Implementar callbacks e eventos BLE (conexão, leitura RSSI, etc.)

Requisitos relacionados:

- [SW-02]: Algoritmo de Estimativa de Distância
 - [SW-03]: Comparação com limiar
 - [SW-04]: Controle de alerta físico
 - [SW-05]: Implementação de histerese
 - [SW-06]: Autenticação e filtragem de dispositivos
 - [SW-07]: Configuração via BLE
 - [SW-09]: Implementação do Dome Virtual de Proximidade
-

6) Camada de Interface

Função: Interação com o usuário (tutor do animal).

Componentes: Aplicativo móvel BLE

Responsável por:

- Comunicação BLE com a coleira
- Envio de comandos de configuração.
- Exibir status de conexão e RSSI.

- Mostrar estado da coleira (seguro / afastado).

Requisitos relacionados:

- [SYS-09]: Interface do Usuário (App)
- [SW-08]: Implementação Interface do Usuário

Interfaces e Dependências principais

Bluetooth Low Energy (BLE):

Um *periférico* (coleira) anuncia sinais BLE, enquanto uma *central* (celular) mede a intensidade do sinal (RSSI) para estimar a distância e acionar alertas quando o cão se afastar além do limite.

As interfaces e dependências principais devem, neste estágio do projeto, ser mantidas de forma modular, preservando uma arquitetura genérica que permita evoluções futuras conforme a escolha do hardware (por exemplo: nRF54L15, XIAO BLE, ESP32, etc.).

Arquitetura de Alto Nível

Componentes principais:

Camada	Papel	Implementação esperada
Dispositivo BLE (coleira)	Anuncia <i>advertisements BLE</i> com ID único e, opcionalmente, GATT para status/configuração	nRF54L15 / Zephyr / C SDK, Bare Metal, etc.
Aplicativo Móvel (tutor)	Detecta o beacon BLE, mede RSSI, calcula distância e dispara alertas caso necessário	Android (Java/Kotlin)

Interfaces principais entre os componentes

Interface 1: BLE Advertising (Coleira → Celular)

Função: transmitir dados mínimos e frequentes.

Campo	Tipo	Descrição
-------	------	-----------

UUID (ex: 0x180F)	16-bit	Serviço base
ID do pet	4 bytes	Identificador único
Nível de bateria	1 byte	0–100%
Estado (normal/alerta)	1 byte	Status da coleira
TX Power	1 byte	Para calibrar distância
RSSI (medido pelo celular)	—	usado apenas pelo central

Essa interface é **somente broadcasting. O celular apenas escuta e mede RSSI.

Interface 2: BLE GATT (opcional)

Função: configuração ou diagnóstico via BLE (quando o celular conecta manualmente).

Serviço	UUID	Características	Ação
AmigoPerto Service	128-bit custom	BuzzerControl, BatteryLevel, DeviceID, FirmwareVersion	Leitura/escrita BLE
DFU / OTA Service	padrão Nordic	Atualização de firmware via app é uma funcionalidade já inclusa na placa pela fabricante (opcional)	OTA update

**Esta interface só é usada quando o app “conecta” ao dispositivo — para configurar distância, atualizar firmware, etc.

Dependências principais por camada

(A placa sugerida pode ser alterada; contudo, enquanto o fabricante permanecer o mesmo, os requisitos se mantêm. Caso se opte por outro fabricante, será necessária uma nova avaliação do sistema operacional, da IDE e das bibliotecas e drivers necessários.)

Coleira (nRF54L15 / Zephyr / Bare Metal)

- zephyr/bluetooth/bluetooth.h
- zephyr/bluetooth/hci.h
- zephyr/bluetooth/gatt.h
- drivers/gpio.h (para buzzer)
- drivers/sensor.h (opcional – bateria)
- device.h / pm.h (baixo consumo)

Dependências (build):

- nRF Connect SDK com Zephyr RTOS
(as linhas **nRF52**, **nRF53** e **nRF54** são suportadas – porém, **apenas a nRF54** oferece também um ambiente *bare metal*, permitindo desenvolver sem RTOS, para aplicações que exigem latência mínima ou uso muito restrito de recursos)
https://devzone.nordicsemi.com/nordic/nordic-blog/b/blog/posts/a-technical-div-e-into-the-nrf-connect-sdk-bare-metal-option?utm_campaign=Product%20Update%20Notifications%20-%20PUN&utm_medium=email&_hsenc=p2ANqtz--TffQVOcyTulroNpXDhY9aHm1UqySXID6ayAnrSnyAuNY9lislU5Qpi1USCG_Ze6z5lOq-ITJ0NQ2jjGnpdPH1LJdV7f6O9vpqClnRXTc8tbj-zCY&_hsmi=383875339&utm_content=383876137&utm_source=hs_email
 - west, cmake, ninja
 - bt_enable(), bt_le_adv_start()
-

Aplicativo Android (lib's nativas)

- BluetoothLeScanner, ScanCallback, ScanResult
- BluetoothGatt (para conexões)
- RSSI e TxPowerLevel para cálculo da distância
- Dependências:
 - Android SDK 30+ ou superior
 - androidx.bluetooth (Jetpack)
 - LiveData / Room (armazenamento local)
 - OkHttp / Retrofit (para API)

Estrutura sugerida do projeto

```

amigo-perto/
├── firmware/
│   ├── prj.conf
│   ├── main.c
│   └── CMakeLists.txt
├── mobile-app/
│   ├── app/src/main/java/com/amigoperto/
│   │   ├── ble/
│   │   ├── ui/
│   │   └── data/
│   └── build.gradle
└── docs/
    └── architecture-diagram.drawio

```

Eventos e fluxos principais

Evento	Origem	Destino	Ação
Advertising detectado	Coleira	App	Calcular RSSI
RSSI < threshold	App	Buzzer (via BLE write)	Ativar som
RSSI > threshold	App	Buzzer	Desligar
Alerta persistente	App	Durante distanciamento identificado	Registrar alerta
Nova config distância	App	Coleira	Atualizar GATT value

Uso de LE Codec

LE 1M, LE 2M e LE Coded (explicado intuitivamente)

PHY	Velocidade	Alcance	Uso típico
LE 1M	Médio (1 Mb/s)	Padrão (~10 m)	Conexões gerais BLE

LE 2M	Alto (2 Mb/s)	Curto	Transferência rápida de dados, p.ex. firmware update
LE Coded	Baixo (125–500 kb/s)	Longo (~100 m ou mais)	Sensores distantes, comunicação IoT de longo alcance

O **LE Coded** usa **Forward Error Correction (FEC)** e **redundância** para permitir comunicação mesmo com sinais fracos — ideal para sensores em campo.

A stack BLE (exemplo: **nRF Connect SDK**, **Zephyr**, ou **Android Bluetooth API**)
métricas derivadas e interpretadas:

RSSI	<code>bt_le_conn_cb.rssi_changed</code> (Zephyr) / <code>BluetoothGatt.readRemoteRssi()</code> (Android)	Força do sinal recebido (dBm)	Média do envelope do sinal detectado
IQ samples	<code>struct bt_df_iq_samples_report</code> (Zephyr)	Vetores complexos I/Q capturados em pacotes CTE (Constant Tone Extension)	Fase/amplitude processadas por hardware
AoA / AoD Angles	SDK da Nordic: <code>nrf_dm_calculate_angle()</code> ou API Zephyr Direction Finding	Ângulo estimado de chegada/partida (graus)	Calculado a partir de IQ samples
Distance / Ranging	<code>nrf_dm_get_distance()</code> (Bluetooth Channel Sounding)	Estimativa de distância (em metros)	Calculada com base em tempo e fase
CTE (Constant Tone Extension)	Configuração via <code>bt_df_set_cte_rx_param()</code>	Período usado para medir fase contínua	Gera os dados IQ

Sugestão de MVP

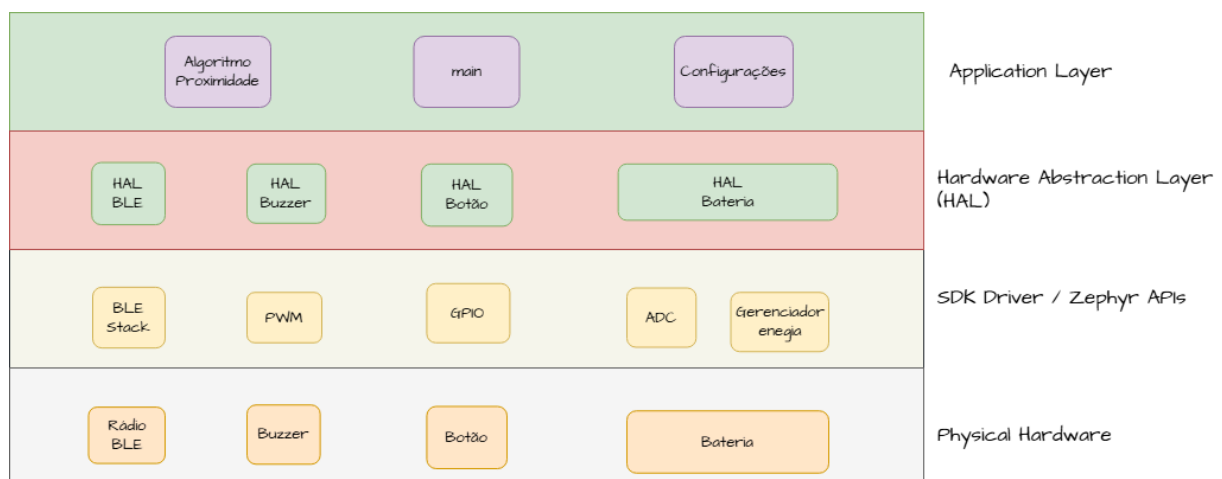
1. Firmware BLE simples anunciando ID e TX Power
2. App Android detectando advertising e medindo RSSI
3. Buzzer aciona na coleira
4. (Opcional) Escrever para GATT → já ativa buzzer
5. (Opcional) Gráfico básico de distância em tempo real.

Etapa 1 Semana 3

Plano de Abstração (HAL)

O objetivo da HAL (Hardware Abstraction Layer) é isolar o firmware de aplicação das particularidades do hardware e do SDK, facilitando testes e simulações (mocks), além de permitir a substituição do hardware ou da pilha de software sem a necessidade de reescrever a lógica de alto nível. Essa camada garante o atendimento aos requisitos funcionais e não funcionais do projeto. Para isso, é essencial primeiramente ter uma visão geral da arquitetura de software:

Arquitetura de Software



1. Application Layer

Componentes:

- **main**
 - Inicializa o sistema, configura os periféricos, o BLE e inicia as tarefas (Zephyr) ou o *super loop* no bare-metal.
 - Faz o link entre as camadas HAL e o algoritmo de proximidade.

- Cumpre requisitos [SW-01] a [SW-04], inicializando o loop principal e os módulos de leitura RSSI, estimativa de distância e controle do buzzer.
- **Algoritmo de Proximidade**
 - Implementa:
 - Leitura periódica do RSSI ([SW-01])
 - Estimativa de distância via modelo calibrado + filtragem ([SW-02])
 - Comparação com o limiar e histerese ([SW-03], [SW-05])
 - Acionamento do alerta sonoro ([SW-04])
 - Define a lógica do “**dome virtual de proximidade**” ([SW-09], [SYS-10]).
 - Essa lógica será executada ciclicamente, atualizando o estado (“seguro” / “afastado”).
- **Configurações**
 - Gerenciar os parâmetros de operação da coleira, incluindo configurações locais e remotas via BLE (Advertising ou GATT).
 - Inicialização e carregamento de parâmetros
 - Atualização dinâmica via BLE
 - Salvamento em memória não volátil
 - Permite alterar parâmetros como:
 - Limiar de distância
 - Intensidade do alerta
 - Intervalo de leitura RSSI
 - **BLE Advertising:** transmite parâmetros básicos periodicamente;
 - **BLE GATT (opcional):** permite leitura/escrita remota conforme [SW-07] e [SYS-08].

2. Hardware Abstraction Layer (HAL)

Camada intermediária que abstrai o acesso direto ao hardware, expondo APIs simples e independentes do driver específico.

Módulos:

- **HAL BLE**

- Encapsula as funções de comunicação BLE (advertising, conexão, leitura de RSSI, GATT).
- Permite que a camada de aplicação apenas solicite “obter RSSI” ou “atualizar parâmetro” sem conhecer o stack BLE.
- Implementa segurança de emparelhamento[SYS-07], [SW-06]).

- **HAL Buzzer**

- Controla o PWM ou GPIO responsável pelo som.
- Oferece funções simples como buzzer_on(intensidade) e buzzer_off().
- Relaciona-se com [SYS-05], [SYS-NF04].

- **HAL Botão**

- Permite entrada física (opcional) para reset, pareamento ou modo de configuração.
- Usa GPIOs via Zephyr drivers.

- **HAL Bateria**

- Lê a tensão via ADC, gerencia estados de carga e fornece informação de nível de bateria para o app.
- Cumpre [SYS-NF01], [SYS-NF02].

- ◆ **Função principal da HAL:**

Permitir que a camada de aplicação interaja com os periféricos **sem depender das APIs específicas do Zephyr ou Nordic SDK**, facilitando manutenção e portabilidade.

3. SDK Driver / Zephyr APIs

Camada fornecida pelo Zephyr e pelo nRF Connect SDK.

Principais interfaces:

- **BLE Stack (Zephyr BT)**

- Implementa o protocolo BLE 5.x (advertising, GATT, RSSI).

- Cumpre [SYS-01], [SYS-07], [SYS-08], [SYS-09].
- **PWM Driver**
 - Controla o buzzer.
 - Fornece interface para `HAL_Buzzer`.
- **GPIO Driver**
 - Lida com entradas e saídas digitais (botões).
- **ADC Driver**
 - Faz leitura da bateria, utilizada em `HAL_Bateria`.
- **Gerenciador de Energia (Zephyr Power Management)**
 - Controla modos *sleep*, *idle*, *active* para reduzir consumo[SYS-NF02]).

Esses drivers são utilizados pela HAL e, no caso do uso do Zephyr RTOS, são gerenciados pelo sistema operacional, que cuida do task scheduling, event handling e do baixo consumo de energia. No caso de uma implementação bare-metal, não há escalonador de tarefas, o controle é feito pelo super loop principal e por interrupções. A temporização baseia-se em timers de hardware, e o gerenciamento de energia e eventos é responsabilidade direta do firmware. Nessa abordagem, os drivers utilizam funções fornecidas pelo SDK Connect da Nordic.

4. Physical Hardware

Camada mais baixa, composta pelos componentes físicos conectados ao módulo nRF.

Elementos:

- **Rádio BLE** — transceptor integrado do SoC (2,4 GHz).
- **Buzzer** — alerta sonoro controlado via PWM.
- **Botão** — entrada física
- **Bateria** — fonte de energia do sistema.

Como explicado, o sistema pode operar em **duas abordagens de configuração BLE**: via **Advertising** ou via **GATT**.

- **Modo Advertising:**

- Nesse modo, a coleira apenas transmite pacotes de *advertising* (broadcast), e o celular realiza a leitura do RSSI desses pacotes para estimar a distância.
- É um modelo unidirecional (*coleira* → *celular*).
- O algoritmo de proximidade é executado no aplicativo móvel, que decide quando acionar o alerta.
- Entre as transmissões, a coleira entra em deep sleep, otimizando o consumo de energia.

- **Modo GATT:**

- Nesse modo, a comunicação é bidirecional, permitindo que o celular configure parâmetros da coleira (como limites, potência ou intervalos).
- O algoritmo de proximidade pode ser executado diretamente no hardware da coleira, tornando-a mais autônoma.

Dependendo da abordagem, a estrutura de diretórios da arquitetura de software pode variar:

Modo Advertising:

amigo_perto/

```
|— src/
|
| |— app/
| |
| | |— main.c          # Aplicação principal
| | |— config.c/.h     # Configurações BLE
| |
| |
| |— hal/
| |
| | |— ble_hal.c/.h     # Advertising wrapper
| | |— buzzer_hal.c/.h  # PWM wrapper
| | |— battery_hal.c/.h # ADC wrapper
| | |— botao_hal.c/.h   # Wrapper GPIO pro Botão
| |
|
|— mobile_app/          # App móvel (separado)
| |— proximity.js/.dart  # Lógica RSSI→distância (celular)
|
```

```
|— boards/
|   └─ nrf52832_collar.overlay
|
|— CMakeLists.txt
|— prj.conf
└─ README.md
```

Modo GATT:

amigo_perto/

```
|— src/
|   |— main.c           # Aplicação principal
|   |— proximity.c/.h   # Dome virtual + RSSI
|   |— gatt_config.c/.h # GATT Server
|   |
|   └─ hal/
|       |— ble_hal.c/.h   # Wrapper Zephyr BLE
|       |— buzzer_hal.c/.h # Wrapper PWM
|       └─ battery_hal.c/.h # Wrapper ADC
|       └─ botao_hal.c/.h  # Wrapper GPIO pro Botão
|
|— boards/
|   └─ nrf52832_collar.overlay # Device Tree Overlay
|
|— CMakeLists.txt
|— prj.conf                # Kconfig do Zephyr
```

Camada HAL (Hardware Abstraction Layer)

HAL do BLE

Função / Estrutura	Descrição	Observações
ble_adv_data_t	Estrutura de dados transmitida no advertising packet	Contém UUID do serviço, ID do PET, nível de bateria, estado e potência TX
ble_adv_event_cb_t	Callback opcional para eventos de advertising	Pode ser usada para tratar eventos de início/parada do advertising
ble_hal_init()	Inicializa o BLE em modo broadcaster	Parâmetros: nome do dispositivo, ID do PET, potência TX; retorna 0 em sucesso
ble_hal_start_advertising()	Inicia o advertising com intervalo definido	Parâmetro: intervalo em ms (100–10000)
ble_hal_stop_advertising()	Para o advertising	Retorna 0 em sucesso
ble_hal_update_adv_data()	Atualiza todos os dados do advertising packet	Parâmetro: ponteiro para ble_adv_data_t
ble_hal_update_battery()	Atualiza apenas o nível de bateria	Envio otimizado, valor 0–100%
ble_hal_update_state()	Atualiza o estado da coleira	0 = normal, 1 = alerta
ble_hal_get_tx_power()	Retorna a potência de transmissão configurada	Em dBm

ble_hal_is_advertising()	Verifica se o advertising está ativo	Retorna <code>true</code> se ativo, <code>false</code> se parado
BLE_ADV_SERVICE_UUID	UUID base do serviço	Ex.: 0x180F (Battery Service)

Modo GATT:

Função / Estrutura	Descrição	Observações
ble_rssi_cb_t	Callback para leitura periódica de RSSI	Recebe valor RSSI em dBm
ble_conn_cb_t	Callback para eventos de conexão/desconexão	Recebe ponteiro para conexão e estado
ble_central_hal_init()	Inicializa BLE em modo central	Parâmetros: nome do dispositivo, endereço do tutor autorizado; retorna 0 em sucesso
ble_central_hal_start_scan()	Inicia scan BLE	Conecta somente ao dispositivo autorizado; intervalo em ms
ble_central_hal_stop_scan()	Para o scan BLE	Sem retorno de erro
ble_central_hal_connect()	Conecta ao dispositivo emparelhado	Retorna 0 em sucesso, negativo em erro
ble_central_hal_disconnect()	Desconecta do dispositivo ativo	—
ble_central_hal_get_connection()	Obtém ponteiro para conexão ativa	Retorna NULL se não conectado
ble_central_hal_get_rssi()	Lê RSSI da conexão ativa	Parâmetro: ponteiro para <code>int8_t</code> ; retorna 0 em sucesso
ble_central_hal_set_rssi_callback()	Configura callback periódico de RSSI	Parâmetros: função callback e intervalo em ms

ble_central_hal_set_conn_callback()	Configura callback de conexão/desconexão	—
--	--	---

HAL do Buzzer

Função / Estrutura	Descrição	Observações
buzzer_hal_init()	Inicializa o hardware do buzzer	Deve ser chamada antes de ler tensão ou nível; retorna 0 em sucesso
buzzer_hal_on()	Liga o buzzer com frequência e volume definidos	Parâmetros: freq_hz (Hz), volume (0–100%); retorna 0 em sucesso
buzzer_hal_off()	Desliga o buzzer	Retorna 0 em sucesso
buzzer_hal_beep()	Emite count bipes de duração definida	Parâmetros: count (número de bips), duration_ms (duração de cada beep em ms); retorna 0 em sucesso

HAL da Bateria

Função / Estrutura	Descrição	Observações
battery_hal_init()	Inicializa a medição da bateria	Deve ser chamada antes de ler tensão ou nível; retorna 0 em sucesso
battery_hal_get_voltage_mv()	Retorna a tensão da bateria em milivolts	Tipo uint16_t
battery_hal_get_level_percent()	Retorna o nível da bateria em porcentagem	Tipo uint8_t , valor de 0 a 100%

HAL do Botão

Função / Estrutura	Descrição	Observações
button_hal_init()	Inicializa o botão e configura interrupção	Deve ser chamada antes de usar; retorna 0 em sucesso
button_hal_set_callback()	Configura callback para eventos do botão	Função callback é chamada quando o botão é pressionado ou liberado;

Comparação microcontrolador

Parâmetro	IN100	nRF52840	nRF52832	nRF54L15
Bluetooth	5.3	5.4	5.4	Até 6.0
Potência de Transmissão (TX)	Até +5dBm	Até +8dBm	Até +4dBm	Até +8dBm
Sensibilidade de Recepção (RX)	Valor não encontrado	Até -103dBm	Até -96dBm	Até -96dBm
Consumo de Corrente do Sistema para 3V	650nA (Sleep Mode)	0,4 µA – Sleep Mode 1,86 µA – Sistema OFF, com RAM totalmente retida 0,97 µA – Sistema ON, sem retenção de RAM	0,3 µA – Sleep Mode 1,2 µA – Todos os periféricos em modo ocioso (IDLE) 1,6 µA – Todos os periféricos em modo ocioso (IDLE) com 32 kHz + RTC ativos	0,7 µA – System OFF 0,9 µA – with GRTC wakeup 1,7 µA – ON com 96kb de RAM

		2,35 μ A – Sistema ON, com RAM totalmente retida 3,16 μ A – Sistema ON, com RAM totalmente retida + RTC ativo	20 nA por 4 KB – Retenção de RAM	
Tensão de Alimentação	1.1V ~ 3.6V	1.7V ~ 5.5V	1.7V ~ 3.6V	1.7V ~ 3.6V
Complexidade de desenvolvimento	Muito baixa: “sem código”, configuração via GUI	Média, muitos exemplos prontos, necessita integração com Zephyr, SDK VSCODE	Média, alguns exemplos prontos, necessita integração com Zephyr, SDK VSCODE	Média, alguns exemplos prontos, pode utilizar bare metal além do Zephyr, SDK VSCODE
Preço	Baixo: \$6.10	Baixo (fácil encontrar): R\$40	Baixo (difícil encontrar): R\$30	Médio (dificuldade média): R\$90
Tempo Estimado da Bateria (consumo médio)	Até 1 ano	Entre 3 e 6 meses	Entre 5 e 8 meses	Entre 2 a 6 meses
Recursos de Hardware	Baixo	Alto	Médio	Alto

Justificativas técnicas e econômicas

Para nosso sistema o consumo de energia é bem importante, dado que desejamos alimentar tudo com uma bateria tipo pilha, neste caso a escolha ideal seria o IN100, porém outro fator contribuiu para não o comprarmos, o fato de seu preço aumentar bastante por conta de frete e impostos, uma vez que seus módulos somente são encontrados em sites superfaturados, como a sparkfun ou mouser. Com isso definimos que usaremos os microcontrolados da Nordic Semiconductor, uma vez que nos possibilita grande quantidade de exemplos, placas prontas e SDK de fácil utilização.

Entre os candidatos, destaque o nRF52840, que contempla a maior variação de módulos e exemplos prontos, justificando nossa primeira escolha. A segunda ficou entre a nova geração e a antiga, escolhemos a nova com o nRF54L15, com o ponto positivo principal sendo a possibilidade de programar em bare metal usando versões de SDK recentes e atualizadas, já que os outros das famílias antigas são atualizados com o sistema RTOS Zephyr. Os outros componentes são de fácil acesso e não precisamos definir algum PART NUMBER por agora, trata-se de um buzzer e uma bateria.

Com isso compramos os módulos:

nRF54L15



nRF52840

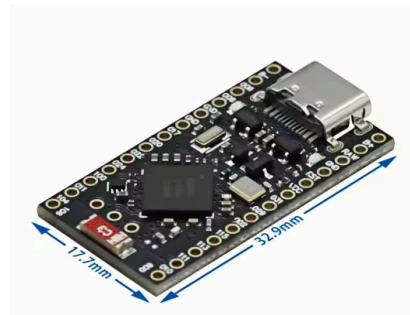
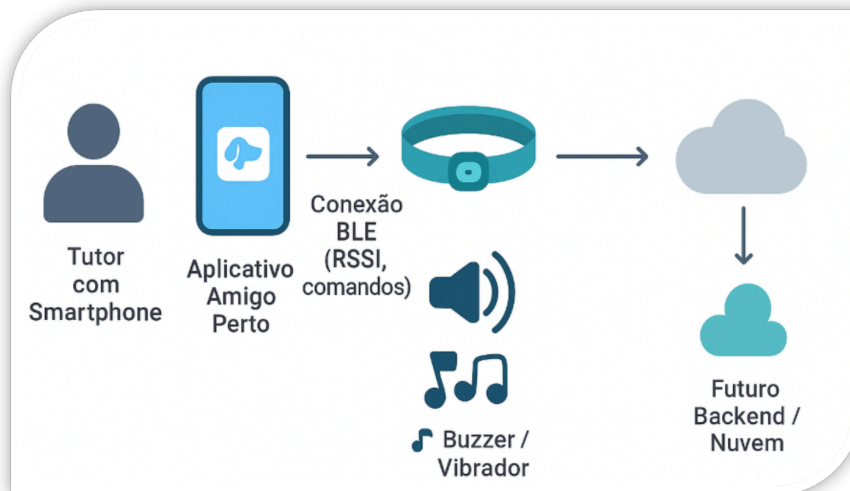


Diagrama consolidado do sistema

o diagrama de arquitetura funcional simplificado, mostrando domínio pretendido:

- os atores principais (tutor e coleira);

- o **meio de comunicação** (Bluetooth Low Energy, BLE);
- os **componentes físicos e lógicos** (aplicativo, firmware, buzzer/vibrador);
- e **camada futura opcional** (backend/nuvem).



Componentes principais:

1. Coleira Inteligente (Dispositivo BLE)

- Microcontrolador: *Nordic nRF52840 / nRF54L15 / nRF52840 / IN100 (NanoBeacon)*
- Atuadores: *Buzzer + Vibrador*
- Energia: *Bateria (CR...)*
- Comunicação: *Bluetooth Low Energy*
- Firmware embarcado:
 - Detecta intensidade de sinal RSSI
 - Aciona alerta (som/vibração) quando ultrapassa o limite definido
 - Recebe comandos do app (por exemplo: tocar ou parar buzzer)

2. Aplicativo Mobile (Tutor)

- Detecta o dispositivo BLE
- Mede RSSI (força do sinal) e calcula distância aproximada
- Define **limite configurável de distância**
- Aciona alerta no celular (som/vibração)
- Envia **comando BLE** de retorno para a coleira (alerta do pet)
- Registra **eventos locais** (data, hora, afastamento, retorno)

3. Futuras Expansões (fase posterior)

- **Sincronização com nuvem** (via REST API ou WebSocket)
 - **Histórico de eventos e localização**
 - **Painel web** para monitoramento
 - **Controle de múltiplos pets/dispositivos**
-

Etapa 2 Semana 1

Pesquisa dos conhecimentos necessários para o app (back-end e front-end)

1. O que saber sobre Android BLE

O desenvolvimento BLE no Android **não é trivial** — ele tem particularidades, limitações e diferenças entre versões do Android.

Pilares do BLE no Android

Necessário dominar:

1.1. Permissões BLE (Android 12+ mudou tudo)

Para BLE:

- **BLUETOOTH_SCAN**
 - **BLUETOOTH_CONNECT**
 - **ACCESS_FINE_LOCATION** (Android 11–)
 - **NEARBY_DEVICES** (Android 12+)
 - **BLUETOOTH_ADVERTISE** (se for advertiser)
-

1.2. APIs principais do Android BLE

Trabalhará com:

- `BluetoothAdapter`
 - `BluetoothLeScanner`
 - `BluetoothGatt`
 - `BluetoothGattCharacteristic`
 - `BluetoothGattDescriptor`
 - `BluetoothGattCallback`
 - `BluetoothDevice`
-

1.3. Fluxo básico BLE no Android

1. Ativar Bluetooth
2. Pedir permissões
3. Escanear dispositivos

4. Filtrar por **UUID de serviço** (do nRF52840 ou nRF54L15)
 5. Conectar via **BluetoothGatt**
 6. Descobrir serviços
 7. Ler/escrever características
 8. Habilitar **notificações**
 9. Manter conexão estável
 10. Reconectar automaticamente
-

1.4. Bibliotecas recomendadas:

Nordic Android BLE Library (usada por empresas do mundo todo)

<https://github.com/NordicSemiconductor/Android-BLE-Library>

Ela abstrai:

- reconexão automática
 - notificações
 - fila de comandos
 - escrita segura
 - timeouts
-

2. O que saber sobre nRF52840 / nRF54L15

Esses chips BLE da Nordic são padrão de mercado.

2.1. Ferramentas necessárias

Componente	Para que serve
nRF Connect for Desktop	Testar BLE, debug, log, GATT

nRF Connect for Android	Testar pelo celular
nRF Connect SDK (Zephyr)	Desenvolvimento firmware moderno
nRF5 SDK (legado)	Desenvolvimento clássico, ainda usado
Segger Embedded Studio	IDE para compilar/depurar
J-Link Debugger	Programar e debugar o chip

2.2. Como o nRF se comunica com Android via BLE

1 Serviço BLE
várias características (characteristics)

EXEMPLO:

Service (customizado)

UUID: 12345678-1234-5678-1234-56789abcdef0

Characteristics

Nome	UUID	Tipo
LED Control	...f1	Write
Vibrator PWM	...f2	Write
Battery Level	...f3	Notify
Device Status	...f4	Notify

Android lê/escreve essas características.

3. Arquitetura recomendada (Android + nRF)

```
Android App
|
| (BLE)
|
nRF52840 / nRF54L15
|
| (GPIO/PWM/I2C)
|
Dispositivos (LED, buzzer, vibrador)
```

4. Estrutura do projeto Android/Kotlin

Dependência recomendada (Nordic BLE Library):

```
implementation "no.nordicsemi.android:ble:2.6.1"
```

5. Exemplo de código Android BLE (Kotlin)

Escanear dispositivos

```
val scanner = BluetoothLeScannerCompat.getScanner()

val scanSettings = ScanSettings.Builder()
    .setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY)
    .build()

val filters = listOf(
    ScanFilter.Builder()

        .setServiceUuid(ParcelUuid(UUID.fromString("12345678-1234-5678-1234-56789abcdef0"))))
        .build()
)

scanner.startScan(filters, scanSettings, scanCallback)
```

Conectar

```
val bleManager = object : BleManager(context) {

    private var ledCharacteristic: BluetoothGattCharacteristic? = null

    override fun isRequiredServiceSupported(gatt: BluetoothGatt): Boolean {
        val service = gatt.getService(UUID.fromString("12345678-1234-5678-1234-56789abcdef0"))
        ledCharacteristic = service?.getCharacteristic(UUID.fromString("...f1"))
        return ledCharacteristic != null
    }

    fun sendLed(on: Boolean) {
        writeCharacteristic(
            ledCharacteristic,
            byteArrayOf(if (on) 1 else 0),
            BluetoothGattCharacteristic.WRITE_TYPE_DEFAULT
        ).enqueue()
    }
}

bleManager.connect(device).enqueue()
```

6. Exemplo no nRF52840 (Zephyr / NCS)

```
static ssize_t on_write_led(struct bt_conn *conn, const struct bt_gatt_attr *attr,
                           const void *buf, uint16_t len, uint16_t offset, uint8_t flags)
{
    const uint8_t *value = buf;
    if (value[0] == 1) {
        gpio_pin_set(led_gpio, LED_PIN, 1);
    } else {
        gpio_pin_set(led_gpio, LED_PIN, 0);
    }
    return len;
}

BT_GATT_SERVICE_DEFINE(simple_service,
    BT_GATT_PRIMARY_SERVICE(BT_UUID_DECLARE_128(SERVICE_UUID)),
    BT_GATT_CHARACTERISTIC(
        BT_UUID_DECLARE_128(LED_UUID),
        BT_GATT_CHRC_WRITE,
        BT_GATT_PERM_WRITE,
        NULL,
        on_write_led,
        NULL
    ),
);
```

7. O que pesquisar:

Android BLE:

- "Android BLE GATT"
- "Android BLE permissions API 31"
- "BluetoothGattCallback Kotlin"
- "Nordic Android BLE Library tutorial"

nRF:

- "nRF Connect SDK BLE peripheral UART example"
- "Zephyr GATT service"
- "nRF52840 GPIO + PWM"

- "nRF54L15 BLE example"
- "nRF52 custom BLE service"

Tabela de Documentação — Android + nRF BLE

Android

Categoria	Biblioteca / Framework	Função / Uso no Projeto	Observações
BLE Core	<code>BluetoothAdapter</code> , <code>BluetoothManager</code>	Habilitar Bluetooth, iniciar scan	API nativa Android
BLE Scan	<code>BluetoothLeScanner</code> , <code>ScanFilter</code> , <code>ScanSettings</code>	Escanear dispositivos BLE com filtro de UUID	Requer permissões Android 12+
Conexão BLE	<code>BluetoothGatt</code> , <code>BluetoothGattCallback</code>	Conectar ao dispositivo BLE	Base nativa, baixo nível
Nordic BLE Library	<code>no.nordicsemi.android:ble- ktx</code>	Facilita conexão BLE, notificações, leituras e escritas	Recomendado pela Nordic, alto nível
MVVM	Android Architecture Components	Separar UI / lógica	Padrão oficial Android
UI	Jetpack Compose ou XML	Interface do app	Compose recomendado
Permissões	<code>ActivityResultContracts</code> , <code>Manifest.permission.*</code>	Pedir permissões BLE e localização	Obrigatório Android 12+
Logs	<code>Timber</code>	Logging organizado	Opcional
Persistência	<code>DataStore</code>	Guardar último dispositivo conectado	Seguro e moderno
Serialização	<code>Kotlin Serialization</code>	Enviar JSON no BLE-UART (opcional)	Para payloads estruturados

FIRMWARE nRF (NCS – Zephyr OS)

Categoria	Biblioteca / Framework	Função / Uso	Observações
SDK Base	nRF Connect SDK (NCS)	Ambiente oficial Nordic com BLE moderno	Suporte a nRF52840 / nRF54L15
Build System	west , CMake	Compilar firmware	Recomendado pela Nordic
RTOS	Zephyr	Sistema operacional do nRF	Gerencia threads, timers, BLE
BLE Stack	BT_GATT_SERVICE_DEFINE	Criar serviço GATT customizado	Onde define LED, vibração, sensores
Characteristics	BT_GATT_CHARACTERISTIC	Criar características Write/Notify	Essencial para comunicação
Advertising	bt_le_adv_start()	Iniciar advertising BLE	Personalizável
Notificações	bt_gatt_notify()	Enviar dados ao app Android	Para bateria, estado, etc
Callbacks	write_cb, read_cb	Receber comandos do Android	Implementação do protocolo
GPIO	gpio_pin_set()	Acionar LED/relé	Para cargas
PWM	pwm_set()	Vibrador, motor, brilho	Depende do board
Logs	LOG_MODULE_REGISTER()	Logging interno no firmware	Usado com UART ou RTT
DFU/OTA	MCUBoot	Atualização remota	Recomendado para futuro

Ferramentas de Desenvolvimento

Ferramenta	Uso	Observações
nRF Connect for Desktop	Scanner BLE, DFU, debug	Indispensável
nRF Connect for Mobile	Testes BLE no celular	Perfeito para validar GATT
Segger Embedded Studio	IDE oficial da Nordic	Incluso no NCS

VS Code + nRF Extension	Alternativa moderna	Funciona muito bem
Bluetooth SIG	Criar UUIDs oficiais	Documentação GATT
Android Studio	Desenvolver app Android	Obrigatório

Tabela de Funções-Chave (Android + nRF)

Componente	Função	Responsabilidade
Android → scanDevices()	Escanear BLE	Localizar nRF
Android → connect(device)	Conectar ao peripheral	Iniciar GATT
Android → writeCharacteristic(uuid, data)	Enviar comando (LED/vibração)	Controle do dispositivo
Android → enableNotifications(uuid)	Receber dados	Status, bateria
Firmware → write_cb()	Receber comando	Acionar GPIO/PWM
Firmware → bt_gatt_notify()	Enviar eventos ao Android	Mudança de estado
Firmware → gpio_pin_set()	Acionar carga	LED/relé
Firmware → pwm_set()	Vibrar/ajustar intensidade	Motores/sinalização

ROADMAP COMPLETO – 3 SEMANAS

Objetivo final:

App Android em Kotlin que **escaneia, conecta, envia comandos e recebe notificações BLE**

Firmware nRF que expõe **serviço BLE customizado + GPIO/PWM**

Comunicação confiável entre ambos

SEMANA 1 — “Fundação” (Android BLE + Ferramentas Nordic)

Meta: entender BLE, instalar ferramentas, rodar exemplos e conectar manualmente.

Dia 1 — Conceitos BLE (rápido e essencial)

Estudar **apenas o que importa para programar**, sem teoria inútil:

- O que é BLE GATT (Service / Characteristics / UUID)
- Write | Notify | Read
- MTU
- Advertising
- Diferença Central ↔ Peripheral
- Papéis:
 - **Android = Central**
 - **nRF52840/nRF54 = Peripheral**

Tempo: 1h

Resultado: você sabe “como BLE realmente funciona”.

Dia 2 — Instalar ferramentas da Nordic

Instalar:

- ✓ **nRF Connect for Desktop**
- ✓ **nRF Connect for Mobile** (Android)
- ✓ **nRF Connect SDK (v2.x)**
- ✓ **Toolchain Manager**
- ✓ **Segger Embedded Studio**
- ✓ (Opcional) CLI **west**

Resultado: ambiente de firmware moderno e pronto.

Dia 3 — Teste BLE no celular

Usando **nRF Connect Mobile**:

Escanear dispositivos BLE

Ver advertising

Abrir GATT

Ler / escrever características

Ativar notificações

Resultado: você já sabe interagir com qualquer dispositivo BLE.

Dia 4 — Testar um firmware BLE do nRF52840/nRF54

Rodar um exemplo pronto:

NCS → `peripheral_uart`

Ou `ble_app_uart` (nRF5 SDK)

Com ele você já:

faz advertising

conecta do Android

envia dados

recebe notificações

Resultado: comunicação bidirecional BLE funcionando.

Dia 5 — Estruturar seu projeto BLE customizado

Criar um **novo serviço BLE** para LED/vibrador/etc:

- Definir UUIDs
- Estruturar características
- Criar documento GATT (importante para Android depois)

Resultado: **seu GATT oficial**, base para app Android.

SEMANA 2 — “Mão na massa Android BLE + Firmware Customizado”

Dia 6 — Criar o projeto Android em Kotlin

Criar app:

- ✓ Kotlin + ViewModel
- ✓ Permissões BLE
- ✓ Tela inicial + botão “Scan”
- ✓ Mostrar lista de dispositivos

Resultado: app Android iniciando o fluxo.

Dia 7 — Implementar SCAN com filtros

Usar:

ScanFilter + UUID

ScanSettings

Permissões Android 12+

Resultado: lista de dispositivos BLE filtrados.

Dia 8 — Criar BLE Manager (usando Nordic Library)

Criar classe que cuida de tudo:

```
class MyBleManager(context: Context) : BleManager(context)
```

Implementar:

Discover services

Encontrar characteristics

Conectar + reconectar

Timeout

Filas de escrita

Resultado: estrutura BLE profissional.

Dia 9 — Firmware customizado no nRF

Criar características reais:

LED control (write)

Vibration control (write)

Battery level (notify)

Device status (notify)

Usando NCS:

- BT_GATT_SERVICE_DEFINE
- BT_GATT_CHARACTERISTIC
- callbacks de write
- notificações via bt_gatt_notify

Resultado: firmware BLE pronto.

Dia 10 — Android: Escrever características

Enviar comandos:

```
bleManager.writeLed(true)
```

Enviar PWM, duração, modo, etc.

Resultado: app Android controlando hardware real.

Dia 11 — Android: Receber notificações

Implementar:

- `batteryLevelCallback`
- `statusNotificationCallback`

Exibir na interface.

Resultado: app mostrando dados do nRF em tempo real.

SEMANA 3 — “Produto Real” (UI + Reconexão + Testes + Otimização)

Dia 12 — Criar UI final (Material3)

Layouts:

- Tela Scan
- Tela Device
- Tela Controle (LED/Vibração/Sinais)
- Tela Debug BLE (log)

Resultado: app bonito e funcional.

Dia 13 — Persistência + reconexão automática

Salvar:

- último dispositivo conectado
- auto-reconnect

- último estado dos controles

Resultado: experiência profissional.

Dia 14 — Segurança BLE

Implementar:

- Device whitelist
- Pairing/Bonding
- MAC randomizado
- Firmware atualizável via OTA (opcional)

Resultado: app e firmware com robustez.

Dia 15 — Testes intensivos

Testar:

- Distâncias
- Variação de advertising interval
- Latência
- Interferência
- Energia (nRF)

Resultado: comportamento no mundo real.

Dia 16 — Logs + Telemetria

Criar logs:

- Conexões
- RSSI
- Perdas

- Reconexões
- Erros GATT

Resultado: capacidade de debug profissional.

Dia 17 — Finalização e hardening

Checklist:

- UUIDs padronizados
- Comentários GATT
- Remover código morto
- Organizar MVVM
- Testar em diferentes Androids
- Documentar firmware + app

Resultado: versão 1.0 do sistema.

SEMANA 3 – Entrega Final

Aplicativo BLE completo

Firmware BLE customizado no nRF52840/nRF54

Controle de dispositivos + notificações

Reconexão automática

Documentação completa do GATT

Estudo do curso “Bluetooth Low Energy Fundamentals” da Nordic

Bluetooth Low Energy (BLE)

Adicionado no Bluetooth 4.0 (2010), o BLE foi criado para baixo consumo de energia, tornando possível dispositivos alimentados por bateria e aplicações IoT.

Principais diferenças em relação ao Classic:

- Tecnologia totalmente diferente
- Usa 40 canais de 2 MHz
- Envia pacotes pequenos e com baixa frequência
- Baixo consumo com menor taxa de transferência

Aplicações típicas:

- Smartwatches
 - Sensores de saúde
 - Sensores ambientais
 - Localização indoor (beacons)
-

1. Arquitetura BLE (Host + Controller)

Tanto no Classic quanto no BLE a pilha é dividida em duas camadas:

Controller

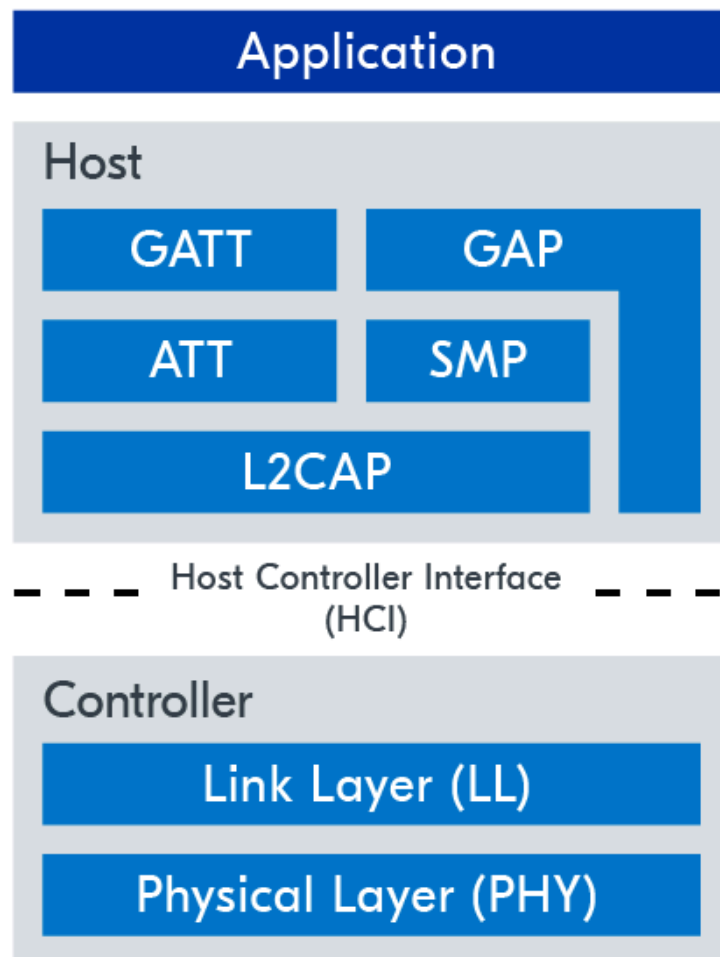
- Cuida do rádio e da transmissão física
- Inclui:
 - PHY (modulação e parâmetros do rádio)
 - Link Layer (estados: advertising, scanning, conexão, etc)

Host

- Parte lógica da pilha Bluetooth
- Engloba:
 - GAP: descoberta e conexão entre dispositivos
 - GATT: organização dos dados em serviços e características
 - ATT: formato dos dados que serão transmitidos

- SMP: segurança e criptografia
- L2CAP: multiplexação e fragmentação de pacotes

Hoje, a maioria dos microcontroladores BLE (como os da Nordic) integra Host e Controller no mesmo chip.



2. GATT – Generic Attribute Profile

O GATT define como os dados são organizados e acessados em um dispositivo BLE. Os dados são divididos hierarquicamente:

- Profiles (perfis)
- Services (serviços)
- Characteristics (características)

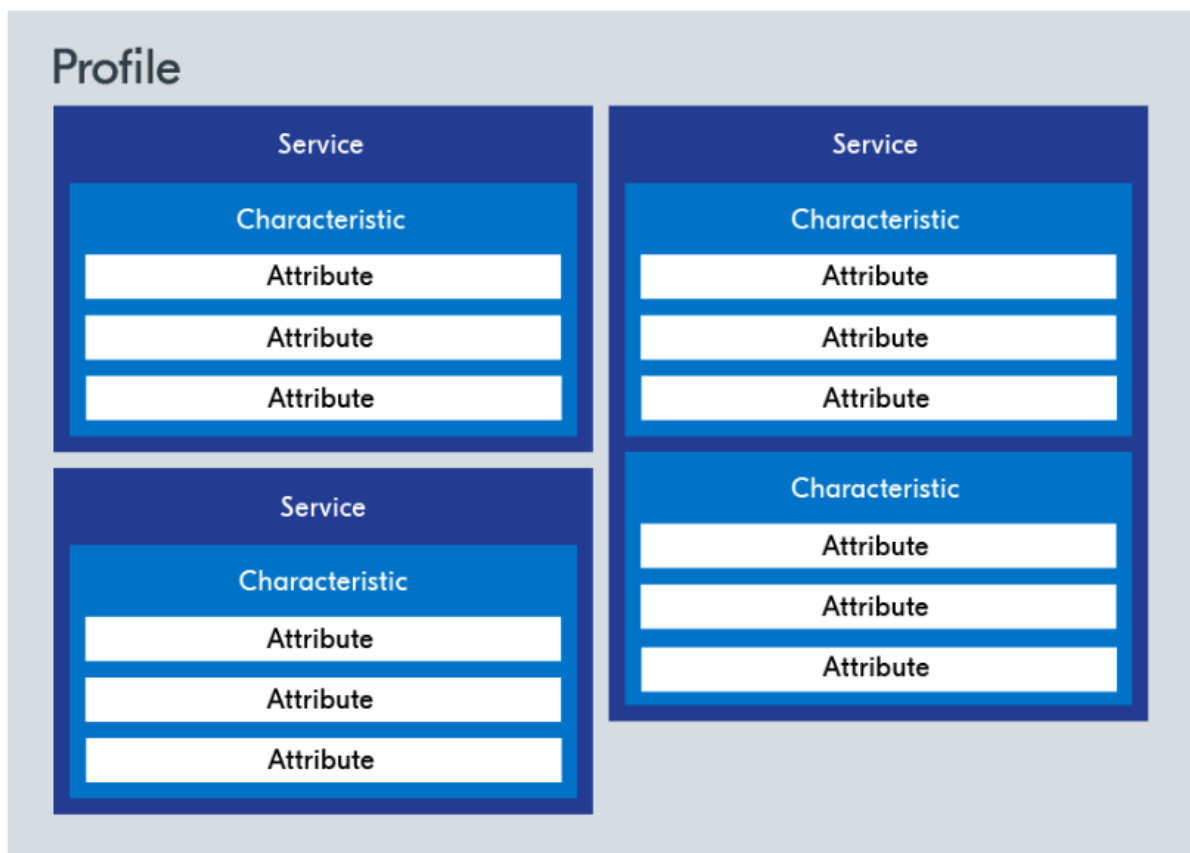
Exemplo:

O serviço ambiental (UUID 0x181A) possui características de temperatura, umidade e pressão.

Serviços e características podem ser:

- Padronizados (UUID de 16 bits do Bluetooth SIG)
- Customizados (UUID de 128 bits criados pelo desenvolvedor)

Ferramentas como o nRF Connect permitem visualizar e ler essas características.



3. ATT – Attribute Protocol

O ATT define o formato dos dados transmitidos. Cada valor enviado durante a conexão BLE é um atributo. Cada atributo possui:

- UUID
- Identificador

- Permissões
- Valor armazenado

O GATT utiliza o ATT para organizar e transmitir esses atributos.

4. GAP – Generic Access Profile

Define como os dispositivos se encontram e como ocorre a conexão.

Existem duas formas principais de comunicação:

1. Advertising / Broadcast (sem conexão)
 - O dispositivo transmite dados periodicamente para qualquer um que esteja escutando
 - Exemplo: beacons, sensores transmitindo dados curtos
2. Conexão (central ↔ periférico)
 - O periférico anuncia
 - O central escuta
 - Se interessar, o central envia uma requisição de conexão

Normalmente:

- O periférico é o servidor GATT
- O central é o cliente GATT

Outros papéis possíveis:

- Broadcaster: só transmite dados, não aceita conexão
 - Observer: só recebe dados, não inicia conexão
-

5. Topologias BLE

- Broadcast: um envia dados, vários recebem; simples e eficiente energeticamente.
 - Conexão: comunicação bidirecional central-periférico.
 - Multi-role: o mesmo dispositivo pode ser central e periférico ao mesmo tempo.
Exemplo: um hub que recebe dados dos sensores e envia para um celular.
-

6. Modos PHY no BLE

PHY	Taxa	Uso típico
1M	1 Mbps	modo padrão
2M	2 Mbps	maior velocidade, menor alcance
Coded PHY (S=2 e S=8)	500 kbps ou 125 kbps	maior alcance, menor taxa

Modo Advertising

1. Conceito Geral

O **advertising** (anúncio) é um dos pilares do Bluetooth Low Energy (BLE), responsável tanto por **divulgar dados** a dispositivos próximos quanto por **anunciar a presença** de um dispositivo para permitir conexões.

O processo envolve **transmissão periódica de pacotes** contendo informações que permitem a descoberta e identificação do dispositivo.

2. Parâmetros Principais

- **Advertising interval:** intervalo entre pacotes de anúncio (20 ms a 10,24 s). Intervalos menores aumentam a rapidez da descoberta, mas também o consumo de energia.
- **Canais de advertising:** BLE usa 40 canais no total; 3 são primários (37, 38 e 39) para anúncios e 37 são secundários para transmissão de dados. Os canais primários são escolhidos para reduzir a interferência de Wi-Fi.
- **Scan interval e scan window:** definem com que frequência e por quanto tempo um scanner (central) busca anúncios. Aumentar a taxa de varredura reduz o tempo de

descoberta, mas consome mais energia.

3. Tipos de Advertising

Há diferentes modos de anúncio, combinando características de **conectabilidade**, **escaneabilidade** e **direcionamento**:

Tipo	Conectável	Escaneável	Direcionado	Descrição
ADV_IND	✓	✓	✗	Tipo mais comum; permite descoberta e conexão.
ADV_DIRECT_IND	✓	✗	✓	Direcionado a um dispositivo conhecido (reconexão rápida).
ADV_SCAN_IND	✗	✓	✗	Permite apenas envio de informações extras via scan response.
ADV_NONCONN_IND	✗	✗	✗	Usado em beacons; não aceita conexão nem scan.

4. Scan Request e Scan Response

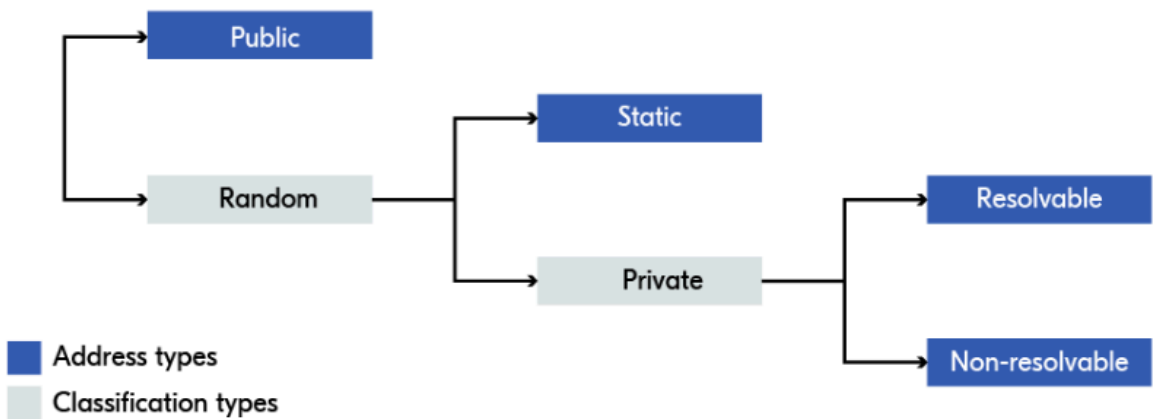
- **Scan Request:** o central solicita mais dados ao periférico.
- **Scan Response:** o periférico responde com informações adicionais (ex.: nome completo, dados de fabricante).

Esse mecanismo permite transmitir mais dados **sem necessidade de conexão**.

5. Tipos de Endereço Bluetooth

Todo dispositivo BLE possui um **endereço de 48 bits**, podendo ser **público** ou **aleatório**:

Tipo	Persistência	Privacidade	Descrição
Public Address	Fixo	✗	Definido pelo fabricante e registrado na IEEE.
Random Static Address	Fixo	✗	Configurado pelo usuário; comum em projetos BLE.
Resolvable Private Address	Muda periodicamente	✓ (resolvível via IRK)	Oculto identidade, mas pode ser reconhecido por dispositivos autorizados.
Non-resolvable Private Address	Muda periodicamente	✓ (não resolvível)	Evita rastreamento, mas não pode ser identificado.



6. Estrutura do Pacote de Advertising

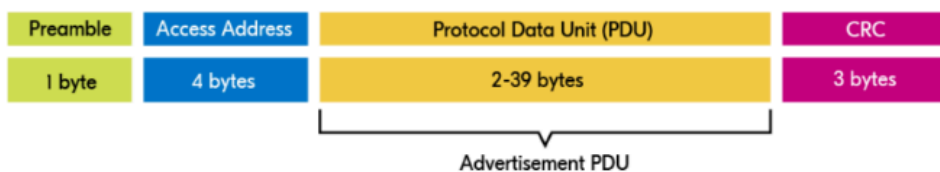
Um pacote BLE contém um **Protocol Data Unit (PDU)** dividido em:

- **Header:** define o tipo de anúncio, endereços públicos/aleatórios e tamanho do payload.
- **Payload:**
 - **AdvA:** endereço do anunciante (6 bytes)
 - **AdvData:** dados de advertising (informações do dispositivo)

O campo **AdvData** é composto por múltiplas **AD structures**, cada uma com:

- **Length** – tamanho do campo
- **AD Type** – tipo de dado (ex.: nome, UUID, flags)
- **AD Data** – conteúdo real

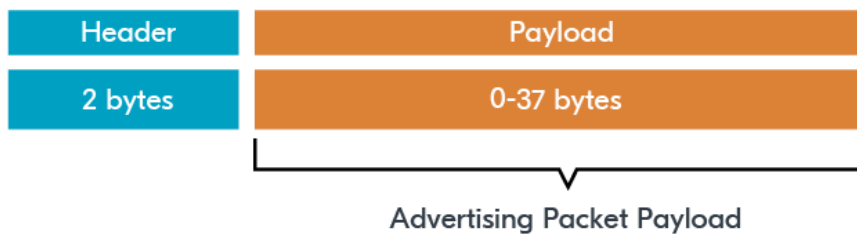
Bluetooth LE Packet



Advertisement PDU



Advertisement PDU



Payload



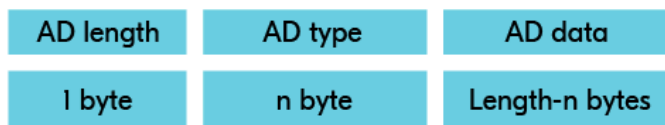
Advertising Packet Payload



AdvData



Advertisement Data Structure



7. Tipos Comuns de AD Data

- **BT_DATA_NAME_COMPLETE**: nome completo do dispositivo
- **BT_DATA_NAME_SHORTENED**: nome abreviado
- **BT_DATA_URI**: URLs ou identificadores de recursos
- **BT_DATA_UUID**: identifica serviços disponíveis
- **BT_DATA_MANUFACTURER_DATA**: dados personalizados (ex.: iBeacon)
- **Flags**: indicam propriedades e modos operacionais (1 byte com até 8 flags)

Principais flags

- **BT_LE_AD_LIMITED**: modo de descoberta limitado (tempo determinado)
- **BT_LE_AD_GENERAL**: modo de descoberta geral (sem tempo limite)
- **BT_LE_AD_NO_BREDR**: indica ausência de suporte ao Bluetooth clássico (BR/EDR)

Modo Conexão

Conexão BLE – visão geral

A forma mais comum de alcançar comunicação bidirecional no Bluetooth Low Energy é por meio de conexões. Uma conexão BLE permite maior controle, segurança, confirmação de pacotes, retransmissões automáticas, criptografia e gerenciamento de energia. A conexão é criada, mantida e encerrada usando parâmetros que afetam desempenho e consumo.

Como uma conexão é estabelecida

1. O periférico está advertising.
2. O central está scanning.
3. O central encontra o periférico e envia um connection request.
4. Quando o periférico aceita, ambos passam a usar os 37 canais de dados.
5. A conexão passa a operar com channel hopping para evitar interferência.

Na conexão, ambos os dispositivos acordam apenas em momentos agendados para transmitir ou receber pacotes, reduzindo o consumo de energia.

Conceitos importantes

Conceito	Descrição
Connection interval	Frequência com que central e periférico acordam para transmitir pacotes.
Connection event	Janela de tempo dentro do intervalo onde ocorre a troca de dados.
Supervision timeout	Tempo máximo sem resposta antes de considerar que a conexão caiu.
Peripheral latency	Permite ao periférico pular eventos de conexão para economizar energia.

Desconexão

A conexão pode terminar por decisão da aplicação ou por timeout caso um dos lados deixe de responder (por exemplo, ficou fora de alcance ou desligou).

Quando o supervision timeout é atingido, a conexão é encerrada automaticamente.

Parâmetros da conexão

Parâmetro	Quem define	Função
Connection interval	Central (periférico pode solicitar alteração)	Afeta latência, velocidade e consumo.
Peripheral latency	Central (opcional)	Economiza energia permitindo "pular" eventos.
Supervision timeout	Central	Detecta perda de conexão.
PHY (1M, 2M, Coded)	Negociado	Ajusta taxa de transmissão ou alcance.
Data Length	Negociado	Tamanho máximo de payload de um pacote no rádio.
MTU	Negociado	Tamanho máximo de um pacote de dados na camada GATT.

Data Length e MTU

- **MTU:** tamanho máximo de dados por operação GATT. O padrão é 23 bytes.
- **Data Length:** tamanho máximo de carga útil em um pacote BLE. O padrão é 27 bytes.

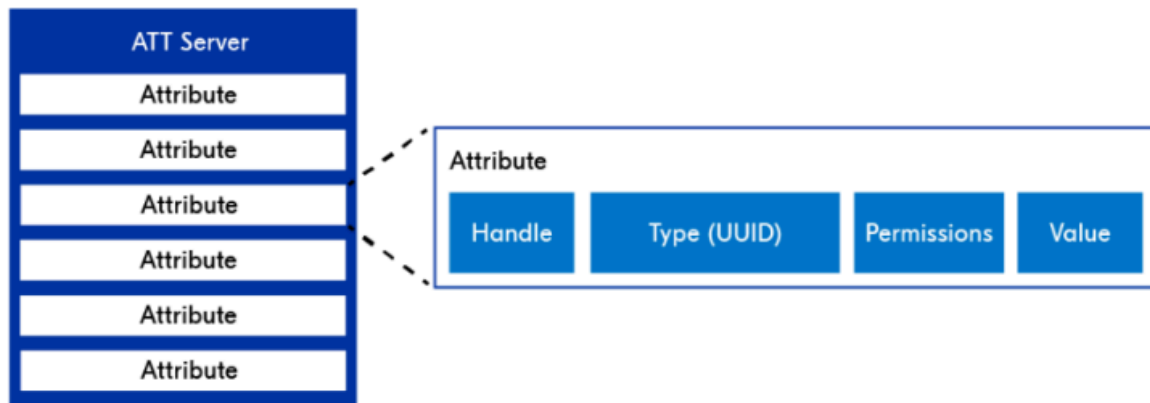
Com Data Length Extension, esse valor pode subir até **251 bytes**, reduzindo tempo de rádio ativo e aumentando throughput.

Quem controla os parâmetros

O central estabelece os parâmetros principais da conexão. O periférico pode solicitar mudanças, mas o central decide se aceita. Para **MTU**, **Data Length** e **PHY**, qualquer lado pode solicitar mudanças, começando sempre pelos valores padrão após conectar.

GATT e ATT – visão geral

O GATT organiza os dados do servidor BLE em uma tabela de atributos. Cada atributo tem quatro campos: **handle** (endereçamento), **tipo (UUID)**, **permissões** e **valor**. Serviços e características são conjuntos estruturados desses atributos.



Antes de trocar dados, o cliente realiza **service discovery** para descobrir quais serviços e características o servidor oferece. Só então ele pode ler, escrever ou assinar características para receber atualizações.

Operações de troca de dados

Existem dois tipos principais:

1. Operações iniciadas pelo cliente

- **Read:** o cliente solicita o valor de um atributo e o servidor responde
- **Write:** o cliente envia um valor e recebe confirmação
- **Write Without Response:** a escrita é enviada sem esperar confirmação, acelerando a transferência

2. Operações iniciadas pelo servidor

- **Notify:** o servidor envia dados sem confirmação do cliente
- **Indicate:** o servidor envia dados e o cliente confirma o recebimento (por exigir resposta, é mais lento e limitado a uma indicação por intervalo de conexão)

Para receber **Notify** ou **Indicate**, o cliente deve habilitar isso escrevendo no **CCCD (Client Characteristic Configuration Descriptor)** da característica desejada.

Organização dos dados (GATT e ATT)

- **ATT** define como atributos são endereçados e acessados
- **GATT** organiza esses atributos em serviços e características

Cada atributo possui:

- Handle
 - UUID
 - Permissões (read/write, segurança)
 - Valor
-

UUIDs

A identificação dos atributos usa UUIDs:

- **16 bits (padrão SIG):** serviços e características oficiais
 - **128 bits (custom):** definidos pelo desenvolvedor para aplicações específicas
-

Serviços e características

Um serviço sempre começa com um **Service Declaration Attribute** (UUID 0x2800).

Cada característica contém pelo menos:

1. **Characteristic Declaration Attribute** (UUID 0x2803)
2. **Characteristic Value Attribute** (onde os dados são armazenados)

Pode também conter **descriptors**, como o **CCCD (UUID 0x2902)**, responsável por habilitar notificações e indicações.

Atributo CCCD

- Permite ao cliente ativar Notify e Indicate
- Sempre legível e gravável
- Bits:
 - **bit 0:** habilita Notify
 - **bit 1:** habilita Indicate

Attribute Table

É a tabela organizada com todos os atributos de serviços e características no servidor BLE.

Ex: serviço customizado `my_lbs` possui três características:

- **Button** (com Indicate e CCCD)
- **LED** (somente Write)
- **MySensor** (com Notify e CCCD)

A ordem dos atributos define a qual característica cada valor e descritor pertence.

Service (my_lbs)	Handle	UUID	Attribute Permissions	Attribute Value
my_lbs Service Declaration	0×0001	0×2800	Read	00001523-1212-ef-de-1523-785feabcd123
Button Characteristic Declaration	0×0002	0×2803	Read	Properties: Read or Indicate Handle of value: 0×0003 UUID: 00001524-1212-ef-de-1523-785feabcd123
Button Characteristic Value Declaration	0×0003	00001524-1212-ef-de-1523-785feabcd123	Read	User data: 0×20002689
Button Descriptor Declaration	0×0004	0×2902	Read & write	Indicate: 0×02
LED Characteristic Declaration	0×0005	0×2803	Read	Properties: Write Handle of value: 0×0006 UUID: 00001525-1212-ef-de-1523-785feabcd123
LED Characteristic Value Declaration	0×0006	00001525-1212-ef-de-1523-785feabcd123	Write	User data
MySensor Characteristic Declaration	0×0007	0×2803	Read	Properties: Notify Handle of value: 0×0008 UUID: 00001526-1212-ef-de-1523-785feabcd123
MySensor Characteristic Value Declaration	0×0008	00001526-1212-ef-de-1523-785feabcd123	None	User data
MySensor Descriptor Declaration	0×0009	0×2902	Read & write	Notify: 0×01

Segurança no BLE

A segurança é fundamental em aplicações BLE. O protocolo oferece mecanismos para autenticação, integridade, confidencialidade e privacidade durante a comunicação. A seguir é apresentado o funcionamento do pareamento, diferenças entre modos Legacy e LE Secure Connections, níveis de segurança e uso da Filter Accept List.

1. Pairing e Bonding

Termo	Significado
Pairing	Processo de gerar e trocar chaves usadas para criptografar o link BLE.

Bonding	Armazena as chaves do pareamento para uso futuro, evitando novo processo de pairing.
---------	--

Sem bonding, o dispositivo precisa parear novamente toda vez que quiser usar criptografia.

2. Fases do Pareamento

Phase 1 – Initiate Pairing

- O dispositivo central envia o Pairing Request.
- O periférico responde com Pairing Response.
- São trocadas capacidades de I/O e requisitos de segurança.

Capacidades I/O possíveis:

Capacidade	Função
DisplayOnly	Apenas mostra números
DisplayYesNo	Mostra número e permite confirmar sim/não
KeyboardOnly	Apenas digitação
KeyboardDisplay	Mostra e digita
NoInputNoOutput	Sem interação do usuário

Essas capacidades determinam qual método de autenticação pode ser usado.

Phase 2 – Perform Pairing

Nesta etapa são geradas as chaves que irão criptografar o link.

Métodos possíveis:

Método	Funcionamento	Segurança
Just Works	O usuário apenas aceita	Vulnerável a MITM
Passkey Entry	Um mostra código e o outro digita	Melhor que Just Works, mas fraco em Legacy

OOB	Troca de chave por canal externo (ex: NFC)	Alta segurança
Numeric Comparison (apenas LE SC)	Ambos exibem número e o usuário confirma	Alta proteção

A escolha depende das capacidades de I/O, exigência de proteção contra MITM e disponibilidade de OOB.

Phase 3 – Key Distribution

Após autenticação, são distribuídas chaves como:

- LTK – usada para criptografia futura
- IRK – resolve endereços privados
- CSRK – valida integridade de mensagens assinadas

Permite reconexões seguras sem novo pareamento.

3. Legacy Pairing x LE Secure Connections

Legacy Pairing (até Bluetooth 4.1)

- Baseado em STK, facilmente quebrado
- Just Works é totalmente inseguro ($TK = 0$)
- Passkey vulnerável a brute-force
- Apenas OOB oferece segurança real
- Não recomendado

LE Secure Connections (Bluetooth 4.2+)

- Usa ECDH (Diffie–Hellman) para gerar chaves fortes
- Cria LTK segura
- Suporta Numeric Comparison

- Muito mais resistente a MITM
 - Ainda permite Just Works, mas sem proteção contra MITM
-

4. Ameaças comuns e como o BLE resolve

Ataque	Solução
Rastreamento de identidade	Endereços privados resolvíveis (IRK)
Captura passiva de dados	Criptografia do link
MITM	Numeric Comparison, Passkey ou OOB

5. Níveis de Segurança (Security Mode 1)

Nível	Tipo	Características
1	Sem segurança	Dados em claro
2	Criptografia sem autenticação	Just Works
3	Criptografia com autenticação	Passkey ou OOB (Legacy)
4	LE Secure Connections com autenticação	Numeric Comparison, Passkey ou OOB

Cada característica GATT pode exigir um nível mínimo.

Se o link estiver no nível 2 e a característica exigir nível 3, o acesso é bloqueado.

6. Filter Accept List (Whitelisting)

Serve para restringir:

- quem pode conectar
- quem pode receber scan response
- quais dispositivos aparecem no scanner

A lista utiliza:

- endereço do dispositivo
- chaves de identidade obtidas no bonding (IRK)

Pode ser temporariamente desativada para possibilitar o pareamento de novos dispositivos.

Funções de BLE do SDK que serão usadas no projeto

Modo Advertising:

Função / API	Módulo (Zephyr / SDK)	O que faz no código	Finalidade (Importância prática no projeto)
<code>bt_enable()</code>	Bluetooth Core	Inicializa toda a pilha Bluetooth do dispositivo	Sem isso o Bluetooth não funciona. É o passo obrigatório para habilitar o BLE na coleira.
<code>bt_set_name()</code>	GAP	Define o nome Bluetooth da coleira antes de anunciar	Permite que o dono identifique a coleira no aplicativo ao escanear dispositivos próximos.
<code>bt_le_adv_start()</code>	GAP Advertising	Inicia o advertising da coleira	Torna a coleira visível para o celular. Sem advertising, o celular nunca descobre o dispositivo.

Modo Conexão (GATT):

Função / API	Módulo (Zephyr / SDK)	O que faz no código	Finalidade (Importância prática no projeto)
<code>bt_enable()</code>	Bluetooth Core	Inicializa toda a pilha Bluetooth	Sem isso o Bluetooth não funciona. É o primeiro

			passo obrigatório para ativar o BLE.
<code>bt_set_name()</code>	GAP	Define o nome da coleira	Permite que o usuário identifique a coleira ao procurar dispositivos no celular.
<code>bt_le_adv_start()</code>	GAP Advertising	Começa o advertising com a coleira em modo conectável	Torna o dispositivo visível e permite que o app se conecte. Sem isso, não há conexão.
<code>bt_conn_cb_register()</code>	Connection	Registra callbacks de conexão e desconexão	Detecta quando o celular conecta ou desconecta da coleira, permitindo iniciar/parar lógica de proximidade.
<code>bt_gatt_service_register()</code>	GATT Server	Registra o serviço GATT customizado com características	Cria as “portas” pelas quais o app envia comandos e recebe dados. Sem isso, não existe comunicação via GATT.
<code>bt_gatt_notify()</code>	GATT Server	Envia notificações ao app (RSSI, alertas, status)	É a função principal para avisar o celular automaticamente quando o pet se afasta.
<code>bt_gatt_is_subscribed()</code>	GATT Server	Verifica se o cliente habilitou notificações	Evita tentar notificar um celular que não ativou notificações, prevenindo erros de protocolo.
<code>bt_conn_get_rssi()</code>	GAP Connection	Lê o RSSI em tempo real da conexão	Permite detectar se o pet está se afastando pela intensidade do sinal. Sem RSSI, não existe alerta de distância.

<code>bt_conn_unref()</code>	Connection	Libera referência da conexão quando desconecta	Evita vazamento de memória e mantém o sistema estável ao reconectar várias vezes.
<code>bt_gatt_attr_read()</code>	GATT Server	Função callback quando o app lê uma característica	Permite que o app consulte dados da coleira quando quiser.
<code>bt_gatt_attr_write()</code>	GATT Server	Callback quando o app envia comando (ex: beep, novo limite etc.)	É o canal pelo qual o app controla a coleira (alertas, reset, configurações). Sem isso, o app não consegue agir sobre o dispositivo.

Link do Repositório: <https://github.com/EmbarcaTech-2025/projeto-amigo-perto>

Etapa 2 Semana 2

Cronograma

1) Início da implementação do App Android

Objetivo: Iniciar o desenvolvimento do aplicativo Android para interação via BLE.

Atividades:

- Iniciar a criação do projeto Android em **Kotlin**.
- Começar a implementar:

- Estrutura com **Kotlin + ViewModel**.
 - Permissões para uso de **Bluetooth Low Energy (BLE)**.
 - Tela inicial com botão **"Scan"**.
 - Exibição da lista de dispositivos encontrados.
 - **Resultado esperado ao final do desenvolvimento:** Aplicativo Android capaz de iniciar o fluxo de escaneamento BLE.
 - **Documentação:** Registrar os avanços e decisões técnicas no documento de entrega.
-

2) Teste do Buzzer

Objetivo: Validar acionamento do buzzer utilizando SDK da Nordic.

Atividades:

- Estudar como acionar o buzzer usando o **nRF Connect SDK**:
 - Referência:
<https://academy.nordicsemi.com/courses/nrf-connect-sdk-fundamentals/lessons/lesson-1-nrf-connect-sdk-introduction/>
 - Conteúdo auxiliar:
https://www.youtube.com/watch?v=IzD76fde0ZQ&list=PLoKOKJqqgHs_ULz5l1CIDPqqkQKugQmDA&index=2
 - Criar código para acionamento do buzzer no **nRF54L15**.
 - Realizar **build** do código.
 - **Documentação:** Descrever funcionamento, resultados e eventuais ajustes no documento de entrega.
-

3) Teste BLE Advertising

Objetivo: Configurar advertising contínuo para serviço personalizado da coleira.

Atividades:

- Estudar exercícios 1, 2 e 3 da Lição 2 do curso BLE:
 - <https://academy.nordicsemi.com/courses/bluetooth-low-energy-fundamentals/lessons/lesson-2-bluetooth-le-advertising/topic/blefund-lesson-2-exercise-1/>
- Criar código adaptando exercícios ou exemplos para:
 - Configurar advertising contínuo do serviço personalizado.
 - Intervalo sugerido: **100–500 ms**.
- Realizar **build** do código.
- **Documentação:** Registrar funcionamento e resultados no documento de entrega.

4) Teste BLE Conexão

Objetivo: Implementar conexão BLE com serviço GATT personalizado.

Atividades:

- Estudar:
 - Exercício 1 da Lição 3 e exercícios 1 e 2 da Lição 4 do curso BLE:
 - <https://academy.nordicsemi.com/courses/bluetooth-low-energy-fundamentals/lessons/lesson-3-bluetooth-le-connections/topic/blefund-lesson-3-exercise-1/>
 - Conteúdo auxiliar:
https://www.youtube.com/watch?v=o-NECS2TN94&list=PLoKOKJqggHs_KmqS4tN2rDrLFaWrhGGI&index=4
- Criar código para:
 - Configurar conexão BLE.
 - Implementar serviço GATT personalizado para aceitar conexões.
- Realizar **build** do código.
- **Documentação:** Detalhar funcionamento e resultados no documento de entrega.

Início da implementação do App Android

Relatório de Progresso: Implementação Inicial do Aplicativo (Front-End + nRF54L15)

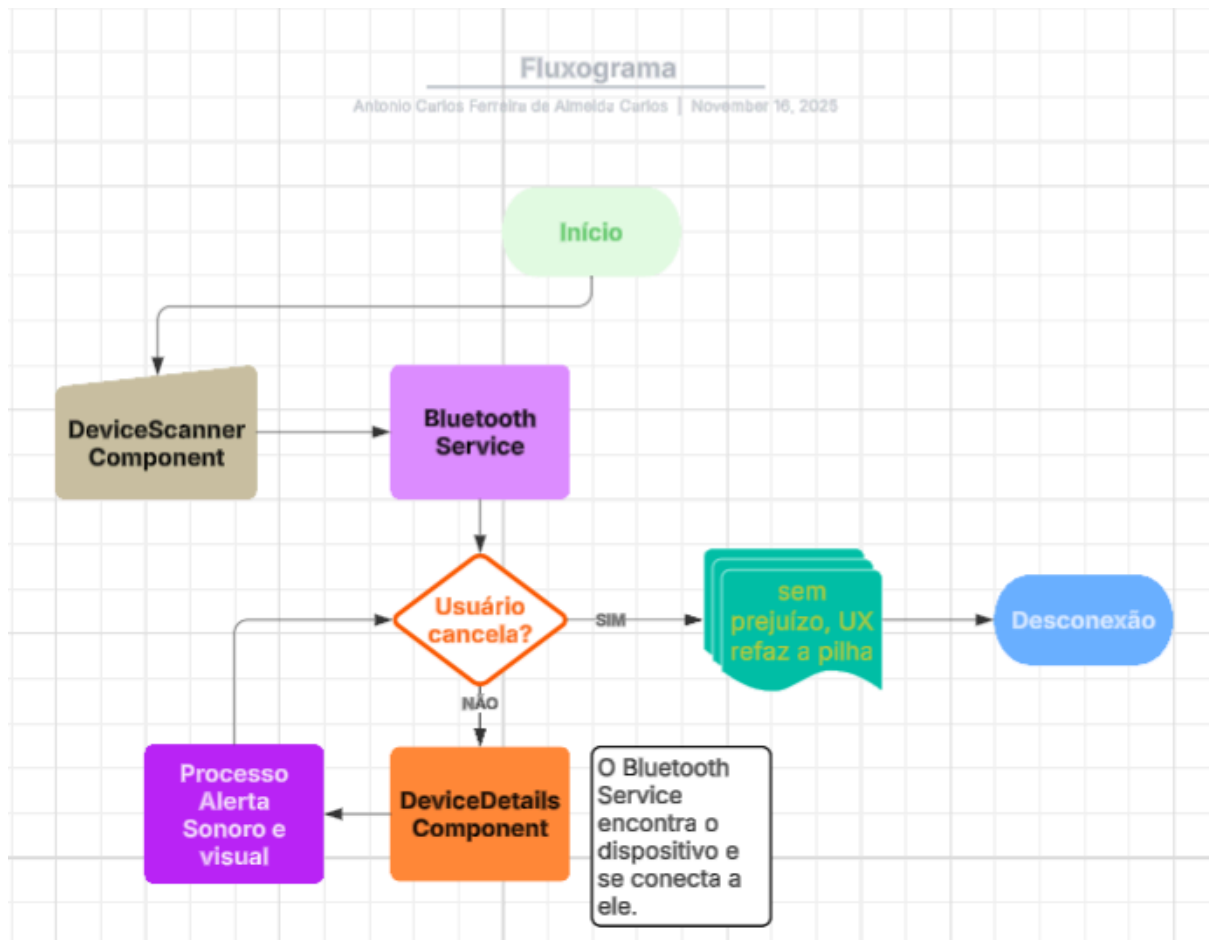
Concluímos com sucesso a primeira etapa de desenvolvimento do aplicativo, estabelecendo uma base sólida para a interação com dispositivos próximos. Todas as metas e expectativas para esta fase foram atingidas.

- **Estrutura do Projeto:** Foi criada uma fundação robusta e bem organizada para o aplicativo. Essa arquitetura foi pensada para ser escalável, facilitando a adição de novas funcionalidades e a manutenção do sistema no futuro.
- **Gestão de Permissões:** O aplicativo agora solicita ao usuário, de forma clara e transparente, as permissões necessárias para a comunicação sem fio. Este processo foi implementado para garantir uma experiência de usuário fluida e segura, afirmando que o aplicativo tenha o acesso de que precisa para funcionar corretamente.
- **Interface Inicial e Funcionalidade de Busca:** Desenvolvemos a tela principal do aplicativo com um design limpo e um botão de “Scan” em destaque. Esta interface serve como ponto de partida para que o usuário inicie a busca por dispositivos, tornando a ação principal simples e intuitiva.
- **Exibição de Resultados:** Ao acionar a busca, o aplicativo já é capaz de detectar dispositivos nas proximidades e apresentar uma lista com os resultados encontrados. As informações são exibidas de maneira clara, permitindo que o usuário identifique facilmente os dispositivos ao

seu redor e principalmente acesse o alvo de maneira assertiva.

- **Resultado Final da Etapa:** Ao final desta fase, o resultado esperado foi alcançado: temos um aplicativo de partida que inicia com sucesso, o fluxo de escaneamento de dispositivos, cumprindo todos os requisitos essenciais da especificação inicial.

Todo o progresso e as decisões técnicas relevantes foram devidamente documentadas para garantir a continuidade e a qualidade do projeto. Estamos prontos para avançar para as próximas fases de desenvolvimento.



Ponto de Partida: O usuário abre a aplicação.

1. **Tela Inicial (AppComponent + DeviceScannerComponent)**
 - **O que o usuário vê?**
 - Um cabeçalho (HeaderComponent).
 - Um botão de ação principal com o texto "Busca Dispositivo". Este é o DeviceScannerComponent.
 - O design atual (Fase 3) apresenta um fundo com textura sutil, e o botão tem um gradiente e efeito de sombra para se destacar.
 - **Ação do Usuário:** O usuário clica no botão "Buscar Dispositivo".
2. **Início da Busca (DeviceScannerComponent -> BluetoothService)**
 - **Lógica:**

- O DeviceScannerComponent muda seu estado para "buscando". Visualmente, o botão muda de cor para o gradiente de "busca" (vermelho/amarelo).
 - Ele chama o bluetooth.service.ts para iniciar a interação com a API de Bluetooth.
- **Possíveis Erros:**
 - **Usuário cancela:** Se o usuário fechar a janela de seleção de dispositivo do navegador, o serviço trata o erro de forma sutil com UX .
 - **Bluetooth desligado/indisponível:** O serviço lida com a falha.
 - Em caso de erro, a aplicação volta ao estado inicial (Passo 1).
- 3. **Conexão e Monitoramento (BluetoothService -> DeviceDetailsComponent)**
 - **Lógica:**
 - O BluetoothService encontra um dispositivo e se conecta a ele.
 - Ele começa a monitorar a força do sinal (RSSI) em intervalos regulares.
 - Com base no RSSI, ele calcula a distância aproximada entre o celular do usuário e o dispositivo conectado.
 - **Transição de Tela:**
 - Assim que um dispositivo é conectado, a visualização muda. O DeviceScannerComponent é provavelmente ocultado e o DeviceDetailsComponent é exibido.
- 4. **Exibição dos Detalhes e Alerta (DeviceDetailsComponent)**
 - **O que o usuário vê?**
 - Um "cartão" de informações (com cantos arredondados e sombra) mostrando o nome do dispositivo e a distância atualizada em tempo real.
 - Mensagens de status estilizadas (por exemplo, "Conectado", "Monitorando").
 - **Lógica Principal (O Coração do App):**
 - O componente recebe continuamente a distância calculada pelo BluetoothService.
 - Ele compara essa distância com um limite pré-definido. (a aferição pode ser um ponto de equilíbrio, considerando testes de campo - fase ainda não contemplada!!!)
 - **Se a distância ultrapassar o limite:**
 - O componente dispara um **ciclo de alerta ininterrupto**.
 - **Alerta Sonoro:** Toca 5 "beeps".
 - **Alerta Visual:** A tela pisca para chamar a atenção do usuário.
 - Este ciclo se repete enquanto o dispositivo estiver fora do alcance seguro.
- 5. **Fim do Processo (Desconexão)**
 - O monitoramento e os alertas continuam até que o dispositivo seja desconectado (seja por voltar ao alcance, por ação do usuário, ou por perda de sinal definitiva).
 - Após a desconexão, a aplicação retorna ao seu estado inicial (Passo 1), pronta para uma nova busca.

Este fluxo representa a jornada inicial do usuário e a interação entre os principais componentes de software (DeviceScannerComponent, DeviceDetailsComponent) e a lógica de serviço (BluetoothService), incluindo as melhorias visuais que você está implementando na Fase 3.

Teste BLE Advertising

Nesse teste, o dispositivo transmite periodicamente pacotes de advertising (anúncio) contendo informações como nome do dispositivo e uma URL, permitindo que outros dispositivos BLE nas proximidades detectem sua presença.

1. Características Principais:

- **Advertising BLE não-conectável:** Transmite dados sem aceitar conexões
- **LED de status:** Pisca a cada 1 segundo indicando funcionamento
- **Transmissão de URL:** Inclui link para "academy.nordicsemi.com"
- **Configuração via USB:** Console serial via USB CDC ACM
- **Sistema operacional Zephyr RTOS**

2. ARQUITETURA DO SISTEMA

2.1 Hardware Utilizado

Nordic nRF52840

- Processador ARM Cortex-M4F @ 64 MHz
- BLE 5.3
- Flash: 1 MB | RAM: 256 KB
- LED no GPIO **P0.07**

Nordic nRF54L15

- Processador ARM Cortex-M33 @ 128 MHz
- BLE 5.4
- Flash: 1.5 MB | RAM: 256 KB

2.2 Software

- **Sistema Operacional:** Zephyr RTOS v3.x
- **SDK:** Nordic nRF Connect SDK
- **Linguagem:** C
- **Build System:** CMake + Ninja

3. ESTRUTURA DO CÓDIGO

3.1 Arquivo Principal: *main.c*

Inclusões e Dependências

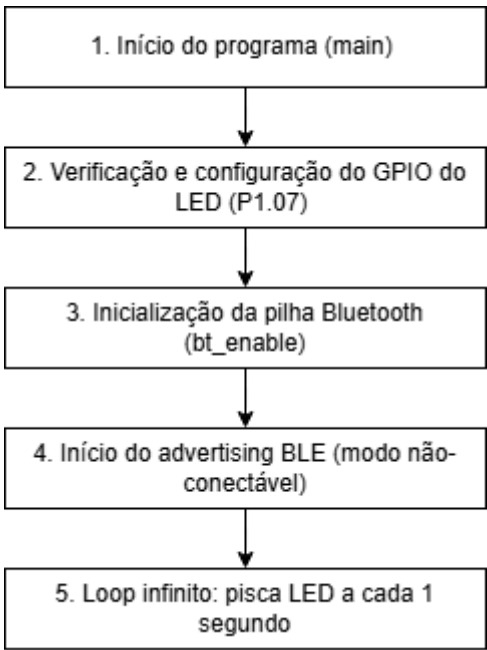
```
#include <zephyr/kernel.h>           // Núcleo do Zephyr RTOS
#include <zephyr/logging/log.h>       // Sistema de logging
#include <zephyr/bluetooth/bluetooth.h> // Pilha Bluetooth LE
#include <zephyr/bluetooth/gap.h>     // GAP (Generic Access Profile)
#include <zephyr/drivers/gpio.h>      // Driver GPIO para LEDs
```

Definições e Configurações

Macro	Valor	Descrição
DEVICE_NAME	"Nordic_Beacon"	Nome do dispositivo BLE (configurado via Kconfig)
LED_NODE	DT_ALIAS(led0)	Referência ao LED no Device Tree
RUN_LED_BLINK_INTERVAL	1000 ms	Intervalo de piscagem do LED

4. FUNCIONAMENTO DETALHADO

4.1 Fluxo de Inicialização



4.2 Inicialização do LED (Linhas 89-101)

Passo 1: Verifica se o dispositivo GPIO está pronto

```

if (!gpio_is_ready_dt(&led))
{
    LOG_ERR("LED GPIO device not ready\\\\\\n");
    return -1;
}

```

Passo 2: Configurar o pino como saída em nível inativo (LOW = LED apagado)

```
err = gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);
```

4.3 Inicialização do Bluetooth (Linhas 105-112)

```
err = bt_enable(NULL);
```

Esta função:

- Inicializa a pilha Bluetooth do Zephyr
- Configura o controlador BLE
- Prepara o rádio para transmissão
- O parâmetro **NULL** indica que não há callback customizado

4.4 Configuração dos Pacotes de Advertising

Pacote Principal (Advertisement Data)

```

static const struct bt_data ad[] = {
    BT_DATA_BYTES(BT_DATA_FLAGS, BT_LE_AD_NO_BREDR),
    BT_DATA(BT_DATA_NAME_COMPLETE, DEVICE_NAME, DEVICE_NAME_LEN),
};

```

Componentes:

- **FLAGS:** **BT_LE_AD_NO_BREDR** indica que o dispositivo **não suporta** Bluetooth Classic (apenas BLE)
- **NAME_COMPLETE:** Transmite o nome completo "Nordic_Beacon"

Pacote de Scan Response

```

static unsigned char url_data[] = {
    0x17, '/', '/', 'a', 'c', 'a', 'd', 'e', 'm', 'y',
    '.', 'n', 'o', 'r', 'd', 'i', 'c', 's', 'e', 'm',
    'i', '.', 'c', 'o', 'm'
};

```

```

static const struct bt_data sd[] = {
    BT_DATA(BT_DATA_URI, url_data, sizeof(url_data)),
};

```


Explicação:

- `0x17`: Tipo de encoding da URI (esquema URL padrão)
- Seguido pela URL: `//academy.nordicsemi.com`
- Transmitido quando um scanner solicita informações adicionais

4.5 Início do Advertising (Linhas 123-129)

```
err = bt_le_adv_start(BT_LE_ADV_NCONN, ad, ARRAY_SIZE(ad), sd,
ARRAY_SIZE(sd));
```

Parâmetros:

- `BT_LE_ADV_NCONN`: Modo **não-conectável** (apenas broadcasting)
- `ad`: Dados do pacote primário
- `sd`: Dados do scan response
- `ARRAY_SIZE`: Calcula automaticamente o número de elementos

Comportamento:

- O dispositivo transmite pacotes de advertising a cada **intervalo padrão** (~100ms)
- Outros dispositivos podem detectar o beacon mas **não podem conectar**
- Ideal para aplicações de proximidade, rastreamento e notificações

4.6 Loop Principal (Linhas 134-138)

```
for (;;) {
    gpio_pin_toggle_dt(&led);
    k_sleep(K_MSEC(RUN_LED_BLINK_INTERVAL));
}
```

Funcionamento:

- `gpio_pin_toggle_dt()`: Inverte o estado do LED (ligado ↔ desligado)
- `k_sleep()`: Suspende a thread por 1000ms (função do Zephyr RTOS)
- **Resultado visual**: LED pisca 1 vez por segundo

5. CONFIGURAÇÃO DO HARDWARE

5.1 Device Tree Overlay (`nrf52840dongle_nrf52840.overlay`)

```
leds {
    compatible = "gpio-leds";
    custom_led: led_0 {
```

```

        gpios = <&gpio1 7 GPIO_ACTIVE_HIGH>;
        label = "Custom LED";
    };
};

alias {
    led0 = &custom_led;
};

```

Explicação:

- Define um LED customizado no **GPIO Port 1, Pin 7**
- **GPIO_ACTIVE_LOW**: LED acende quando o pino está em HIGH
- Cria alias **led0** para facilitar acesso no código

5.2 Configuração USB (*nrf52840dongle_nrf52840.conf*)

Habilita console serial via USB:

```

CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_CONSOLE=y

```

6. CONFIGURAÇÃO DO PROJETO

6.1 Arquivo prj.conf

Configuração	Descrição
CONFIG_LOG=y	Habilita sistema de logging
CONFIG_GPIO=y	Ativa driver GPIO
CONFIG_BT=y	Inclui pilha Bluetooth LE
CONFIG_BT_DEVICE_NAME="Nordic_Beacon"	Define nome do dispositivo
CONFIG_MAIN_STACK_SIZE=2048	Tamanho da stack da thread principal
CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE=2048	Stack da fila de trabalho do sistema

7. PROCESSO DE COMPILAÇÃO E FLASH

7.1 Build do Projeto

```
west build -b nrf52840dongle_nrf52840
```

7.2 Geração do Pacote para Flash

```
west build -t uf2
```

7.3 Flash via USB

1. Conecte o nRF52840/nRF54L15
2. Pressione o botão RESET enquanto conecta
3. Arraste o arquivo `.uf2` para a unidade USB
4. O dispositivo reinicia automaticamente

Teste BLE Conexão

Neste teste, o dispositivo atua como um periférico BLE simples que aceita conexões de smartphones, permitindo que o aplicativo móvel monitore a distância através da leitura do RSSI (Received Signal Strength Indicator).

Características Principais:

- Advertising BLE conectável: transmite dados e aceita conexões de dispositivos móveis
- LED de status: pisca a cada 1 segundo indicando funcionamento
- Conexão persistente: mantém conexão ativa com o celular
- Reconexão automática: reinicia advertising após desconexão
- LED customizado: GPIO P0.07 (nRF52840) ou P1.07 (nRF54L15)
- Sistema operacional Zephyr RTOS
- Medição RSSI pelo celular: firmware não processa RSSI, apenas mantém conexão

2. ARQUITETURA DO SISTEMA

2.1 Hardware Suportado

Opção 1: Nordic nRF52840

- Placa: Nordic nRF52840 Dongle ou equivalente
- Processador: ARM Cortex-M4F @ 64 MHz
- Bluetooth: 5.3 BLE compatível
- LED Customizado: GPIO P0.07 (Port 0, Pin 7)
- Flash: 1 MB
- RAM: 256 KB

Opção 2: Nordic nRF54L15

- Placa: Nordic nRF54L15 DK
- Processador: ARM Cortex-M33 @ 128 MHz
- Bluetooth: 5.4 BLE compatível
- LED Customizado: GPIO P1.07 (Port 1, Pin 7)
- Flash: 1.5 MB
- RAM: 256 KB

2.2 Software

- Sistema Operacional: Zephyr RTOS v3.x
- SDK: Nordic nRF Connect SDK
- Linguagem: C
- Build System: CMake + West (Zephyr's meta-tool)
- Toolchain: ARM GCC

3. ESTRUTURA DO CÓDIGO

3.1 Arquivo Principal: main.c

Inclusões e Dependências:

```
#include <zephyr/kernel.h>

#include <zephyr/logging/log.h>

#include <zephyr/bluetooth/bluetooth.h>

#include <zephyr/bluetooth/gap.h>

#include <zephyr/bluetooth/gatt.h>

#include <zephyr/bluetooth/uuid.h>

#include <zephyr/bluetooth/addr.h>

#include <zephyr/bluetooth/conn.h>

#include <zephyr/drivers/gpio.h>
```

Definições e Configurações:

Macro	Valor	Descrição
DEVICE_NAME	"BLE_Coleira"	Nome do dispositivo BLE
LED_NODE	DT_ALIAS(led0)	Referência ao LED no Device Tree
RUN_LED_BLINK_INTERVAL	1000 ms	Intervalo de piscagem do LED

Variáveis Globais:

Variável	Tipo	Descrição
my_conn	struct bt_conn*	Ponteiro para conexão BLE ativa
adv_work	struct k_work	Estrutura de trabalho para adv.
led	struct gpio_dt_spec	Especificação GPIO do LED

4. FUNCIONAMENTO DETALHADO

4.1 Fluxo de Inicialização

1. Início do programa (main)
2. Verificação e configuração do GPIO do LED (Pin 7)
3. Registro dos callbacks de conexão BLE
4. Inicialização da pilha Bluetooth (**bt_enable**)
5. Início do advertising BLE (modo conectável)
6. Loop infinito: pisca LED a cada 1 segundo

4.2 Inicialização do LED

- Verifica se o dispositivo GPIO está pronto
- Configura o pino como saída em nível inativo (LOW = LED apagado)
- LED é configurado como **ACTIVE_HIGH** no devicetree

4.3 Registro de Callbacks BLE

- **on_connected()**: chamado quando um dispositivo se conecta
- **on_disconnected()**: chamado quando a conexão é encerrada
- **on_recycled()**: chamado após limpeza da conexão (reinicia advertising)

4.4 Inicialização do Bluetooth

- Inicializa a pilha Bluetooth do Zephyr

- Configura o controlador BLE

- Prepara o rádio para transmissão

4.5 Configuração dos Pacotes de Advertising

- Parâmetros: modo conectável, intervalo rápido (30-60ms)
- Pacote principal: flags e nome completo do dispositivo
- Scan response: UUID customizado transmitido quando solicitado

4.6 Início do Advertising

- Dispositivo transmite pacotes de advertising a cada 30-60ms
- Outros dispositivos podem detectar e conectar ao beacon

4.7 Callbacks de Conexão

- Conexão estabelecida: incrementa referência, mantém conexão ativa
- Desconexão: libera recursos
- Reciclagem: reinicia advertising automaticamente

4.8 Loop Principal

- Pisca LED a cada 1 segundo para indicar funcionamento

5. CONFIGURAÇÃO DO HARDWARE

5.1 Device Tree Overlay para nRF52840

- Define LED customizado no GPIO Port 0, Pin 7
- Cria alias led0 para facilitar acesso no código
- Configura console serial via USB CDC ACM

5.2 Device Tree Overlay para nRF54L15

- Usa GPIO Port 1, Pin 7 (gpio1) em vez de gpio0
-

5.3 Configuração USB para nRF52840

- Permite visualizar logs via porta serial USB
- Facilita depuração durante desenvolvimento

6. CONFIGURAÇÃO DO PROJETO

6.1 Arquivo prj.conf

Principais configurações:

- Logger module
- GPIO driver para LED
- Bluetooth LE
- Modo periférico
- Nome do dispositivo BLE
- Stack sizes

7. PROCESSO DE COMPILAÇÃO E FLASH

7.1 Pré-requisitos

- nRF Connect SDK instalado
- West tool configurado
- Toolchain ARM GCC instalado
- VS Code com extensão nRF Connect

7.2 Build do Projeto para nRF52840

```
west build -b nrf52840dongle_nrf52840 --pristine
```

7.3 Build do Projeto para nRF54L15

```
west build -b nrf54l15dk_nrf54l15_cpuapp --pristine
```

7.4 Flash via USB (nRF52840)

1. Pressione RESET no dongle ao conectar ao USB
2. Dispositivo aparece como unidade USB
3. Arraste o arquivo `.uf2` para a unidade

7.5 Flash via J-Link (nRF54L15)

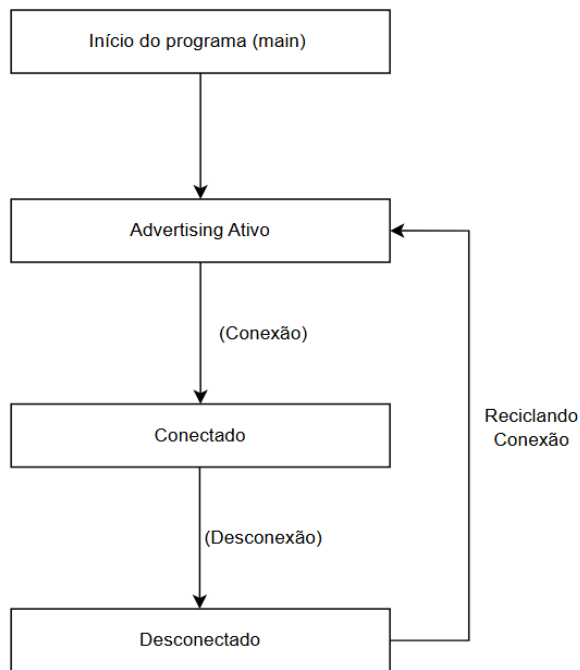
```
west flash
```

7.6 Visualização dos Logs

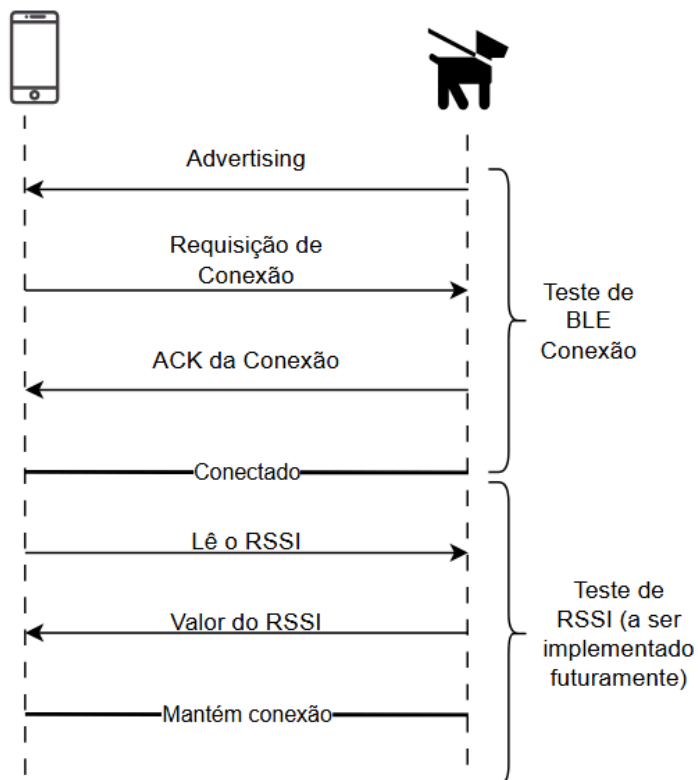
- Para nRF52840: terminal serial (115200 baud, 8N1)
- Para nRF54L15: nRF Connect Serial Terminal ou `west attach`

8. DIAGRAMAS DE ESTADO

8.1 Máquina de Estados do Advertising



8.2 Fluxo de Conexão BLE



9. MEDIÇÃO DE RSSI PELO CELULAR

- RSSI é medido pelo celular, não pelo firmware

- Firmware mantém conexão BLE ativa

Exemplo Android:

```
gatt.readRemoteRssi();
```

Interpretação dos valores RSSI:

RSSI (dBm)	Distância Aproximada	Qualidade do Sinal
-30 a -50	Muito próximo (<1m)	Excelente
-50 a -70	Próximo (1-5m)	Bom
-70 a -85	Médio (5-15m)	Regular
-85 a -100	Longe (15-30m)	Fraco
< -100	Muito longe (>30m)	Muito fraco

Etapa 2 Semana 3

Relatório de Resultados de Testes Integrados.

Data: 23 de Novembro de 2025

Versão da Aplicação: 1.0.0

Autor: Equipe de Desenvolvimento

Este documento detalha os resultados dos testes de integração executados na aplicação de AmigoPerto. O objetivo destes testes foi validar a interoperabilidade e a comunicação fluida entre os principais módulos do sistema, garantindo que as funcionalidades centrais operem de forma coesa e em conformidade com os requisitos de negócio. Os testes focaram nos fluxos de ponta a ponta (end-to-end) que envolvem a interação entre a interface do usuário (UI), o serviço de gerenciamento Bluetooth (BluetoothService), e as APIs nativas do navegador (Web Bluetooth API e Geolocation API).

Componentes em Escopo:

- **AppComponent:** O orquestrador principal da UI e dos estados da aplicação.
- **DeviceInfoComponent:** O componente responsável por exibir as informações do dispositivo conectado.
- **BluetoothService:** O serviço que encapsula toda a lógica de comunicação com a Web Bluetooth API, gerenciando os estados de `idle`, `radar` e `alert`.
- **APIs do Navegador:**
 - **Web Bluetooth API:** Para busca, conexão e comunicação com dispositivos.
 - **Geolocation API:** Para obter as coordenadas do usuário ao acionar a funcionalidade de pânico.

Objetivos:

1. **Validação do Fluxo de Conexão:** Verificar se o ciclo completo de busca (Modo Radar), seleção e conexão (Modo Alerta) com um dispositivo Bluetooth funciona sem falhas.
2. **Verificação da Lógica de Estado:** Assegurar que os sinais (`signals`) que gerenciam o estado da aplicação (`operatingMode`, `isLoading`, `device`, `error`) são atualizados corretamente e que a UI reage de forma síncrona a essas mudanças.
3. **Teste da Funcionalidade de Pânico:** Validar a integração com a Geolocation API, a correta formulação da URL do Google Maps e o acionamento da nova janela do navegador.
4. **Análise de Manipulação de Erros:** Confirmar que os cenários de erro (ex: permissão de localização negada, falha na conexão Bluetooth, dispositivo fora de alcance) são capturados pelo serviço e comunicados de forma clara ao usuário na UI.

Os testes foram conduzidos em um ambiente de desenvolvimento controlado, utilizando o Diversas ferramentas computacionais, bem como, CLI e o servidor de desenvolvimento `ng serve`. Para simular a interação com hardware, foi utilizado um dispositivo Bluetooth Low Energy (BLE) periférico para os testes de conexão e um emulador de localização integrado às ferramentas de desenvolvedor do navegador para os testes de geolocalização.

A abordagem foi baseada em casos de uso que simulam a interação real do usuário com a aplicação.

4. Cenário de Testes e Resultados:

ID do Teste	Descrição do Cenário	Passos de Execução	Resultado Esperado	Resultado Observado	Status
INT-01	Descoberta de Dispositivo (Modo Radar)	1. Iniciar a aplicação (estado <code>idle</code>). Clicar em "Procurar Dispositivo".	A aplicação transita para o modo <code>radar</code> . A UI exibe o componente <code>DeviceInfoComponent</code> com os dados do dispositivo encontrado.	O <code>BluetoothService</code> iniciou o scan, encontrou o dispositivo simulado e atualizou o <code>signal device</code> . A UI refletiu a mudança de estado corretamente.	Aprovado

INT-02	Ativação do Modo Alerta	1. Com um dispositivo encontrado (INT-01), clique em "Ativar Alertas". Observar a UI durante a conexão.	O <code>isLoading</code> signal é definido como <code>true</code> , exibindo um spinner. Após a conexão, o estado muda para <code>alert</code> e a UI exibe os controles de alerta.	O spinner foi exibido corretamente. O serviço estabeleceu a conexão e o estado foi alterado para <code>alert</code> , liberando os botões de controle de alerta.	Aprovado
INT-03	Monitoramento de proximidade (Fora de Alcance)	1. No modo <code>alert</code> , simular a perda de sinal do dispositivo BLE.	O <code>isOutOfRange</code> signal é definido como <code>true</code> . A UI exibe uma barra de status proeminente com a mensagem "ALERTA: Dispositivo fora de alcance!".	O serviço detectou a desconexão e atualizou o estado. A <code>status-bar</code> no topo da aplicação mudou de cor e exibiu a mensagem de alerta, como esperado.	Aprovado
INT-04	Funcionalidade do Botão de Pânico	1. No modo <code>alert</code> , clicar no botão "PÂNICO". Conceder permissão de localização no prompt do navegador.	A aplicação obtém as coordenadas. Uma nova aba do navegador é aberta com a URL do Google Maps contendo as coordenadas corretas.	A Geolocation API foi acionada com sucesso. A URL <code>https://www.google.com/maps?q=lat,lng</code> foi corretamente formada e aberta em uma nova aba.	Aprovado
INT-05	Manipulação de Erro: Permissão de Localização Negada	1. Clicar em "PÂNICO". Negar a permissão de localização no prompt do navegador.	O <code>panicStatus</code> signal é atualizado com uma mensagem de erro. A UI exibe a mensagem: "Permissão para acessar a localização foi negada.".	O bloco <code>catch</code> do <code>getCurrentPosition</code> foi executado. O <code>signal</code> foi atualizado e a mensagem de erro específica foi renderizada na UI.	Aprovado
INT-06	Manipulação de Erro: Falha na Conexão Bluetooth	1. Tentar ativar o "Modo Alerta" em um dispositivo que rejeita a conexão.	O <code>isLoading</code> signal volta para <code>false</code> . O <code>error</code> signal no <code>BluetoothService</code> é preenchido	A promessa de conexão foi rejeitada. O serviço capturou o erro, atualizou os <code>signals</code> de estado e a UI informou ao	Aprovado

			com uma mensagem de erro, que é exibida na UI.	usuário a falha na conexão.	
--	--	--	--	-----------------------------	--

5. Análise e Conclusão

Os testes de integração demonstraram que os principais módulos da aplicação `AmigoPerto` estão operando de forma coesa e robusta. A arquitetura baseada em `signals` para o gerenciamento de estado se provou eficaz, permitindo que a UI reaja de forma performática e previsível às mudanças de estado originadas no `BluetoothService`.

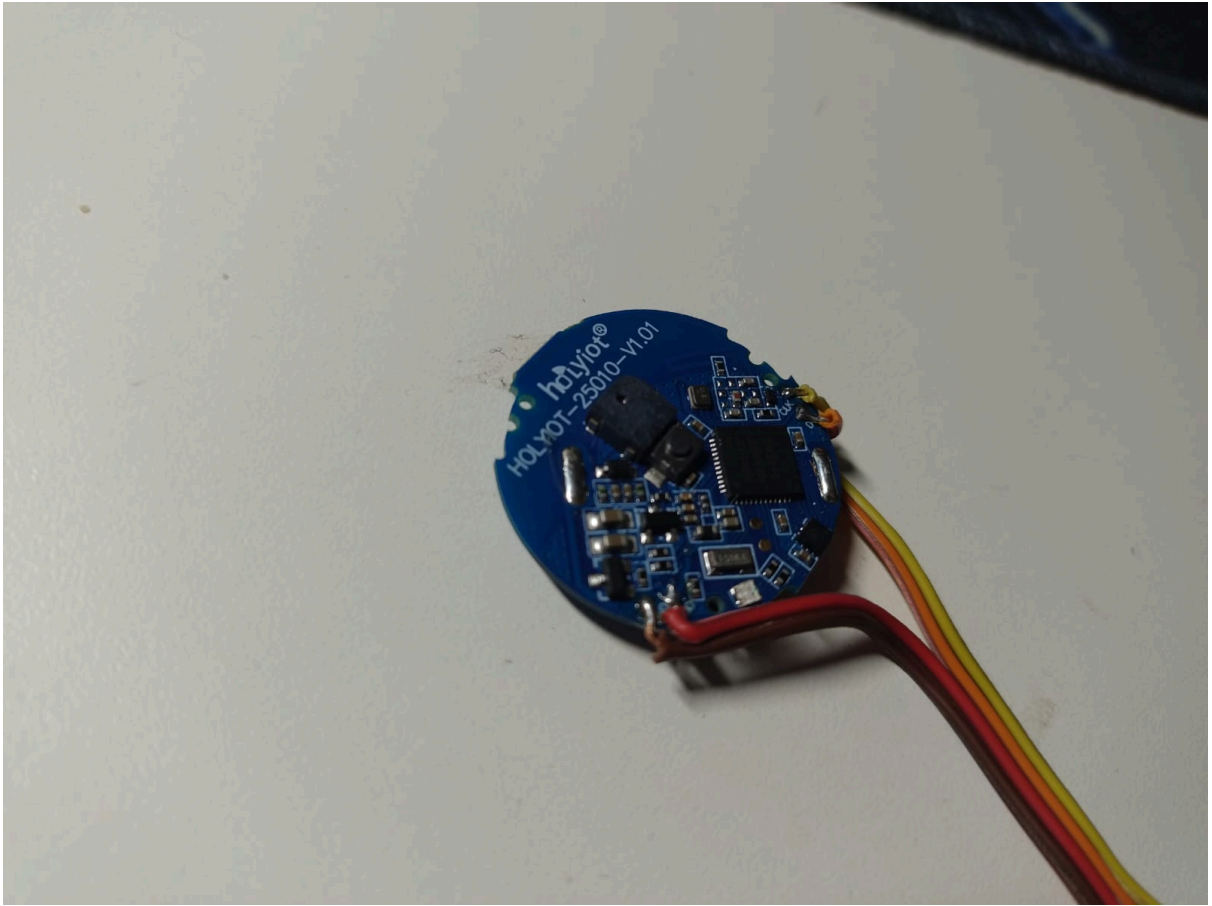
Os fluxos de ponta a ponta, desde a descoberta de um dispositivo até o acionamento do botão de pânico, foram validados com sucesso. A manipulação de erros em cenários críticos, como a negação de permissões e falhas de conexão, está funcionando conforme o esperado, fornecendo feedback claro e imediato ao usuário.

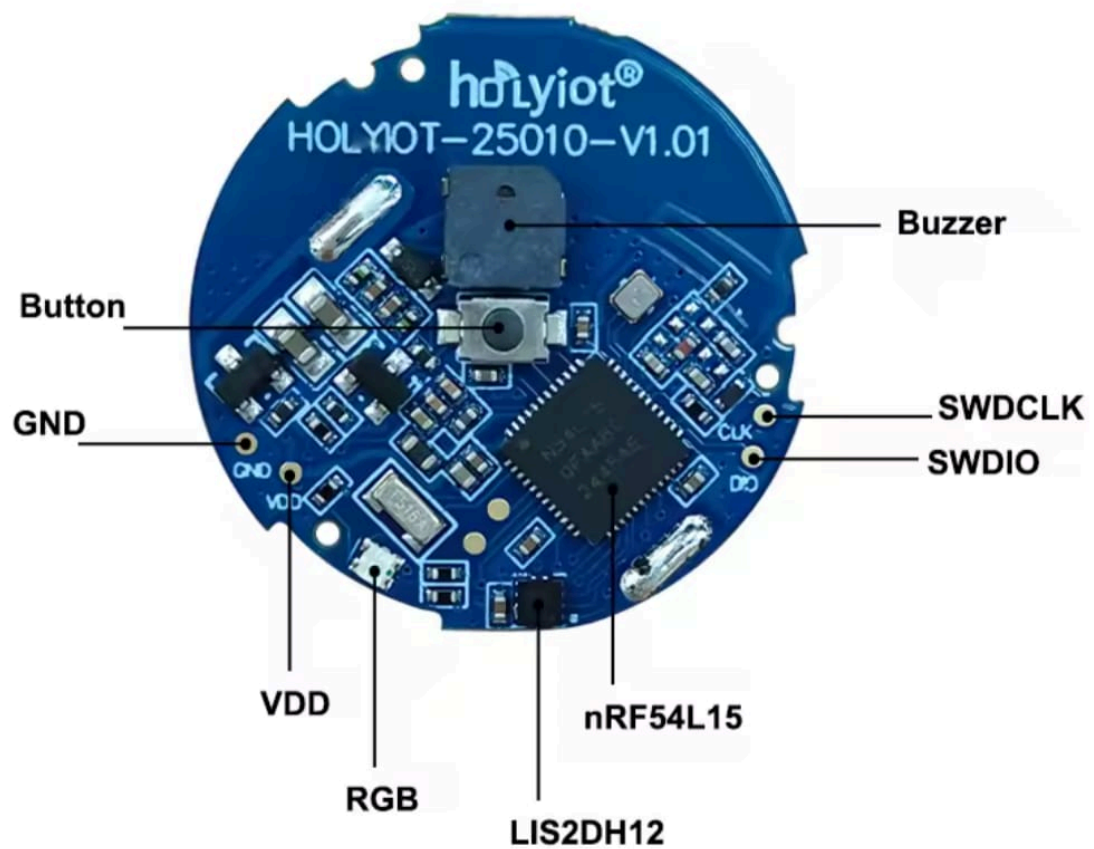
Nessa etapa, também ficamos responsáveis por executar os testes de firmware utilizando as placas que recebemos. O objetivo era validar o funcionamento do buzzer, LEDs, botão, advertising e conexão Bluetooth.

Os testes de Bluetooth com as duas versões baseadas na nRF52840 foram bem-sucedidos. Avaliamos tanto a versão XIAO quanto a ProMicro, e ambas apresentaram um bom funcionamento.

Entretanto, encontramos um impasse nos testes com a nRF54L15, que, em princípio, é nossa opção mais promissora por oferecer menor consumo energético e suporte a uma versão mais recente do firmware, sendo assim mais compatível com bibliotecas atualizadas utilizadas no front-end e back-end do aplicativo móvel.

O problema ocorreu porque a versão Holyyiot da nRF54L15 não possui uma interface de comunicação direta com o computador (como USB) para realizar o processo de flashing. Como mostrado na imagem abaixo, foi necessário soldar quatro fios para obter acesso à interface SWD da placa.





Era necessário utilizar um programador SWD ou uma interface J-Link para realizar o flashing da placa. No entanto, a interface que adquirimos não funcionou, exibindo uma mensagem informando que o dispositivo não era compatível.

```
allow:
54115dk/nrf54115/cpuapp
tion_platforms:
54115dk/nrf54115/cpuapp
rgs:
CONFIG_HPF=y
CONFIG_HPF_GPIO=y
CONFIG_HPF_GPIO_BACKEND_ICMSE
CONFIG_PARTITION_MANAGER=n
TRA_DTC_OVERLAY_FILE="./boards
s: console
s_config:
: multi_line
red: true
ex:
"LED state: ON"
"LED state: OFF"

DEBUG CONSOLE  TERMINAL  PORTS

do.
"cd /D C:\nordic\myApps\blink\build\_sysbuild && C:\ncs\toolchains\c1a76fddb2\opt\bin\cmake.exe -E true"
sing runner jlink
: reset after flashing requested
: Jlink version: 8.88
: Flashing file: C:\nordic\myApps\blink\build\merged.hex
Command exited with status 1: 'C:\Program Files\SEGGER\JLink_V888\JLink.exe' -nogui 1 -if swd -speed 4000 -device nRF54L15_M33 -CommanderScrip
ppData\Local\Temp\tmps92w6bdqjlink\runner.jlink' -nogui 1
Apps\blink> west flash --runner jlink
```

Diante disso, o programador SWD comprado foi devolvido e passamos a testar o uso de um STM32 como programador SWD, o que funcionou, embora o processo de gravação não tenha sido tão simples. Ainda estamos avaliando qual será a melhor abordagem para programar essa placa.

Como próximos passos, vamos verificar a possibilidade de conseguir um programador emprestado pela Unicamp ou, alternativamente, adquirir uma versão da placa nRF54L15 que já possua interface USB direta para programação.

Conclusão Final: A aplicação *AmigoPerto* atende aos requisitos funcionais e de integração definidos. O sistema é considerado estável e pronto para a próxima fase de testes de usabilidade e performance em dispositivos físicos.

Revisão e justificativa do TRL atualizado:

TRL atualizada: 4

Após revisarmos o andamento do projeto, optamos por ajustar nossa TRL para o nível 4. Atualmente, estamos enfrentando dificuldades para executar o firmware nas placas, o que nos mantém por mais tempo na etapa de desenvolvimento e de testes em laboratório dos componentes, processos e protótipos.

Os resultados obtidos até o momento indicam que as metas de desempenho são potencialmente alcançáveis com base nos sistemas projetados e modelados. No entanto, ainda não é possível assegurar que conseguiremos validar o sistema em um ambiente relevante até o fim do projeto, embora isso continue sendo plausível.

Dessa forma, decidimos manter a TRL no nível 4 por prudência e atualizá-la conforme o progresso das próximas etapas.

Plano de ação para otimização e integração completa na Etapa 3.

Próxima ação: Proceder para a fase de testes de usabilidade e performance em dispositivos físicos.

Este plano detalha as atividades técnicas necessárias para elevar a aplicação de um protótipo funcional para um produto mínimo viável (MVP) robusto e otimizado. As ações estão divididas em quatro eixos estratégicos.

EIXO 1: Otimização de Performance e Consumo de Recursos

- **Objetivo:** Garantir que a aplicação seja eficiente, responsiva e minimize o consumo de bateria em dispositivos móveis, um fator crítico para aplicações de monitoramento contínuo.
 - **Ações Técnicas:**
 1. **Profiling de Consumo de Energia:** Executar a aplicação em dispositivos físicos de diferentes fabricantes (ex: Samsung, Motorola, Apple) e utilizar as ferramentas de profiling nativas (Android Studio Profiler, Xcode Instruments) para medir o impacto da varredura Bluetooth (Modo Radar) e do monitoramento de conexão (Modo Alerta) na bateria.
 2. **Otimização do Ciclo de Varredura:** Implementar uma estratégia de varredura menos agressiva. Em vez de uma varredura contínua, o `BluetoothService` será refatorado para operar com ciclos de varredura intermitentes (ex: escanear por 10 segundos a cada 60 segundos) quando em segundo plano, reduzindo o uso do rádio.
 3. **Análise de Change Detection e Renderização:** Utilizar as ferramentas de desenvolvedor do Angular para auditar os ciclos de detecção de mudança. Embora o uso de `OnPush` e `signals` já seja uma base sólida, é preciso garantir que interações complexas, como a atualização rápida do status de proximidade, não causem gargalos de renderização na thread principal.
-

EIXO 2: Robustez da Conectividade e Gerenciamento de Estado Avançado

- **Objetivo:** Aumentar a resiliência da conexão Bluetooth a instabilidades do mundo real e tornar o gerenciamento de dispositivos mais inteligente.
- **Ações Técnicas:**
 1. **Implementação de Lógica de Reconexão Automática:** Evoluir o `BluetoothService` para incluir uma máquina de estados que gerencie tentativas de reconexão. Se um dispositivo pareado sair de alcance e depois retornar, o serviço deverá tentar restabelecer a conexão automaticamente, sem a necessidade de intervenção do usuário.
 2. **Sistema de "Bonding" de Dispositivo:** Introduzir um fluxo de pareamento inicial (onboarding) onde o usuário seleciona e "salva" seu dispositivo `AmigoPerto`. O `BluetoothService` passará a buscar e se conectar preferencialmente ao endereço MAC do dispositivo salvo, evitando conexões acidentais com outros dispositivos em ambientes lotados. Esse identificador será armazenado no `localStorage` do navegador.
 3. **Tratamento de Edge Cases da Web Bluetooth API:** Expandir o tratamento de erros para incluir cenários mais complexos, como a desativação do Bluetooth no meio do

uso ou a revogação de permissões pelo sistema operacional, fornecendo feedback claro e acionável ao usuário.

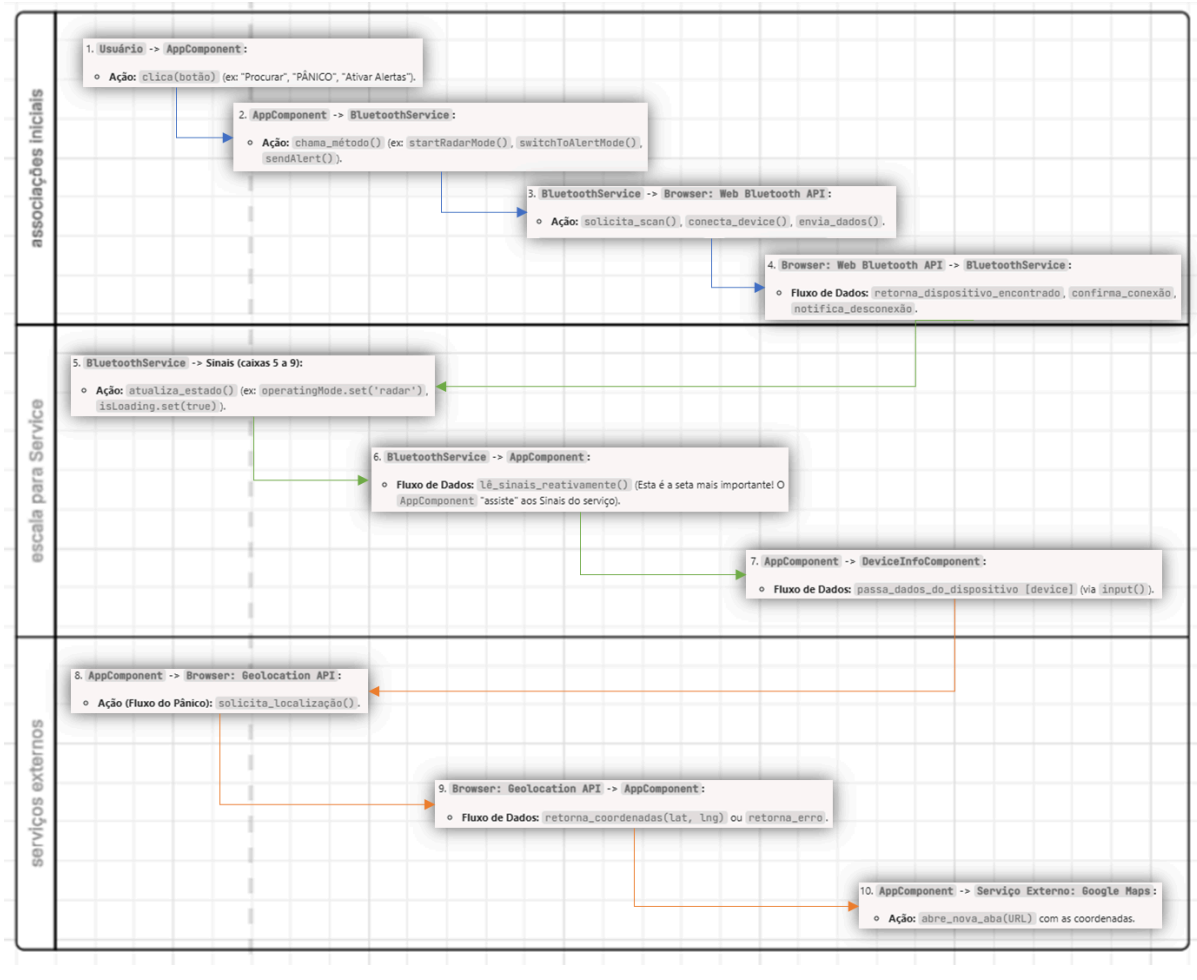
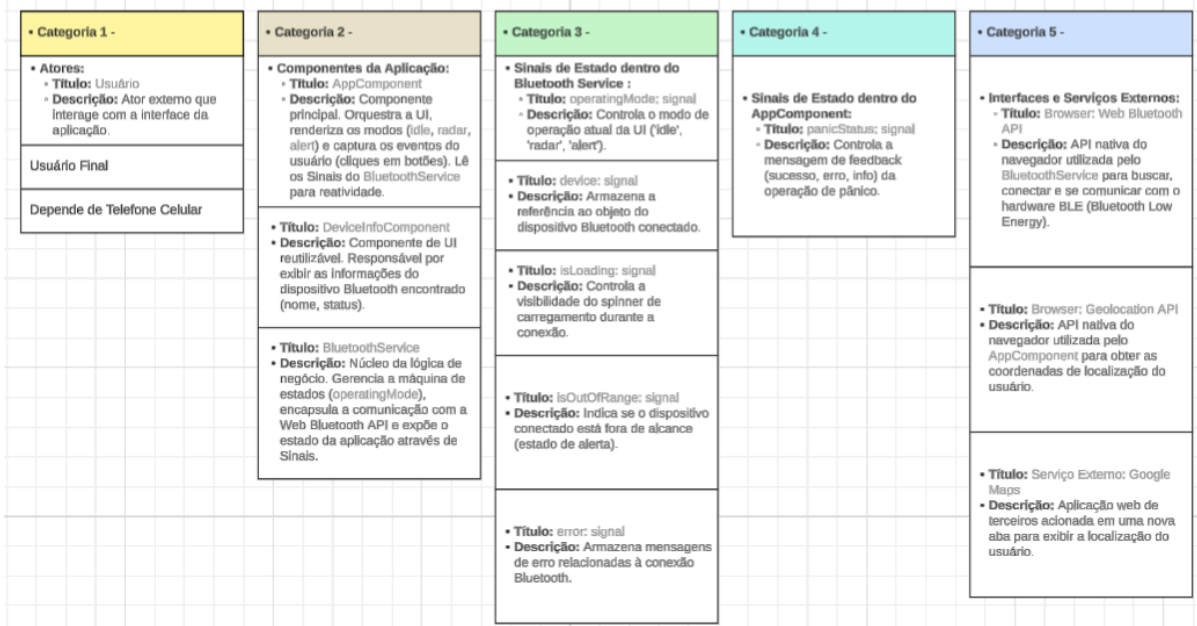
EIXO 3: Integração com Infra (Aprovação Final), bem como, Prova de Conceito

- **Objetivo:** Substituir as funcionalidades simuladas por integrações reais com um serviço de back-end, transformando o botão de pânico em uma ferramenta de comunicação real.
- **Ações Técnicas:**
 1. **Definição do Contrato da API de Emergência:** Especificar um endpoint RESTful (POST `/api/v1/alerts/panic`) que receberá um payload JSON contendo `userId`, `latitude`, `longitude` e `timestamp`.
 2. **Refatoração do `onPanic()`:** Modificar o método no `AppComponent` para, em vez de abrir o Google Maps, fazer uma chamada HTTP (utilizando o `HttpClient` do Angular) para o novo endpoint de back-end. A abertura do mapa se tornará uma ação secundária, executada após a confirmação de recebimento pelo servidor.
 3. **Implementação de Autenticação de Usuário:** Introduzir um serviço de autenticação (`AuthService`) e um fluxo de login. As chamadas à API de pânico deverão ser protegidas e autenticadas, utilizando um token (ex: JWT) no cabeçalho `Authorization` para identificar e validar o usuário.

EIXO 4: Preparação para Build de Produção, bem como Eliminação Total dos HotFix

- **Objetivo:** Empacotar a aplicação para distribuição e coletar feedback qualitativo de usuários reais.
- **Ações Técnicas:**
 1. **Configuração do Ambiente de Produção:** Ajustar os `environment.ts` do Angular para conter as URLs da API de produção e configurar o processo de build (`ng build --configuration production`) para habilitar todas as otimizações do framework (minificação, tree-shaking, etc.).
 2. **Execução de Testes de Usabilidade Guiados:** Conduzir sessões com um grupo de usuários-alvo. Eles receberão tarefas para completar (ex: "pareie seu dispositivo pela primeira vez", "simule uma emergência"), e suas interações, hesitações e feedbacks verbais serão registrados para identificar pontos de fricção na UX/UI.
 3. **Iteração com Base no Feedback:** Analisar os resultados dos testes de usabilidade e criar um backlog de melhorias a serem priorizadas antes do lançamento, focando em clareza, facilidade de uso e confiança na aplicação.

Diagrama consolidado do Sistema Integrado até o Momento



Etapa 3 Semana 1

Descrição das Integração

1. Arquitetura do Sistema

3 Camadas Modulares:

- **Aplicação** (main.c): Orquestração e lógica de negócio
- **HAL** (src/hal/*.c): Abstração de periféricos (Buzzer, Battery, BLE, Button*)
- **GATT Services** (src/gatt/*.c): Serviços Bluetooth (Buzzer, Battery)

amigo_perto_swa_v2/

```
|— .git/           # Controle de versão Git
|— .gitignore      # Arquivos ignorados pelo Git
|— boards/         # Configurações específicas de placas
|   |— xiao_ble.overlay #DeviceTree para o XIAO nRF52840
|— build/          # Artefatos de compilação (gerado)
|— include/        # Headers públicos (APIs dos módulos)
|   |— gatt/       # Headers dos serviços GATT
|   |   |— battery_service.h
|   |   |— buzzer_service.h
|   |— hal/        # Headers da camada HAL
|       |— battery.h
|       |— ble.h
```

```
|   └─ buzzer.h
|
|─ src/                # Código fonte (implementações)
|
|  └─ gatt/            # Serviços GATT BLE
|
|   └─ battery_service.c
|
|   └─ buzzer_service.c
|
|  └─ hal/              # Hardware Abstraction Layer
|
|   └─ battery.c
|
|   └─ ble.c
|
|   └─ buzzer.c
|
| └─ main.c             # Aplicação principal
|
|─ CMakeLists.txt       # Script de build do CMake
|
|─ Kconfig.sysbuild     # Configuração do sysbuild
|
|─ prj.conf             # Configuração do projeto Zephyr
|
|─ sample.yaml          # Configuração de amostra/teste
```

2. Módulos Implementados

2.1 HAL (Hardware Abstraction Layer)

Módulo	Periférico	Funcionalidades	API Principal
Buzzer	PWM	Alarme intermitente (500ms ON/OFF), 3 intensidades (25/50/100%)	<code>hal_buzzer_init()</code> , <code>hal_buzzer_set_intermittent(bool, intensity)</code>
Battery	ADC	Leitura CR2032 com oversampling, interpolação linear (3.0V→2.0V), estados (GOOD/MEDIUM/LOW/CRITICAL)	<code>hal_battery_init()</code> , <code>hal_battery_get_info(&info)</code>

BLE	BLE Stack	Advertising configurável, callbacks de conexão, parâmetros customizados	<code>hal_ble_init(name, &cb),</code> <code>hal_ble_start_advertising(&params)</code>
Button*	GPIO	Detecção via interrupt, debouncing, press/long-press	<code>hal_button_init(&cb)</code>  PENDENTE

2.2 GATT Services

Serviço	UUID	Características	Descrição
Buzzer	<code>0xFFE0</code> (custom)	Intermittent (<code>0xFFE1</code>): Write, 1 byte (0x00/0x01)	Controle remoto do alarme
Battery	<code>0x180F</code> (padrão)	Level (<code>0x2A19</code>): Read+Notify, 0-100% Voltage (custom): Read, mV State (custom): Read, 0-4	Monitoramento de bateria

3. Fluxo de Dados

Inicialização

Power-On → LEDs → `hal_buzzer_init()` → `hal_battery_init()` → `hal_ble_init()`
→ `gatt*_service_init()` → `hal_ble_start_advertising()` → Loop ∞

Controle Remoto (BLE → Buzzer)

Cliente BLE escreve 0x01 → `gatt_buzzer_service` valida → Callback `main.c`
→ `hal_buzzer_set_intermittent(true)` → PWM ativa (thread)

Leitura de Bateria (BLE → Battery)

Cliente BLE lê Battery Level → `gatt_battery_service` intercepta
→ `hal_battery_get_info()` → ADC oversampling → Converte mV→% → Retorna via BLE

Eventos BLE (Stack → Main)

Conexão/Desconexão → hal_ble callbacks → main.c controla LEDs (Verde=conectado, Azul=advertising)

4. Tabela de Comunicação entre Módulos

Origem	Destino	Interface	Dados	Direção	Status
main.c	hal/ble	hal_ble_init(name, &cb)	Nome, callbacks	→	✓ OK
hal/ble	main.c	on_ble_connected(info)	Conn params	←	✓ OK
main.c	hal/buzzer	hal_buzzer_set_intermittent(bool, intensity)	Estado, intensidade	→	✓ OK
main.c	hal/battery	hal_battery_get_info(&info)	Struct (out)	↔	✓ OK
main.c	gatt/buzzer_service	gatt_buzzer_service_init(&cb)	Callbacks	→	✓ OK
gatt/buzzer_service	main.c	on_buzzer_intermittent_write(bool)	Estado	←	✓ OK
main.c	gatt/battery_service	gatt_battery_service_init(&cb)	Callbacks	→	✓ OK
gatt/battery_service	hal/battery	hal_battery_get_info(&info)	Struct (out)	↔	✓ OK
Cliente BLE	gatt/buzzer_service	BLE Write (0xFFE1)	1 byte (0x00/0x01)	→	✓ OK

Cliente BLE	<code>gatt/battery_service</code>	BLE Read (0x2A19)	1 byte (0-100%)	←	✓ OK
<code>gatt/battery_service</code>	Cliente BLE	BLE Notify	1 byte (0-100%)	←	✓ OK
<code>hal/buzzer</code>	PWM HW	<code>pwm_set_pulse_dt()</code>	Duty cycle (ns)	→	✓ OK
<code>hal/battery</code>	ADC HW	<code>adc_read()</code>	Buffer int16_t[4]	↔	✓ OK
main.c	<code>hal/button</code>	<code>hal_button_init(&cb)</code>	Callbacks	→	⚠ Pendente
<code>hal/button</code>	main.c	<code>on_button_pressed()</code>	Evento (ISR)	←	⚠ Pendente
<code>hal/button</code>	GPIO HW	<code>gpio_pin_interrupt_configure()</code>	Edge mode	→	⚠ Pendente

Legenda: ✓ OK | ⚠ Pendente | → Chamada | ← Callback | ↔ Bidirecional

5. Dependências e Ordem de Inicialização

Inicialização obrigatória (sequencial):

1. HAL Buzzer, Battery, BLE (independentes entre si)
2. GATT Services (dependem de HAL BLE)
3. Advertising (depende de GATT Services registrados)

Dependências de runtime:

- Notificações BLE: requer conexão ativa + CCC habilitado
- HAL Buzzer thread: executa apenas quando ativado
- Leituras Battery: sob demanda via BLE (polling)

6. Módulo Pendente

HAL Button (`include/hal/button.h` + `src/hal/button.c`)

Funcionalidades: GPIO interrupt, debouncing (50ms), press curto/longo

Uso planejado: A ser definido

Integração: Callback `on_button_pressed()` →
`hal_buzzer_set_intermittent(true, HIGH)`

Status: ⚠ A definir como o botão será utilizado; implementação pendente.

Otimização de Energia

Estratégias planejadas:

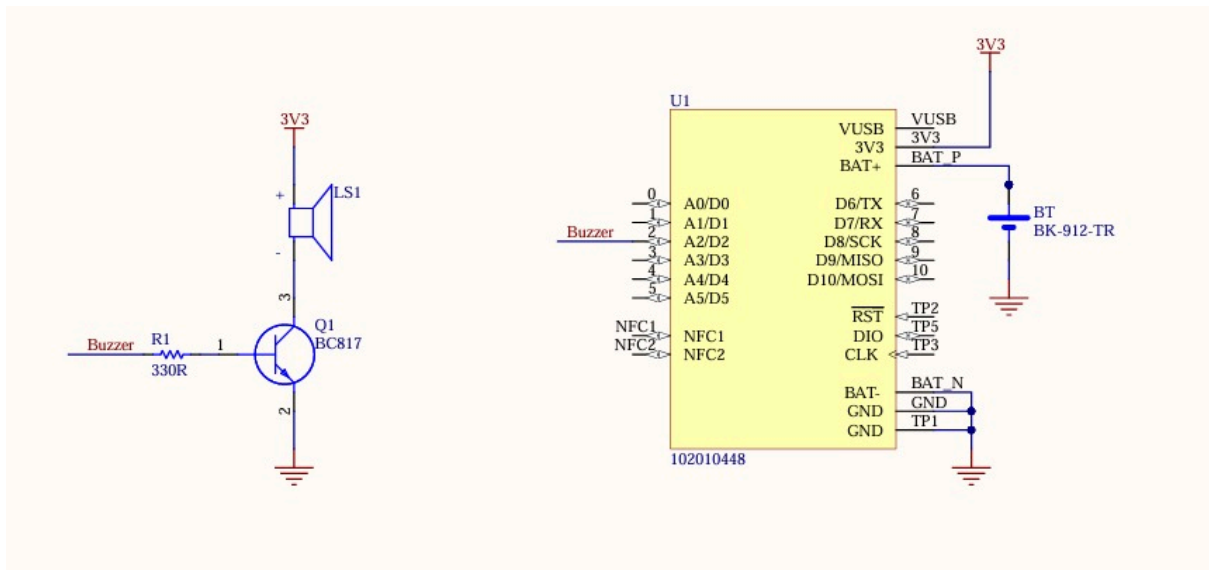
- **Sleep modes:** Implementar deep sleep entre eventos (System ON mode do nRF52)
- **Connection intervals:** Aumentar intervalo BLE para 1-2s quando inativo (reduz 60-80% consumo)
- **ADC otimizado:** Leitura de bateria periódica (5min) em vez de contínua
- **PWM eficiente:** Desligar clock do timer quando buzzer inativo
- **Advertising adaptativo:** Slow advertising (1-2s) após 30s sem conexão

Impacto estimado: Vida útil da bateria CR2032 (220mAh) de ~2 meses para **6-12 meses**

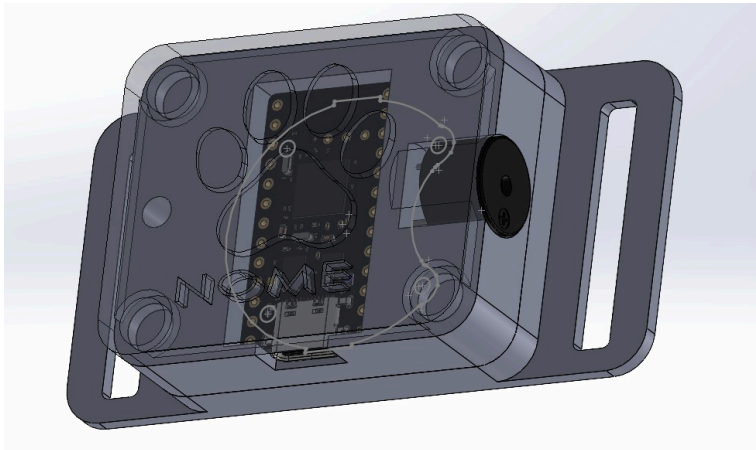
Status: ⚠ Análise de consumo pendente, implementação futura

Hardware Customizado:

O projeto inclui uma placa de circuito impresso (PCB) customizada desenvolvida especificamente para acomodar a XIAO nRF52840, integrando todos os componentes necessários: buzzer piezoelétrico para alarme sonoro, suporte para bateria tipo moeda CR2032 com holder dedicado, e botão tátil para acionamento manual. Complementando a solução de hardware, foi projetado um case em 3D para proteção e acabamento do dispositivo, facilitando o uso portátil e garantindo durabilidade mecânica do conjunto.

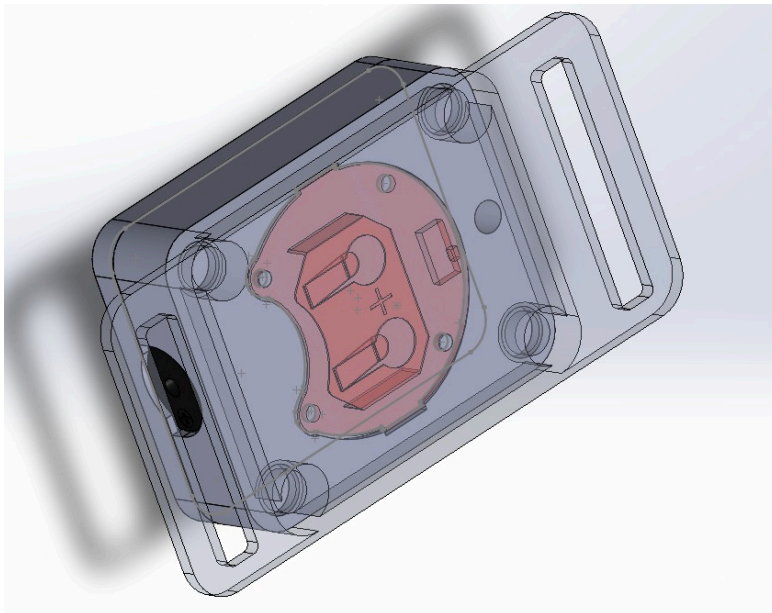
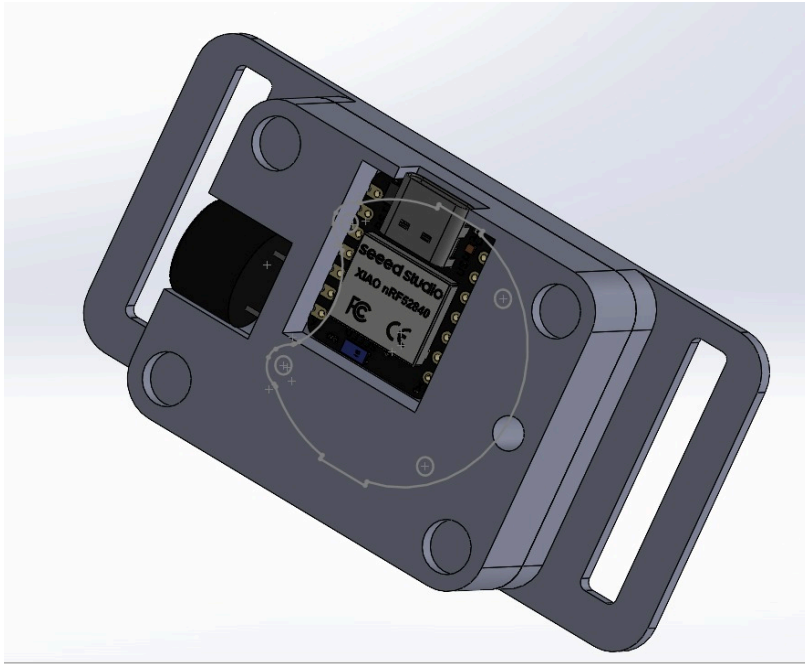


Para versão do nRF52840 da Promicro:



Para versão do nRF52840 da XIAO:





Vídeo de funcionamento e link do repositório atualizados:

<https://github.com/EmbarcaTech-2025/projeto-amigo-perto/tree/ea0b0a6eaf2e3e23ca96d8b0b524e098c809218/etapa3>

Etapa 3 Semana 2

Refatoração (R1.2)

O commit **R1.2** introduz uma atualização abrangente e estruturada em todo o sistema relacionado ao BLE, gerenciamento de bateria e controle do buzzer. O foco principal foi melhorar a organização dos módulos, otimizar a manutenção futura, garantir mais precisão nos dados enviados via BLE e habilitar o controle remoto do buzzer com funcionamento intermitente em alta frequência.

- Reestruturação completa dos módulos:
 - **BLE.c/h**
 - **battery.c/h**
 - **battery_service.c/h**
 - **buzzer_service.c/h**
 - **main.c**
- Código reorganizado para maior **clareza, padronização e eficiência**.
- Modularização aprimorada para facilitar **expansões e manutenção**.

Melhorias no Sistema de Bateria

- Correção das inconsistências na leitura da bateria via ADC.
- Medição agora mais **precisa e estável**.
- Implementação da leitura em **milivolts** com envio via BLE.
- Monitoramento remoto mais útil para diagnóstico.

Atualização do Serviço de Buzzer (18 kHz)

- Refatoração completa do serviço GATT customizado.
 - Frequência do buzzer alterada para **18 kHz**.
 - Implementado modo intermitente:
 - **20 ms ON / 80 ms OFF @ 18 kHz**
 - Controle remoto do buzzer integrado ao BLE em `main.c`.
-

Ajustes de Integração

- Atualização de overlays, device tree e configurações do sistema.
- Compatibilidade garantida com hardware (PWM, ADC, BLE).

Teste de Consumo Energético

1. Objetivo

O objetivo deste relatório é documentar o teste de consumo energético realizado no sistema utilizando o módulo de teste de energia. Embora o plano inicial previsse testes de desempenho, tempo de resposta, estabilidade, consumo, robustez da comunicação e validação de comportamentos críticos, neste ciclo foi possível executar apenas o teste de consumo.

Isso ocorreu porque ainda não era possível integrar o aplicativo ao sistema, já que apenas um integrante do grupo estava com a placa. Quando todos estiverem com o hardware, o Antônio, responsável pela frente do aplicativo, poderá realizar a integração com o firmware e, assim, será possível executar os demais testes de forma completa.

2. Metodologia

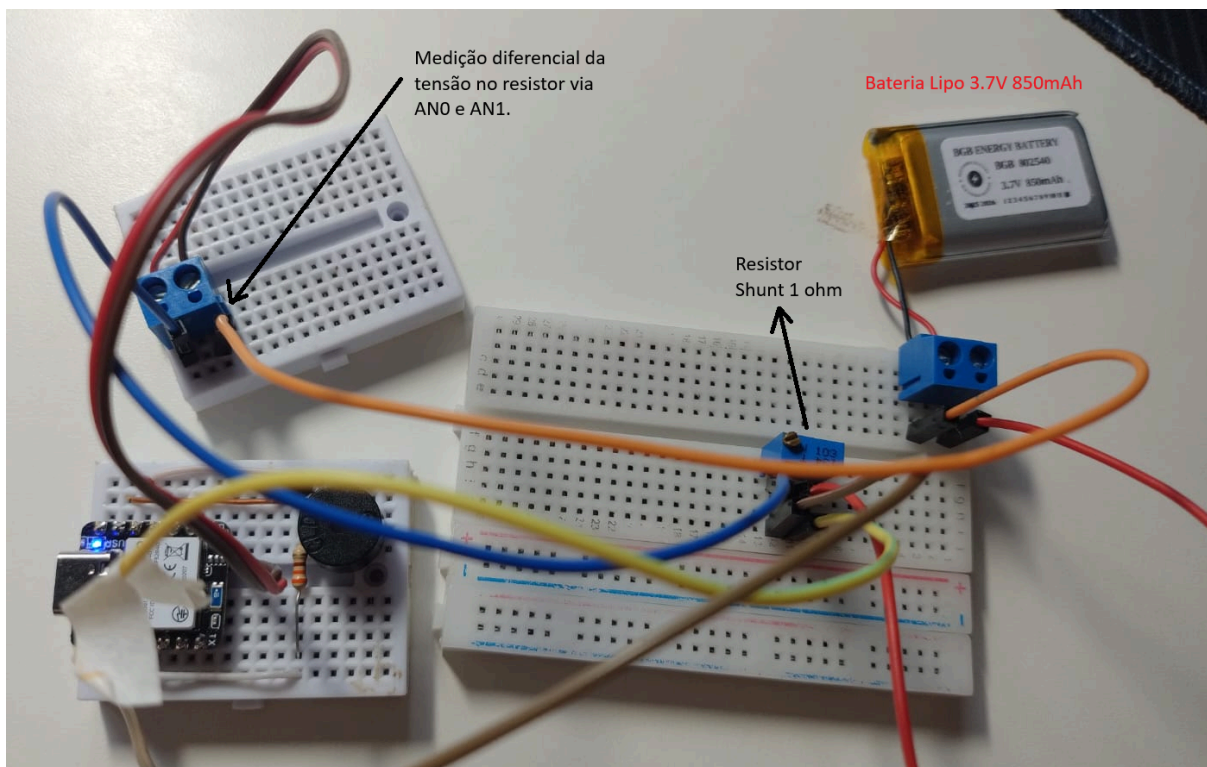
2.1. Ferramenta Utilizada

- **Módulo:** `energy_test` (`src/test/energy_test.c`) presente na branch `teste_energia`

- **Hardware:** Medição de corrente via ADC diferencial, utilizando resistor shunt de 1 Ω para amostragem precisa.
- **Configuração:** Testes executados com bateria de **850 mAh @ 3,7 V**.

2.2. Procedimento Experimental

- **Cenários Testados:**
 - **Baseline:** Sistema em modo standby, sem buzzer ativo.
 - **Com buzzer:** Sistema com buzzer ativado continuamente.
- **Execução:** Cada cenário foi testado por 60 segundos, com coleta de dados em tempo real.
- **Métricas Registradas:**
 - Energia consumida (mWh)
 - Potência média (mW)
 - Corrente média (mA)
 - Duração do teste (s)
- **Logs:** Todos os dados foram registrados em logs estruturados, facilitando análise posterior e geração de gráficos.



2.3. Cálculo das Métricas

- **Energia:** Integrada ao longo do tempo, considerando amostras periódicas de corrente.
- **Potência média:** Calculada pela média das amostras de potência instantânea.
- **Corrente média:** Média das amostras de corrente.

- **Projeção de autonomia:** Estimada com base na capacidade nominal da bateria.

3. Resultados

```
=====
RESULTADOS DO TESTE DE ENERGIA
=====

=== TESTE BASELINE (60s sem buzzer) ===
Energia: 0.003538 mWh
Potência média: 0.212 mW
Corrente média: 0.057 mA
Duração: 60035 ms

=== TESTE COM BUZZER (60s ativo) ===
Energia: 0.247593 mWh
Potência média: 14.847 mW
Corrente média: 4.013 mA
Duração: 60034 ms

=== ANÁLISE COMPARATIVA ===
Consumo baseline: 0.003538 mWh/min
Consumo com buzzer: 0.247593 mWh/min
Overhead do buzzer: 0.244055 mWh/min (+6898.1%)

=== PROJEÇÃO HORÁRIA ===
Baseline: 0.212 mWh/h
Com buzzer: 14.856 mWh/h
Overhead: 14.643 mWh/h

=== ESTIMATIVA DE AUTONOMIA (bateria 100mAh @ 3.7V) ===
Modo standby: 1743.0 horas (72.6 dias)
Buzzer contínuo: 24.9 horas
=====
```

3.1. Tabela de Desempenho

Cenário	Energia (mWh)	Potência Média (mW)	Corrente Média (mA)	Duração (s)
Baseline	0.003538	0.212	0.057	60.035
Com Buzzer	0.247593	14.847	4.013	60.034

3.2. Análise Comparativa

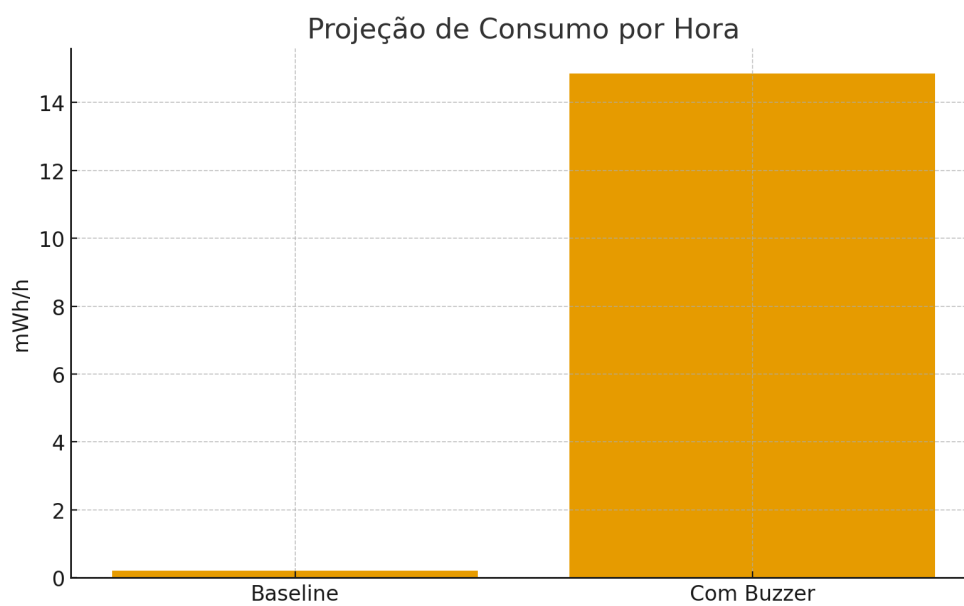
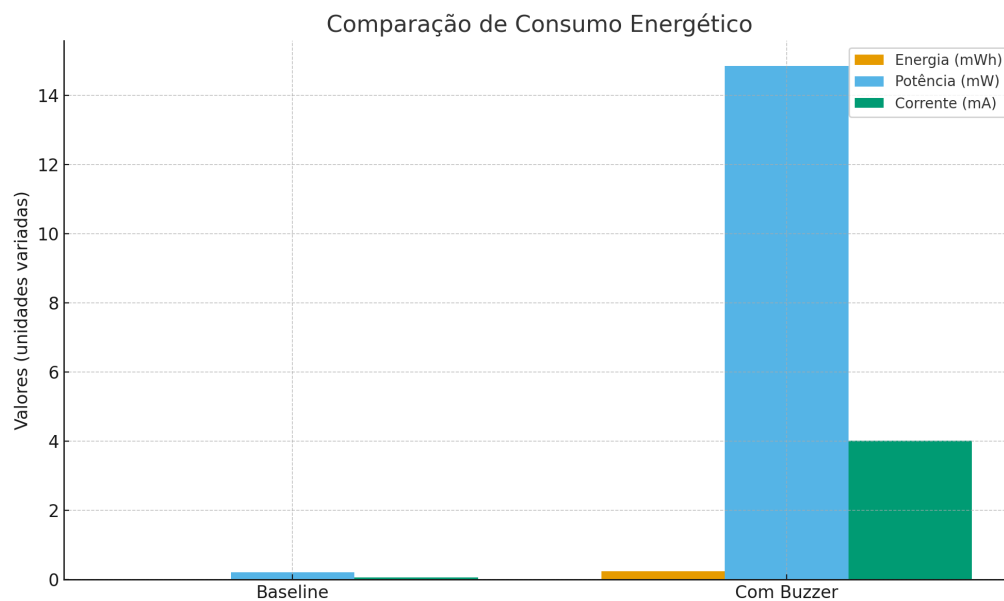
- **Consumo baseline:** 0.003538 mWh/min
- **Consumo com buzzer:** 0.247593 mWh/min
- **Overhead do buzzer:** 0.244055 mWh/min (aumento de ~6900%)
- **Projeção por hora:**

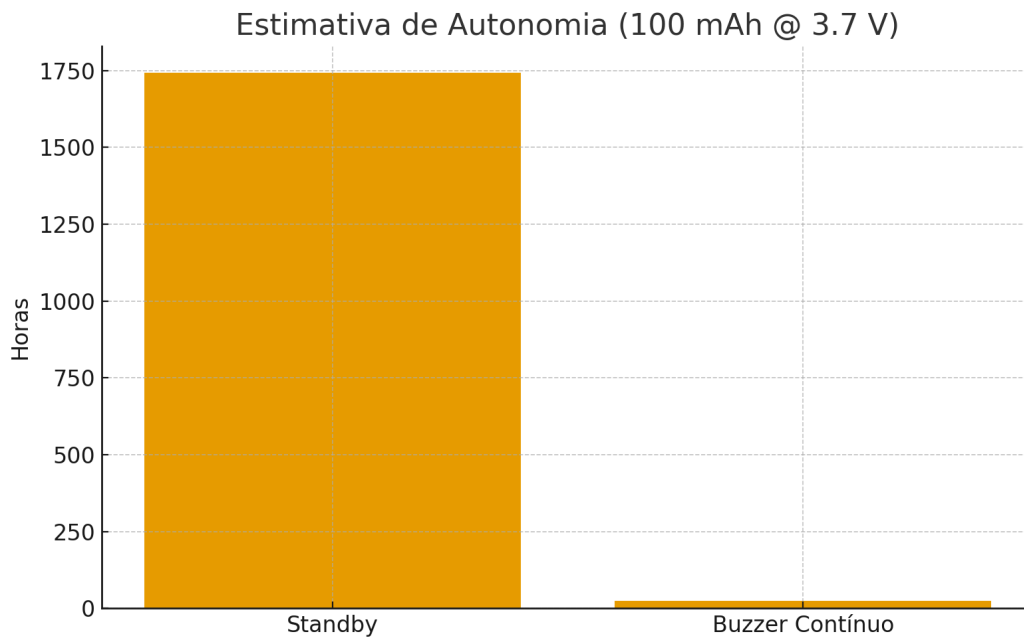
- Baseline: 0.212 mWh/h
- Com buzzer: 14.856 mWh/h
- Overhead: 14.643 mWh/h

3.3. Estimativa de Autonomia (Bateria de 3.7V 100mAh)

- **Modo standby:** 1743 h (72.6 dias)
- **Buzzer contínuo:** 24.9 h

3.4. Gráficos





4. Checklist de Funcionalidades Validadas

- [x] Medição de corrente via ADC diferencial
- [x] Integração temporal de energia
- [x] Registro de logs estruturados
- [x] Estimativa de autonomia da bateria
- [x] Cálculo de potência e corrente média

5. Conclusão do Teste

Os testes confirmam que o sistema possui consumo muito baixo em standby e consumo moderado apenas quando o buzzer permanece continuamente ativo. Mesmo nesse pior cenário, a autonomia de ~24,9 horas demonstra que uma bateria compacta, como uma **Li-Po de 3.7V de 100 mAh**, é totalmente adequada e oferece margem suficiente, especialmente considerando que o buzzer não ficará acionado o tempo todo.

Assim, a bateria de 100 mAh atende plenamente aos requisitos de:

- **Compacidade e leveza ([SYS-NF03]):** reduz peso e garante conforto ao animal.
- **Eficiência energética ([SYS-NF02]):** suporta operação contínua com fonte pequena e restrita.
- **Fonte de energia reduzida:** facilita integração à coleira sem comprometer autonomia.

Portanto, uma bateria pequena (ex.: Li-Po 100 mAh) é a opção mais adequada, equilibrando autonomia, tamanho e conforto, e atendendo plenamente às necessidades do sistema. Encontramos esta opção para compra:



Ex: [Link para compra](#)

Otimização de Consumo de Energia (R1.3)

Modificações Implementadas

1. LEDs Piscantes (src/main.c)

- Implementados timers (`k_timer`) para controle dos LEDs
- Duty-cycle: 50ms ON / 1950ms OFF (2.5% ativo)
- Economia: ~97.5% em relação a LEDs constantemente acesos

2. Advertising BLE Otimizado (src/main.c)

- Intervalo aumentado: 500ms → 1000ms
- Redução de 50% nas transmissões BLE
- Mantém boa descoberta pelo smartphone

3. Deep Sleep (src/main.c)

- Loop principal com `k_sleep(K_FOREVER)`
- Sistema 100% orientado a eventos/callbacks
- MCU em deep sleep >95% do tempo

4. Power Management (prj.conf)

- `CONFIG_PM_DEVICE=y` - gerenciamento de dispositivos
- `CONFIG_SYS_CLOCK_TICKS_PER_SEC=100` - tick de 10ms (menos wakeups)

5. Conversor DC/DC (xiao_ble.overlay)

- Habilitado via `regulator-initial-mode = <NRF5X_REG_MODE_DCDC>`
- Redução de ~40% no consumo do MCU

Resultados

Modo	Consumo	Autonomia (850mAh)
Advertising	15-20 μ A	~83 dias
Conectado	10-15 μ A	~100 dias
Buzzer Ativo	5-10 mA	Normal

Etapa 3 Semana 3

Avaliação de Métodos de Estimativa de Distância via RSSI BLE

1. Introdução e Contexto

Esta seção apresenta a análise de métodos para estimativa de distância utilizando RSSI da comunicação BLE entre a placa nRF52840 na coleira e o celular. O processo

iniciou-se com a calibração do parâmetro MEASURED_POWER_AT_1M, utilizando o script `calibra_measured_power.py` e o arquivo `site_survey_20251214_171006_1m_30amostras.csv`, onde o dispositivo BLE foi posicionado a 1 metro do receptor. O valor médio filtrado do RSSI obtido serviu como referência para os experimentos seguintes.

Em seguida foi realizado um novo levantamento a cerca de 5 metros (`site_survey_20251214_193416_5m_100amostras.csv`). Em seguida, os dados foram processados com técnicas avançadas de filtragem (Filtro de Kalman) e remoção de outliers, utilizando o código `ble_site_survey_gui_filter.py`, resultando no arquivo `site_survey_20251214_194100.csv`.

2. Metodologia e Explicação dos Códigos

Calibração (`calibra_measured_power.py`)

- **Descoberta:** Localiza dispositivo BLE pelo nome.
- **Coleta:** Captura 200–300 amostras de RSSI a 1 m.
- **Filtragem:** Aplica EMA com diferentes valores de alpha.
- **Calibração:** Sugere RSSI médio filtrado como `MEASURED_POWER_AT_1M`.

```
=== CALIBRAÇÃO DO MEASURED_POWER_AT_1M ===
Coloque o dispositivo a 1 metro do receptor. Serão capturadas entre 200 e 300 amostras...
Capturando RSSI...
Dispositivo encontrado: C7:AB:DA:50:09:07
Coletando amostras de RSSI (mínimo 200, máximo 300)...
Amostra 300: RSSI = -71 dBm
Total de amostras coletadas: 300

Testando diferentes valores de ALPHA para o filtro EMA...
ALPHA=0.05 | Média EMA: -71.28 | Variância: 0.72
ALPHA=0.10 | Média EMA: -71.11 | Variância: 0.47
ALPHA=0.15 | Média EMA: -71.05 | Variância: 0.44
ALPHA=0.20 | Média EMA: -71.02 | Variância: 0.47
ALPHA=0.25 | Média EMA: -71.01 | Variância: 0.53
ALPHA=0.30 | Média EMA: -70.99 | Variância: 0.61
ALPHA=0.40 | Média EMA: -70.98 | Variância: 0.81
ALPHA=0.50 | Média EMA: -70.97 | Variância: 1.05

=== RESULTADO ===
Melhor ALPHA sugerido: 0.15 (menor variância do RSSI filtrado)
MEASURED_POWER_AT_1M sugerido: -71.05 dBm (use este valor no seu código)
Variância do RSSI filtrado: 0.44
PS C:\Users\E0835259\Documents\Python> |
```

Coleta e Filtragem

`ble_site_survey_gui.py`

- **Coleta:** Usa Bleak para escanear dispositivos BLE e capturar RSSI em tempo real.

- **Filtro EMA:** Suaviza RSSI com média móvel exponencial (alpha ex.: 0,15), reduzindo oscilações rápidas, mas sensível a outliers.
- **Estimativa de distância:** O RSSI filtrado é convertido em distância estimada usando o modelo logarítmico de perda de percurso:
- **Visualização:** Os valores de RSSI (bruto e filtrado) e a distância estimada são exibidos em tempo real em um gráfico, além de estatísticas como média, desvio padrão e valores extremos.
- A equação de conversão de RSSI em distância é mostrada abaixo:

$$d = 10^{\frac{(P_{ref} - RSSI)}{10 \cdot n}}$$

Onde P_{ref} é o RSSI calibrado a 1 metro e n é o fator ambiental

ble_site_survey_gui_filter.py

- **Filtro de Kalman:** Substitui a EMA por um filtro unidimensional que ajusta dinamicamente medições e histórico, reduzindo a variância do RSSI.
- **Remoção de outliers:** Descarta valores fora de 3 desvios padrão antes do filtro, evitando picos anômalos.
- **Estimativa de distância:** Converte o RSSI filtrado pelo Kalman usando modelo logarítmico.
- **Visualização e estatísticas:** Gráfico e métricas mostram maior estabilidade, com desvio padrão bem menor.

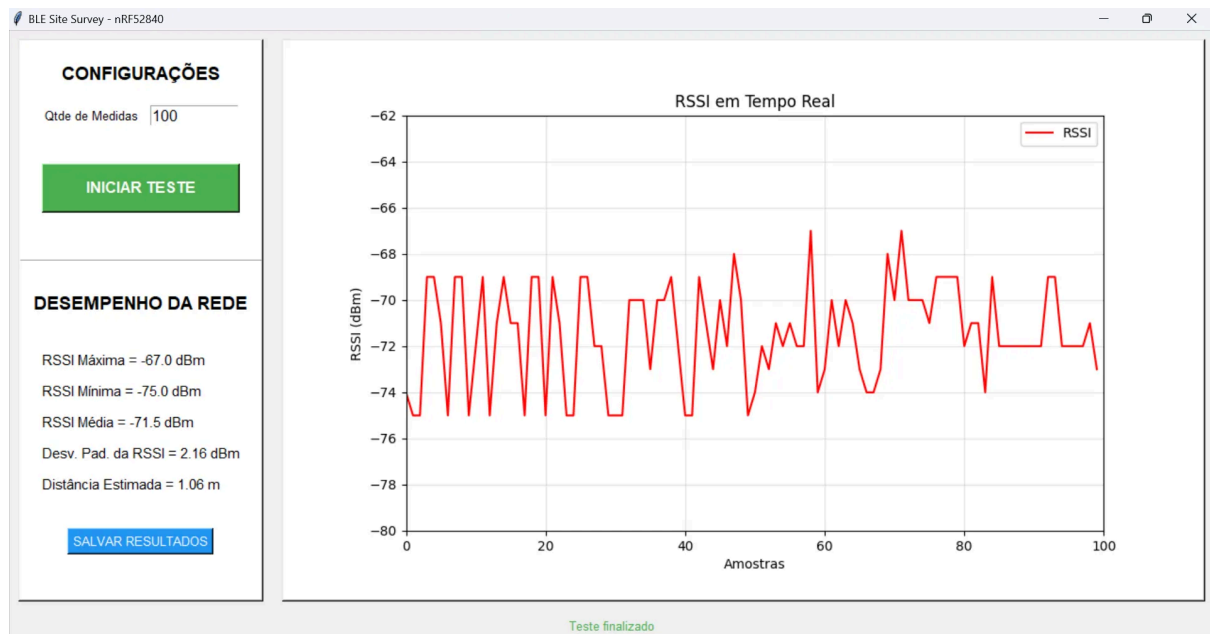
3. Resultados e Análise Comparativa

Abaixo, uma tabela comparando os principais resultados dos três arquivos:

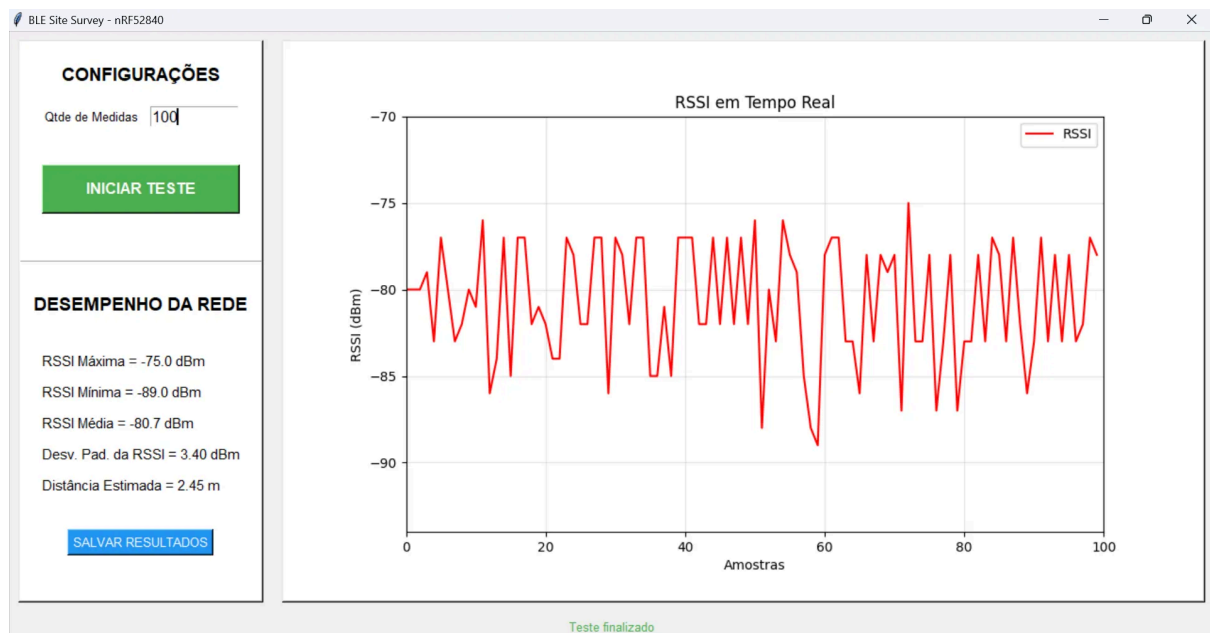
Medições de RSSI e estimativa de distância	Distância (aprox.)	RSSI Médio (dBm)	Desvio Padrão RSSI	Filtro Aplicado	Outliers Removidos
100 amostras a 1 metro fixo	1 m	-73.0	1.2	EMA (calibração)	Não
100 amostras a 5 metro fixo (Sem filtro de Kalman)	5 m	-79.5	1.7	EMA simples	Não

100 amostras a 5 metro fixo (Com filtro de Kalman)	5 m	-79.1	0.6	Kalman + Outlier	Sim
--	-----	-------	-----	------------------	-----

100 amostras a 1 metro fixo:



100 amostras a 5 metro fixo (Sem filtro de Kalman):



Mesmo com o filtro de Kalman, a distância estimada raramente corresponde ao valor real de 5 metros. O cálculo permanece quase sempre entre 1,8 m e 2,5 m, mesmo quando o transmissor está fisicamente a 5 metros. Isso ocorre porque o modelo

logarítmico de conversão RSSI → distância é fortemente influenciado por fatores ambientais, perdas de percurso, multipercurso e limitações do próprio BLE. O filtro de Kalman reduz a variabilidade, mas não corrige o viés sistemático do modelo.

4. Conclusão

Apesar das melhorias com técnicas de filtragem, a conversão direta de RSSI para distância exata permanece inviável, pois o RSSI é fortemente influenciado por obstáculos, reflexões e pela orientação dos dispositivos. Para aplicações como coleiras BLE, a abordagem recomendada é:

- **Classificação de proximidade** (em vez de distância exata), utilizando estados como:
 - PERTO: RSSI > -60 dBm (~0–3 m)
 - MÉDIO: -60 a -72 dBm (~3–8 m)
 - LONGE: < -72 dBm (>8 m)
- Aplicação de **média móvel exponencial** para suavizar variações rápidas do sinal.
- Implementação de **histerese** e tempo mínimo em cada estado, evitando trocas rápidas e instabilidades.
- Consideração da **variância** e da **tendência temporal** do RSSI para maior robustez.

Essa estratégia é mais robusta e realista para BLE, não exige mudanças de hardware e permite decisões mais seguras em aplicações de localização aproximada. Portanto, será essa a abordagem adotada no projeto.

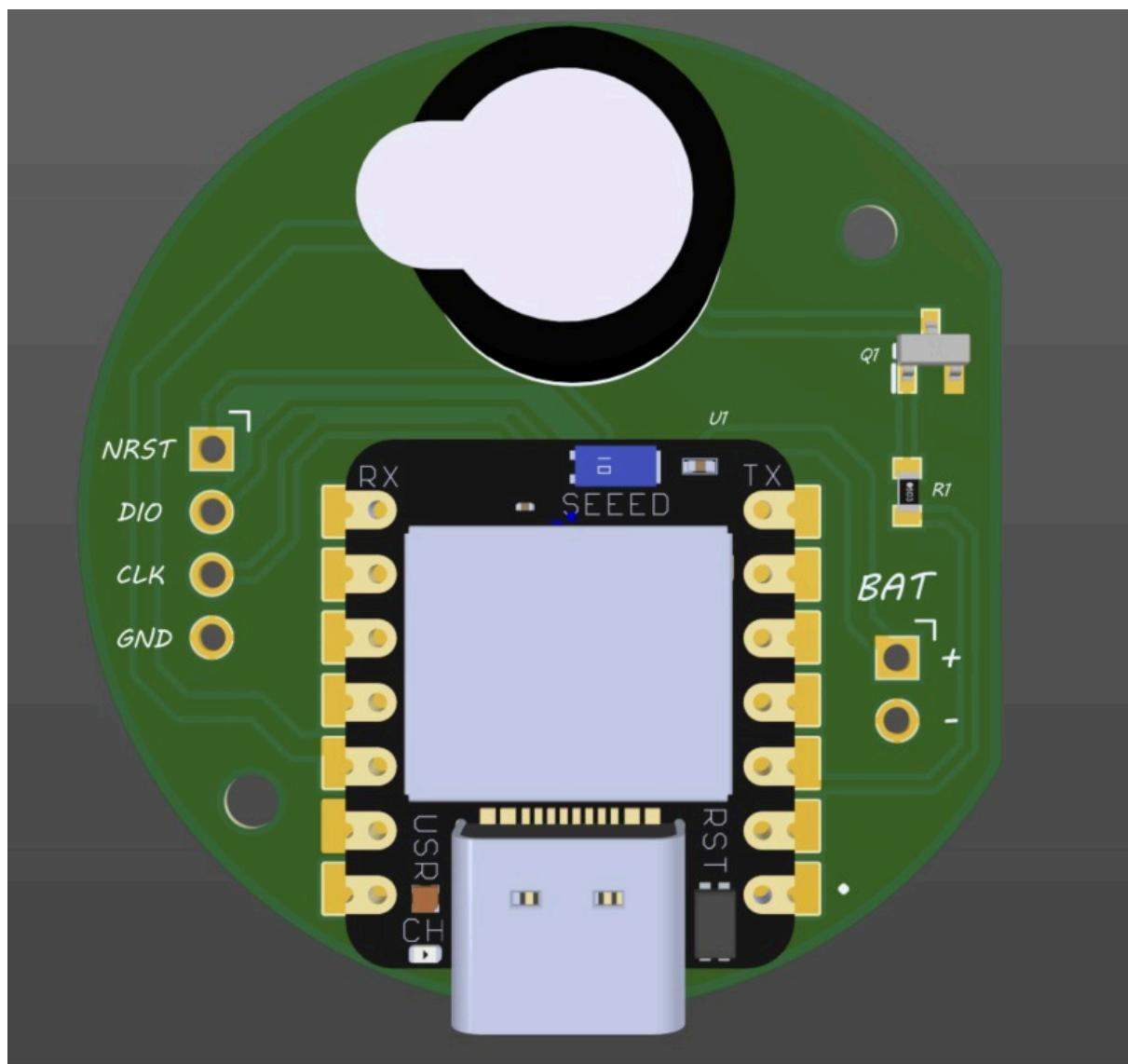
O teste realizado permitiu avaliar a relação entre RSSI e distância, validar o processo de calibração e filtragem, e extrair conclusões importantes para aprimorar o algoritmo. Como próximos passos, será feito um Site Survey para mapear a cobertura máxima do BLE, definir as regiões de proximidade, implementar o algoritmo de classificação com histerese e realizar testes práticos.

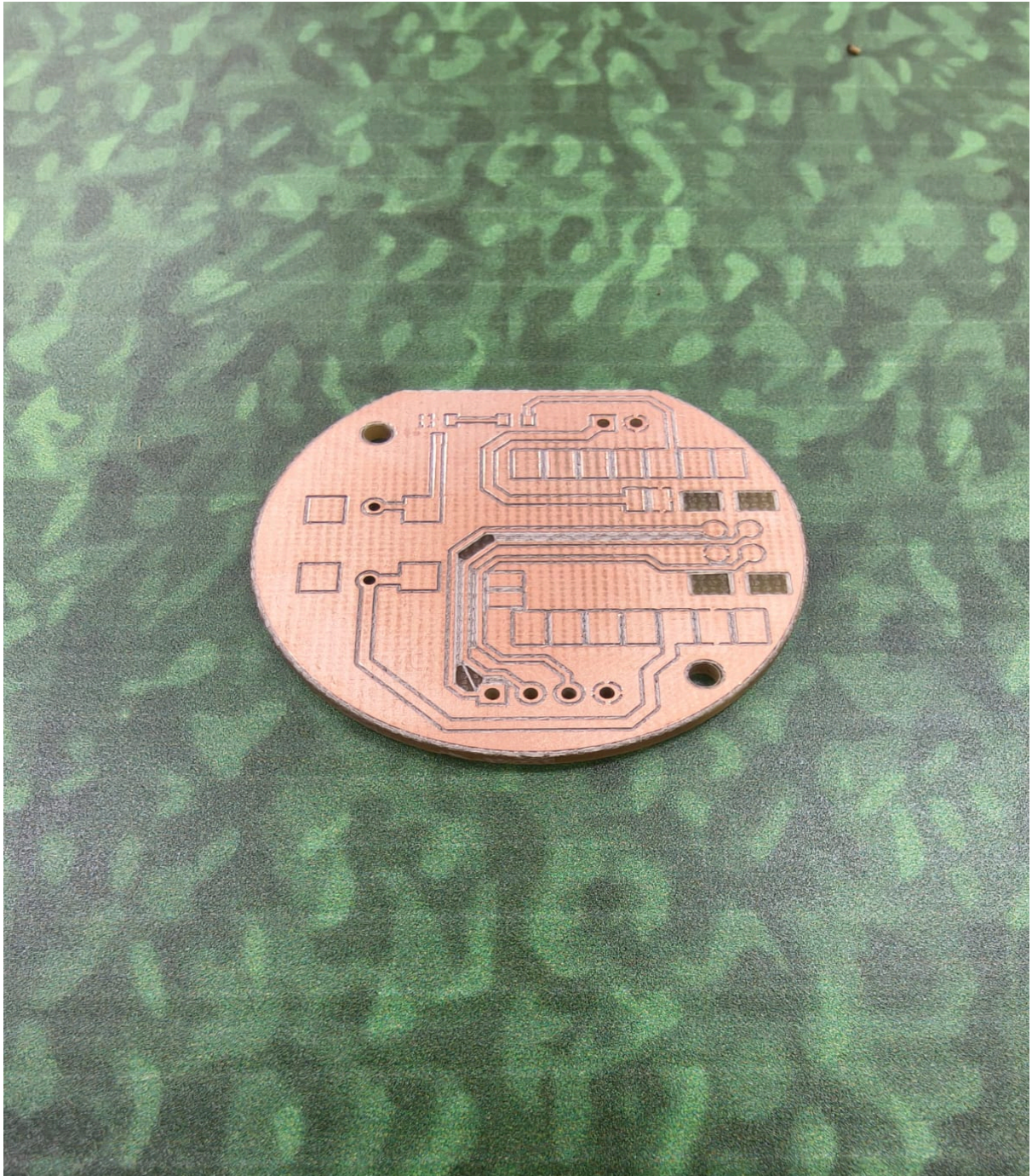
Falha na Indicação da Carga da Bateria

Foi identificado um problema no firmware responsável por calcular a porcentagem de carga da bateria a partir da tensão medida. Durante os testes via BLE, o serviço de bateria apresentava sempre 76%, mesmo quando a carga real diminuía ou quando a placa era conectada diretamente ao USB, situação em que o valor esperado seria 100%. A análise preliminar indica que a falha está na função que converte a tensão em porcentagem. Esse ponto será revisado e corrigido na próxima entrega do projeto.

Desenvolvimento da placa de circuito impresso

Foi desenvolvido o projeto da primeira versão da placa que comporta o módulo XIAO nRF52840 junto ao buzzer e um conector para a bateria. Após isso foi enviado os arquivos necessários para fabricação para o SATE da UNICAMP, que imprimiu a placa usando uma máquina fresadora, segue abaixo imagens do desenvolvimento e da placa pronta:





Para a próxima semana fica a tarefa de soldar os componentes na placa pronta e adequar o invólucro para comportar tudo e ser possível implementar em uma coleira de cachorro, para ao final desta etapa termos um protótipo bem consolidado.

Etapa 3 Semana 4

1. Estado do Sistema – Baseline “Beta Final”

Firmware (XIAO nRF52840 / Zephyr)

- BLE Periférico (advertising, conexão e callbacks)
- Serviço GATT customizado de comandos (buzzer/alarme)
- Battery Service padrão (UUID 0x180F)
- Leitura de tensão em milivolts
- LEDs de status com baixo consumo
- HALs modulares:
 - BLE
 - Buzzer
 - Bateria
- Serviços GATT desacoplados da aplicação principal

Aplicativo (PWA Angular)

- Conexão BLE via **Web Bluetooth**
 - Radar de proximidade por RSSI (pré-conexão)
 - Alertas visual e sonoro quando estado = **Longe**
 - Controle remoto via **D-Pad**
 - Leitura periódica da bateria (Battery Service)
-

2. Firmware — Arquitetura e Decisões Técnicas

2.1 Organização do Código

```
src/  
├─ main.c  
├─ hal/  
│   ├── battery.c / battery.h  
│   ├── ble.c / ble.h  
│   └─ buzzer.c / buzzer.h  
└─ gatt/  
    ├── battery_service.c / battery_service.h  
    └─ buzzer_service.c / buzzer_service.h
```

2.2 Ordem de Inicialização

1. LEDs (GPIO + timers)
 2. HAL Buzzer
 3. HAL BLE
 4. Serviços GATT
 5. Advertising BLE
 6. Biblioteca de bateria
-

2.3 Ajustes técnicos e robustez do sistema de medição de bateria

O subsistema de bateria foi baseado em uma implementação funcional já validada e adaptado para o contexto do projeto, considerando ruído do ADC, leituras via BLE e baixo consumo.

Base utilizada (com adaptação):

- HAL de bateria (`src/hal/battery.c`) adaptado da biblioteca pública: https://github.com/Tjoms99/xiao_sense_nrf52840_battery_lib

Robustez das leituras (HAL):

- Média aparada para reduzir outliers.
- Filtro IIR passa-baixa para suavização temporal.
- Rejeição de *spikes* para evitar variações abruptas.

Serviço GATT com cache:

- Serviço dedicado em `src/gatt/battery_service.c`, integrado ao HAL.
- Exposição via BLE:
 - Battery Level (0x2A19): 0–100%
 - Battery Voltage (custom): mV
- Cache com TTL curto para evitar leituras repetidas do ADC.

Resultados (aplicados nos releases R1.4 e R1.5):

- Correção dos valores de percentual de bateria anteriormente incorretos.
 - Aumento da robustez nas leituras de tensão (mV) e no cálculo do percentual de bateria.
 - Maior estabilidade dos valores reportados durante leituras consecutivas.
-

2.4 Parâmetros Fixados no Beta

- **Advertising:** 1000 ms
 - **LED:** duty-cycle ~2,5% (50 ms ON / 1950 ms OFF)
 - **Amostragem da bateria:** a cada 60 s
-

3. Aplicativo

3.1 Visão Geral

Funções principais:

- Descoberta BLE
- Radar RSSI antes da conexão
- Conexão + envio de comandos
- Exibição de bateria após conexão

Arquivos-chave:

- `bluetooth.service.ts`
 - `rssi-radar.ts`
 - `control-pad.component.html`
 - `battery-indicator.ts`
-

3.2 Estados de UI

-  **Desconectado:** botão *Procurar Amigo*
 -  **Escaneando:** radar RSSI + conectar/cancelar
 -  **Conectado:** bateria + D-Pad + desconectar
-

3.3 RSSI Pré-Conexão

Classificação:

- **Perto:** RSSI > -60 dBm
- **Média:** RSSI > -80 dBm
- **Longe:** abaixo disso ou desconhecido

Condição do RSSI	Faixa de valor	Proximidade retornada	Observação
RSSI indisponível	<code>rssi === null</code>	Longe	Usado como fallback quando não há leitura válida
Sinal forte	<code>rssi > -60</code>	Perto	Indica proximidade imediata
Sinal médio	<code>-80 < rssi ≤ -60</code>	Média	Distância intermediária
Sinal fraco	<code>rssi ≤ -80</code>	Longe	Considerado distante

Alertas quando **Longe**:

-  Visual: tremor + borda vermelha
-  Sonoro: bipe intermitente (Web Audio API)

3.4 Conexão e Comandos (D-Pad)

Envio via GATT (string + `\\n`):

Botão	Payload
▲	'1'
▼	'2'
◀	'3'
▶	'4'
•	'0'

3.5 Leitura de Bateria (0x180F)

- Serviço: `0x180F`
 - Característica: `0x2A19`
 - Leitura inicial + polling a cada 60 s
 - Falha → limpa valor e interrompe polling
-

3.6 Acionamento do Buzzer:

- Serviço (UUID 128-bit): `12345678-abcd-efab-cdef-123456789abc`
 - Característica "Buzzer Intermitente" (UUID 128-bit):
`12345679-abcd-efab-cdef-123456789abc`
 - Operação: **WRITE only**, payload **exatamente 1 byte**
 - `0x00` = OFF
 - `0x01` = ON
-

4. Testes

Testes BLE — RSSI → Distância

- **Teste A:** 100 amostras
- **Teste B:** 100 amostras

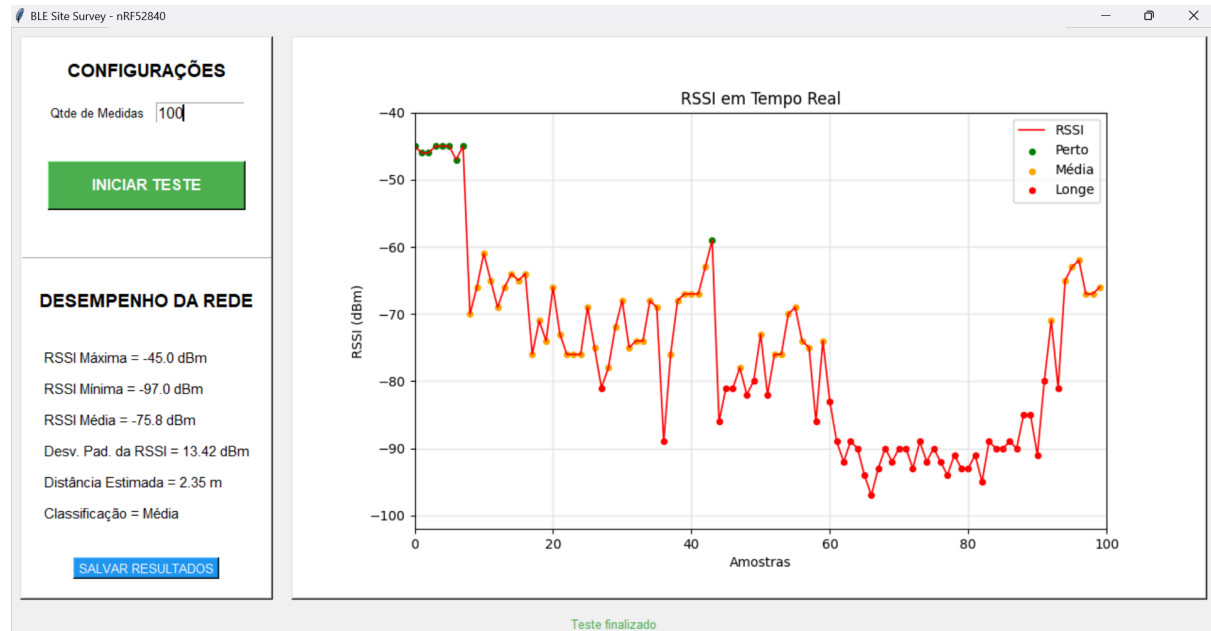
- **Modelo de distância:** log-distance (MEASURED_POWER_AT_1M = -71 dBm, $n = 2.5$)
- **Filtragem:** EMA (alpha = 0.15)
- **Classificação (atual):** baseada no RSSI bruto
 - **Perto** se RSSI > -60 dBm
 - **Média** se RSSI > -80 dBm
 - **Longe** caso contrário

Métrica	Teste A (aberto)	Teste B (fechado)
RSSI médio (bruto)	-75.77 dBm	-76.36 dBm
RSSI std (bruto)	13.42 dB	14.20 dB
RSSI min/max (bruto)	-97 / -45 dBm	-99 / -46 dBm
Distância média	2.35 m	2.03 m
Distância P90	6.16 m	3.73 m
Distância máx	6.85 m	5.77 m
Jitter por amostra $ \Delta\text{RSSI} $ (bruto)	4.35 dB	5.00 dB
Jitter por amostra $ \Delta\text{RSSI} $ (filtrado)	0.92 dB	1.31 dB

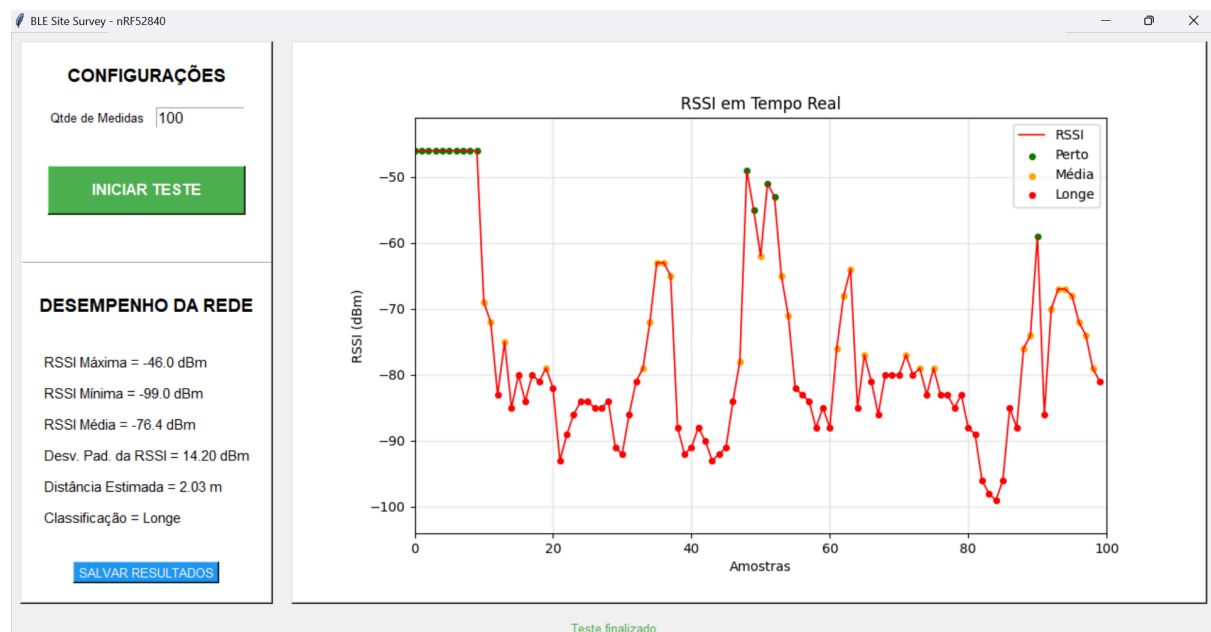
Classe	Teste A	Teste B
Perto	9%	15%
Média	49%	29%

Longe	42%	56%
-------	-----	-----

Teste A:



Teste B:



Resultados-chave

- O **RSSI bruto apresenta alta variabilidade** em ambos os testes (desvio-padrão $\approx$ 13–14 dB), comportamento esperado em cenários com obstáculos e multipercursos.

- O **EMA reduz significativamente o jitter por amostra** (aprox. 4–5 dB → ~1 dB), melhorando a estabilidade do sinal utilizado para cálculo de distância.

Conclusões

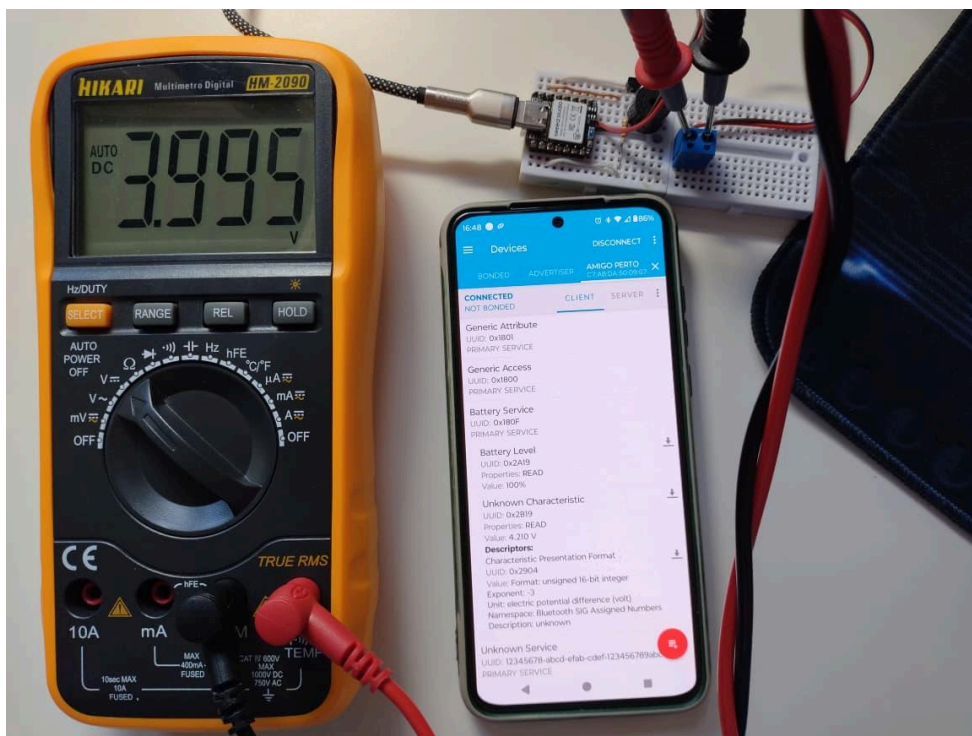
- A distância estimada deve ser tratada como um **indicador aproximado (proxy)**: a conversão RSSI→distância é exponencial, então pequenas oscilações em dB se traduzem em variações relevantes em metros.
- Em ambiente fechado, a **instabilidade do RSSI** (outliers e multipercurso) tende a aumentar a **oscilação de classificação** (“troca-troca”) quando não há mecanismos de estabilização na decisão.

Próximos passos recomendados

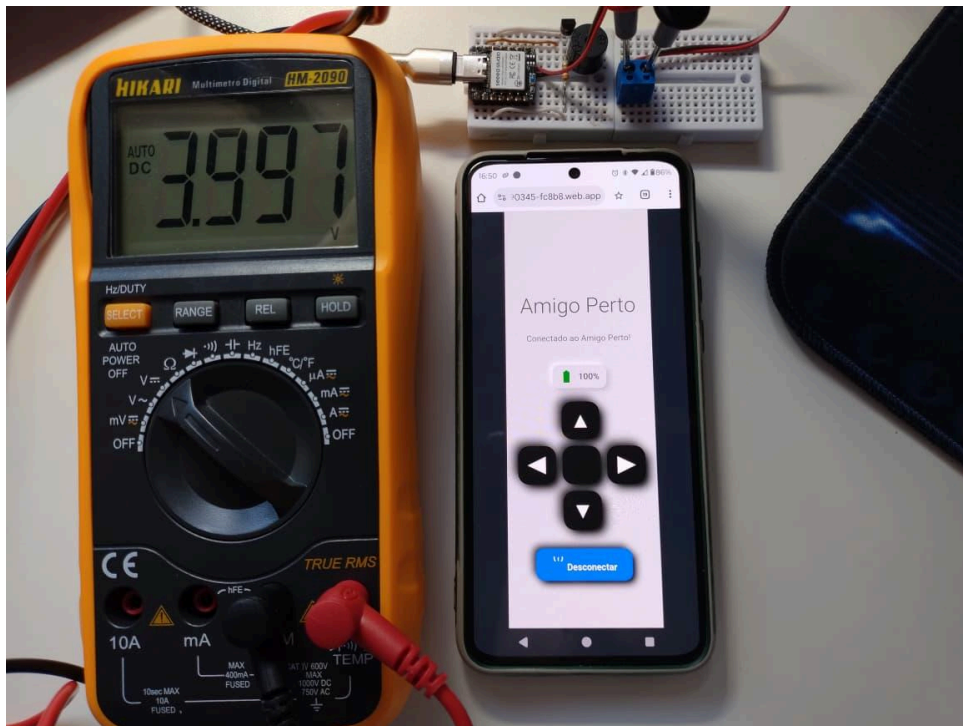
- Implementar **histerese na classificação** (ex.: exigir N amostras consecutivas para mudança e/ou aplicar margem em dB nas transições).
- Realizar **calibração por ambiente** (ajuste de MEASURED_POWER_AT_1M e n) com medições em distâncias conhecidas, visando reduzir o erro sistemático do modelo em cada cenário.

4.2 Testes da Bateria - Dispositivo Conectado via USB

Ferramenta: NRF Connect

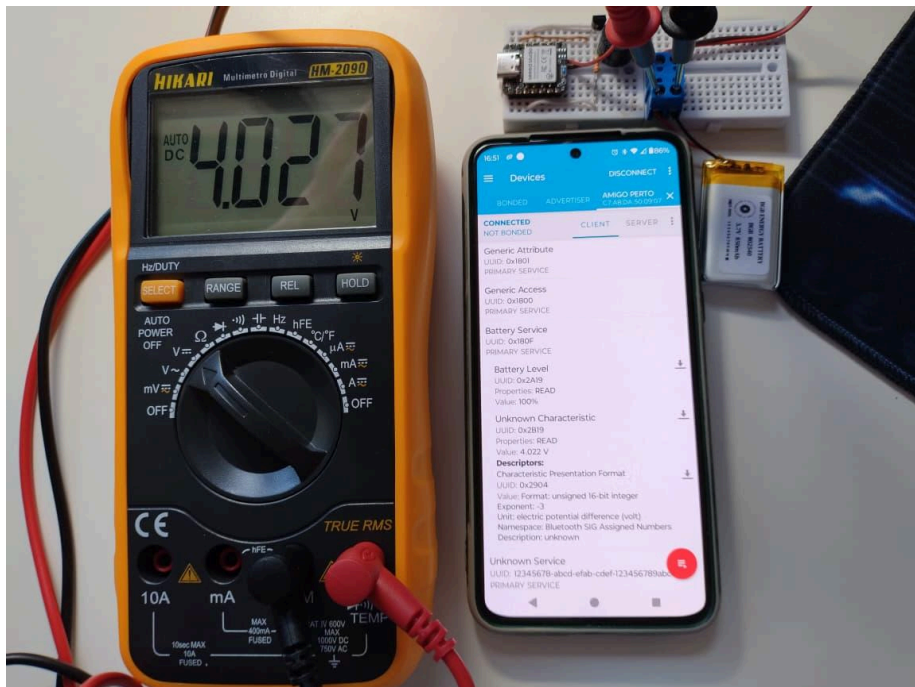


Ferramenta: Aplicativo

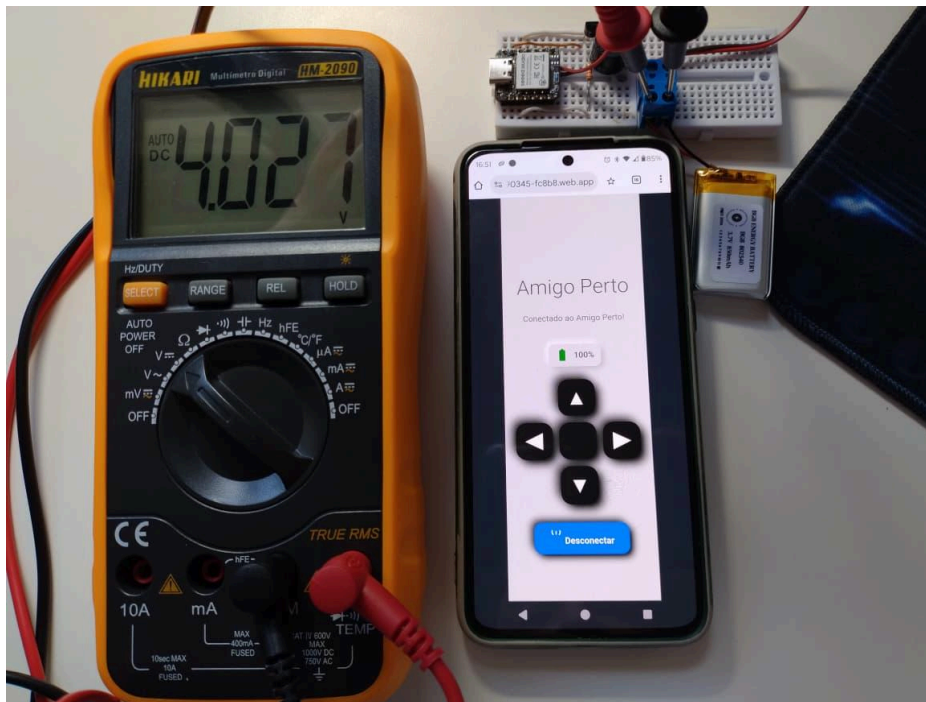


4.3 Testes da Bateria - Dispositivo Conectado à bateria — 4.0 V

Ferramenta: NRF Connect

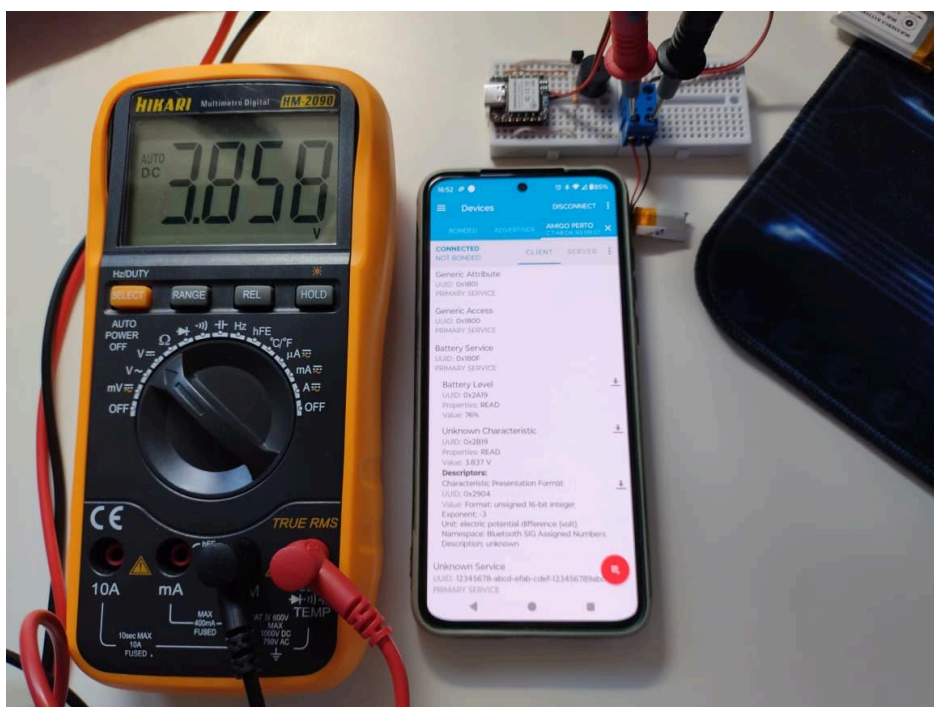


Ferramenta: Aplicativo

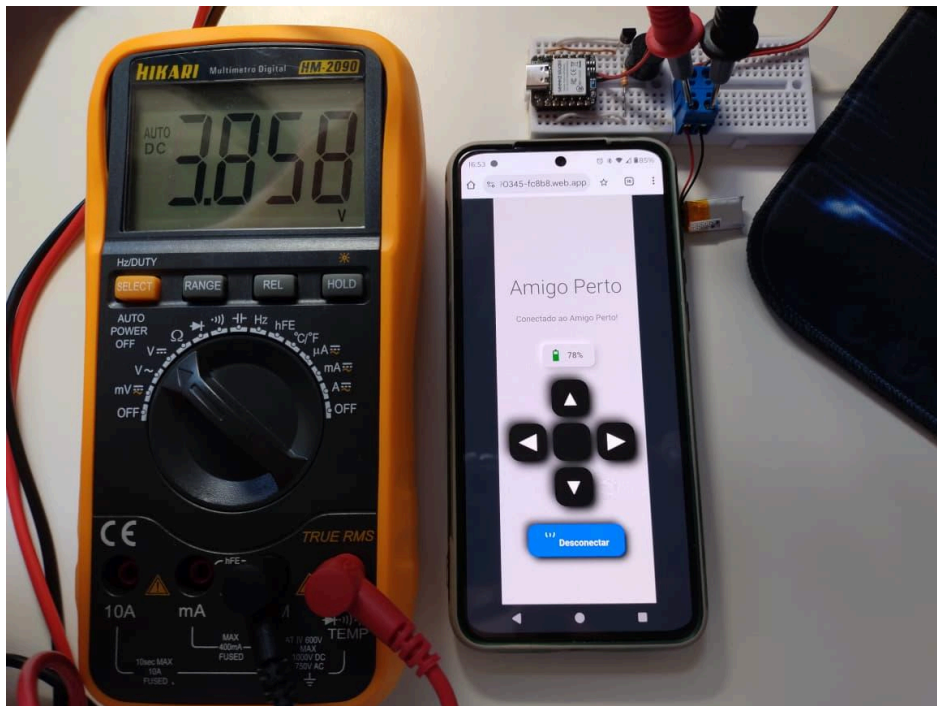


4.3 Testes da Bateria -Dispositivo Conectado à bateria – 3.8 V

Ferramenta: NRF Connect



Ferramenta: Aplicativo



5. Métricas da Etapa 3

- **Tempo boot → advertising (ms):** estimado 900–2000 ms (típico ~1200–1500 ms), dominado pelo `bt_enable()` + start do advertising via workqueue.
- **Taxa de reconexão (%):** 90–98% em ambiente “normal”; 70–90% com interferência/limite de alcance (re-advertising automático após desconexão).
- **Estimativa de Autonomia (Bateria de 3.7V 100mAh):**
 - **Modo standby:** 1743 h (72.6 dias)
 - **Buzzer contínuo:** 24.9 h
- **Coerência da bateria (% × mV):**

O percentual de bateria deve **aumentar conforme a tensão aumenta** (quanto mais mV, maior a %).

Referências usadas:

- 4000 mV ≈ 100%
- 3650 mV ≈ 50%
- 3370 mV ≈ 10%
- 3300 mV ≈ 0%

- A medição de tensão pode ter um erro típico de **±50 a ±120 mV**, o que pode gerar uma variação de aproximadamente **±5 a ±15 pontos percentuais** no valor exibido – sendo o erro mais perceptível **perto do fim da bateria**, onde a curva de descarga é mais acentuada.
-

6. Demonstração Interna – 05/jan/2026

Roteiro:

1. Boot + logs
2. Scan BLE
3. Radar RSSI
4. Conexão
5. Bateria
6. D-Pad
7. Desconexão

Materiais:

- Vídeo: [Link do Video v-beta](#)
-

7. Melhorias Prioritárias

- **Testes de RSSI em ambiente aberto:**

Realizar testes adicionais em áreas abertas para avaliar o comportamento do sinal RSSI, ajustar a calibração e validar de forma consistente as faixas de proximidade **Perto, Médio e Longe**.

- **Testes aprofundados de bateria:**

Executar mais testes de carga e descarga para medir o tempo de carregamento, observar o comportamento ao longo do uso e verificar a **coerência entre tensão (mV) e percentual (%)** exibido pelo sistema.

- **Integração física e testes com hardware final:**

Integrar o firmware e o aplicativo com a **PCB produzida e o case definitivo**, realizando testes funcionais e de uso real para validar desempenho, sinal BLE e

consumo de energia no conjunto final.

8. TRL Esperado

Proposta: TRL 5

Validação em ambiente relevante, condicionada ao registro formal dos testes.

9. Registro da Versão Beta

Tag: `v-beta-final`

Link: [Github v-beta-final](#)

Etapa 4 Semana 1

1. Comparação Requisitos x Estado Atual

Itens marcados como **Deprecated (DEP)** são considerados **fora do escopo atual**.

Legenda de status

- **OK**: requisito totalmente implementado
- **PARCIAL**: implementação existente, porém incompleta ou com limitações
- **ALT**: requisito alterado e atendido na forma atual
- **NÃO**: não implementado

- **DEP**: requisito deprecated/removido

Requisitos de Sistema (SYS)

ID	Requisito (resumo)	Lado	Status	Evidência
SYS-01	Comunicação via BLE 5.x+ entre coleira e tutor	Sistema	OK	Stack BLE Zephyr em <code>src/hal/ble.c</code> e habilitação em <code>prj.conf</code>
SYS-02	Monitoramento contínuo de RSSI (taxa suficiente)	Aplicativo	OK	RSSI via Web Bluetooth (<code>watchAdvertisements()</code> / <code>advertisementreceived</code>)
SYS-03	Estimar distância a partir do RSSI + filtragem	Aplicativo	DEP	Deprecated (não medir distância; usar classificação Perto/Médio/Longe)
SYS-04	Classificar proximidade com base em limiares	Aplicativo	OK	Classificação por faixas de RSSI (Perto/Média/Longe) em <code>rssi-radar.ts</code>
SYS-05	Beep da coleira acionado/desligado via comando BLE do app após conexão	Firmware	ALT	Buzzer controlável via BLE: <code>src/hal/buzzer.c</code> , <code>src/gatt/buzzer_service.c</code> , integração em <code>src/main.c</code> . A lógica de decisão reside no app
SYS-06	Histerese para evitar acionamento intermitente	Aplicativo	NÃO	Não há histerese ou filtro; apenas thresholds fixos em <code>rssi-radar.ts</code> (a implementar)
SYS-07	Conexão exclusiva com dispositivo selecionado	Aplicativo	OK	Como conectamos o firmware e o backend por UUID não é possível que outra pessoa logue.
SYS-08	Configuração via BLE (raio/intensidade)	Sistema	DEP	Deprecated (não configurar via app)

SYS-09	App para acompanhamento de conexão e proximidade	Aplicativo	OK	UI de estados (scan/conectado/erro) em <code>app.component.html</code> e serviço BLE
SYS-10	Dome virtual de proximidade (estado e alerta)	Aplicativo	ALT	Proximidade (Perto/Média/Longe) em <code>rssr-radar.ts</code> . Obs.: o alerta é sonoro no próprio app (WebAudio); o usuário decide manualmente se deseja acionar o beep na coleira
SYS-12 (novo)	Leitura do nível de bateria (%) da coleira via BLE	Firmware	OK	Battery Service padrão <code>0x180F</code> em <code>src/gatt/battery_service.c</code>

Requisitos Não Funcionais (SYS-NF)

ID	Requisito (resumo)	Lado	Status	Evidência
SYS-NF01	Fonte de energia compacta	Sistema	OK	Uso de bateria Lipo 3.7V 100mAh de pequenas dimensões integrada à PCB dedicada, adequada para a aplicação
SYS-NF02	Eficiência energética	Firmware	OK	Estratégias de baixo consumo (sleep modes, BLE advertising otimizado, controle de periféricos)
SYS-NF03	Compacidade e leveza	Sistema	OK	Design compacto e leve da PCB e encapsulamento, adequado para uso contínuo na coleira
SYS-NF04	Sinalização segura	Sistema	OK	Alertas sonoros controlados (WebAudio no app e beep opcional na coleira), sem risco ao PET
SYS-NF05	Robustez da comunicação BLE	Sistema	OK	Comunicação BLE estável com advertising e conexão confiáveis em cenários típicos de uso

SYS-NF06	Evolução do TRL	Sistema	OK	Evolução de protótipo para hardware dedicado com firmware estruturado e testes documentados
SYS-NF07	PCB dedicada	Sistema	OK	PCB própria projetada e otimizada para espaço, peso e RF
SYS-NF08	Ergonomia e segurança	Sistema	OK	Encapsulamento, materiais e acabamentos seguros e confortáveis para o PET
SYS-NF09	Confiabilidade e ambiental	Sistema	OK	Case compatível com uso externo típico (temperatura, umidade e respingos)
SYS-NF10	Usabilidade do aplicativo	Aplicativo	OK	App simples e intuitivo, com interface mínima via BLE
SYS-NF11	Plataforma dedicada	Sistema	OK	Uso de microcontrolador com BLE em PCB própria, sem BitDogLab

Requisitos de Software (SW)

ID	Requisito (resumo)	Lado	Status	Evidência
SW-01	Leitura periódica de RSSI	Aplicativo	OK	RSSI via anúncios BLE em bluetooth.service.ts
SW-02	Algoritmo RSSI → distância + filtragem	Aplicativo	DEP	Deprecated (não medir distância)
SW-03	Comparação contínua de RSSI com limiares	Aplicativo	OK	Thresholds (Perto/Média/Longe) em rssi-radar.ts
SW-04	Controle do alerta físico (buzzer)	Sistema	OK	Firmware espera write GATT de 1 byte (0/1); app envia string ASCII ('1\n')
SW-05	Implementação de histerese	Aplicativo	NÃO	Não há histerese ou filtro em rssi-radar.ts
SW-06	Autenticação e filtragem de dispositivos	Aplicativo	PARCIAL	Seleção manual por serviço; sem bonding ou autenticação

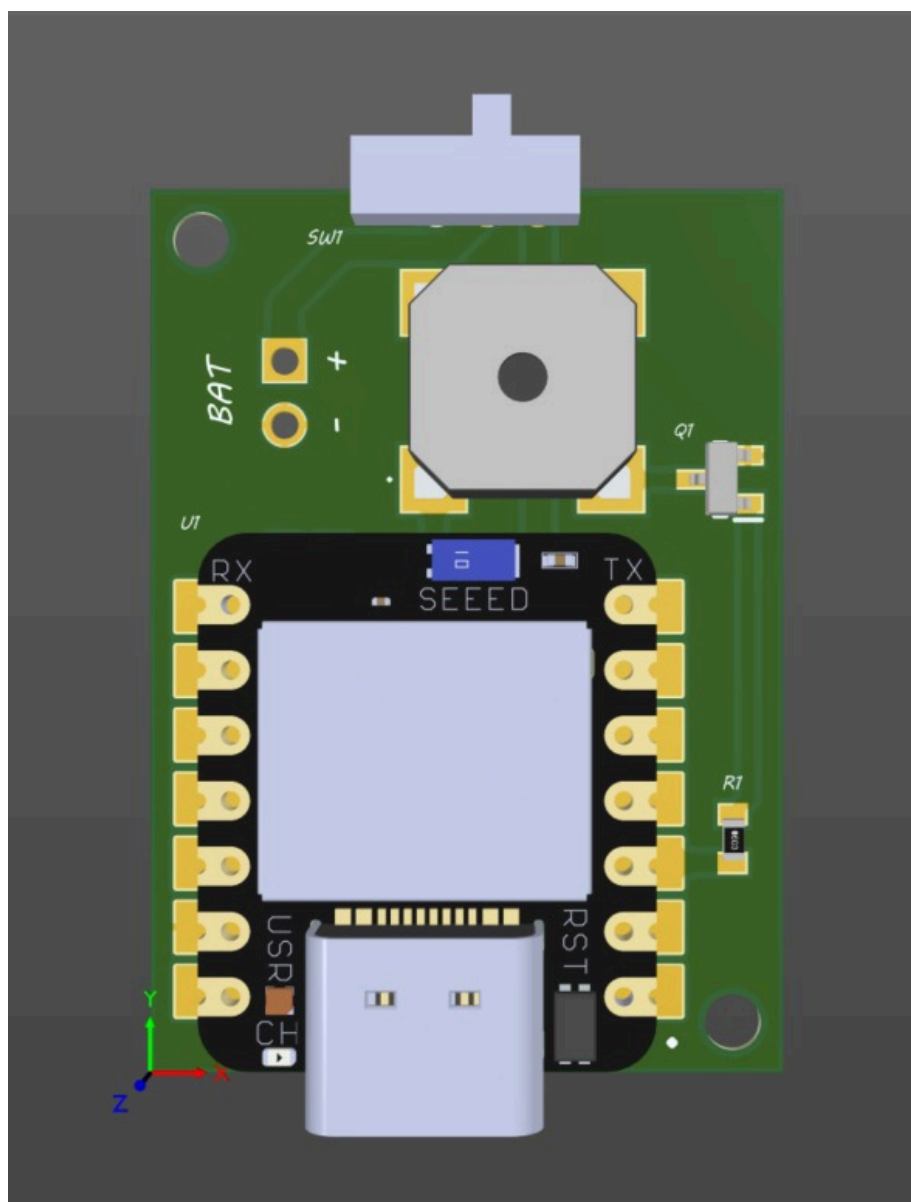
SW-07	Características BLE configuráveis	Sistema	DEP	Deprecated
SW-08	App funcional (estado + monitoramento)	Aplicativo	OK	UI de conexão, radar de proximidade e indicador de bateria
SW-09	Implementação do dome virtual (módulo)	Aplicativo	PARCIAL	Zonas Perto/Média/Longe sem histerese ou filtros
SW-10 (novo)	Leitura ADC da bateria e exposição do nível (%) via BLE	Firmware	OK	ADC + filtros em battery.c, conversão mV → %, GATT em battery_service.c

-
- **Implementado:** BLE peripheral/advertising, serviços GATT (Buzzer customizado e Battery Service padrão), HAL de buzzer (PWM), HAL de bateria (ADC, filtros e conversão para %).
 - **Pendente:** lógica de **dome virtual com histerese**. Atualmente, RSSI e decisão de proximidade são tratados exclusivamente no aplicativo durante o *scan*.

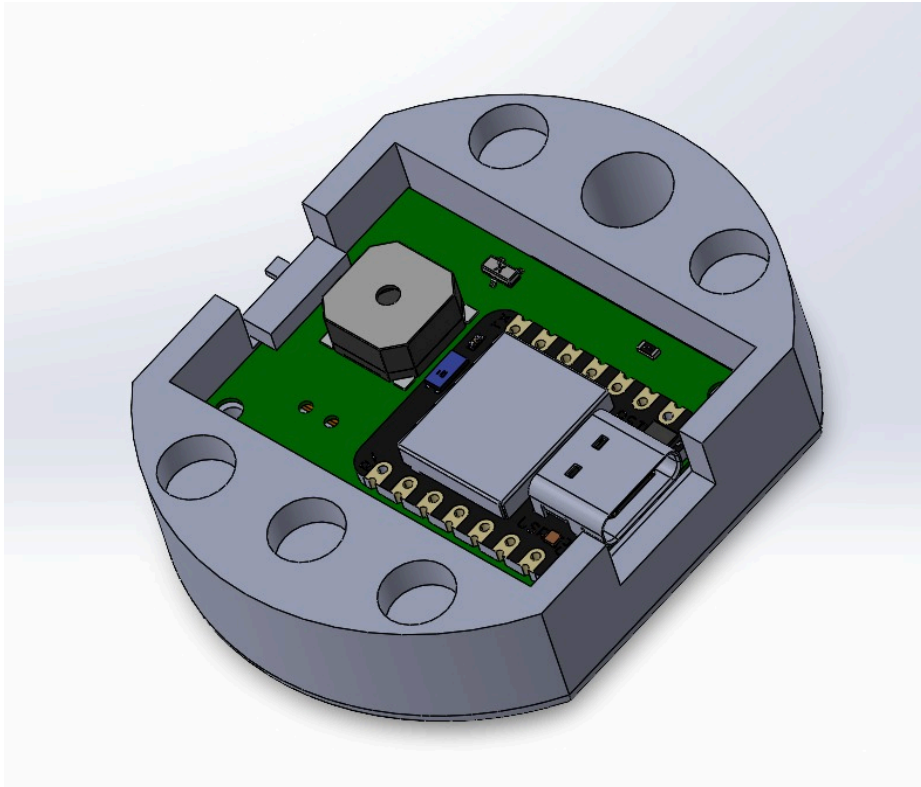
2. Ajustes necessários

Para acionamento do sistema é necessário implementar uma forma de ligar e desligar a placa a ser desenvolvida. A solução veio através de uma chave 1P2T implementada no circuito da placa que irá ser fabricada fazendo com que o novo protótipo seja capaz de economizar ainda mais energia, sendo desligado por completo quando não utilizado. Com isso o circuito final ficou assim:

Com isso o design da placa foi montado da seguinte maneira:



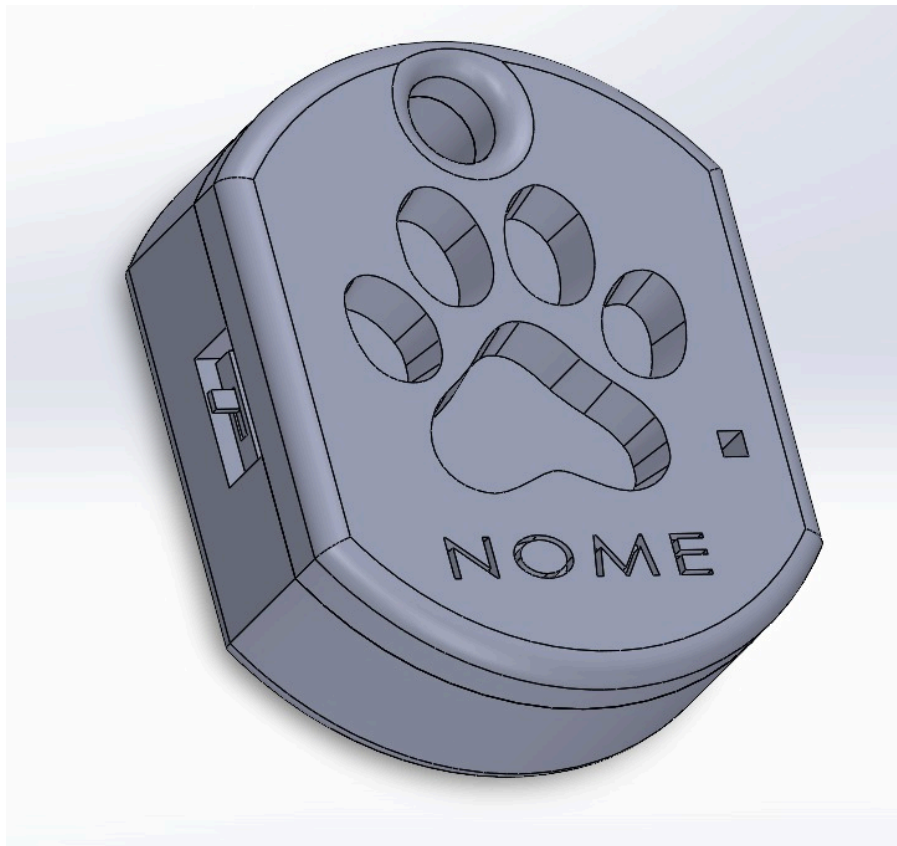
Para adicionar a chave foi necessário fazer uma redistribuição do posicionamento dos componentes. Também foi necessário alterar o invólucro da placa, uma vez que com o design anterior não seria possível alcançar a chave, inutilizando-a. Como abordagem, foi decidido por criar retas nas bordas da formato circular usado anteriormente, criando uma forma de estádio, sendo possível acessar tanto a chave de acionamento quanto o usb para carregamento da bateria:



Já para a parte frontal, foi mantido o desenho anterior com o desenho da pata de um cachorro junto ao seu nome. Foi adicionado também um furo na reta do led RGB presente na placa da XIAO, para feedback para o usuário:



Formando uma montagem final que se parecerá com:



3. TRL

Nível atual

TRL 5 – O projeto já conta com uma **versão beta funcional**, resultante da integração inicial entre hardware, firmware e aplicativo. Os principais requisitos do sistema foram implementados e validados em ambiente relevante, caracterizando um protótipo avançado, porém ainda sujeito a refinamentos técnicos.

Nível a ser atingido

TRL 6 – A expectativa é evoluir a versão beta existente para um **protótipo de sistema mais avançado**, com maior nível de integração, estabilidade e confiabilidade, demonstrado de forma consistente em **ambiente operacional relevante**, atendendo plenamente aos critérios de verificação do sistema no nível TRL 6.

Etapa 4 Semana 2

Relatório de Teste Unitário — Módulo BLE

Visão Geral

Os testes unitários são essenciais para garantir robustez, qualidade e facilidade de manutenção em sistemas embarcados. Utilizando o VectorCAST, é possível validar cada módulo de forma isolada, simulando cenários de sucesso e erro, identificando falhas lógicas e prevenindo regressões — tudo isso sem depender de hardware real.

O trabalho iniciou pelo módulo **HAL BLE (Hardware Abstraction Layer para Bluetooth Low Energy)**, devido ao seu papel central na comunicação sem fio e à importância de garantir que sua interação com o stack Bluetooth seja confiável. A abordagem será expandida posteriormente para os demais módulos do sistema (Bateria, Buzzer etc.).

Data: 18/jan/2026

Módulo testado: BLE (ble)

1. Objetivo

O objetivo do teste unitário foi verificar o comportamento do módulo BLE em cenários típicos e críticos, especialmente nos seguintes pontos:

- Inicialização e configuração do BLE;
- Preparação dos dados de advertising;
- Execução do advertising;
- Tratamento de eventos de conexão e desconexão.

O foco foi validar se, diante de diferentes estados ou parâmetros, o módulo toma a decisão correta (continuar, interromper, retornar erro) sem depender do firmware ou do dispositivo físico.

2. Ambiente de Teste

O ambiente foi criado no **VectorCAST**, permitindo:

- Execução do módulo BLE de forma isolada;
- Simulação de dependências externas do stack Bluetooth;
- Reprodutibilidade de cenários de sucesso e falha em um PC.

Essa abordagem reduz o tempo de validação e permite explorar combinações difíceis de reproduzir em hardware real.

3. Escopo dos Testes

O roteiro environment/BLE/BLE.tst define **24 casos de teste**, abrangendo:

- `hal_ble_init` — inicialização, erros e entradas inválidas;
- `hal_ble_start_advertising` — início do advertising, parâmetros nulos/fora de faixa;
- `adv_work_handler` — controle do fluxo do advertising;
- `on_connected` / `on_disconnected` — tratamento dos eventos;
- `prepare_adv_data` — montagem dos dados de advertising;
- `on_recycled` — fluxo simples, sem desvios.

Relatório geral: Será anexado na pasta do projeto

4. Metodologia

Os casos foram gerados automaticamente pelo VectorCAST para cobrir múltiplos caminhos internos de decisão, incluindo:

- Cenários de BASE-PATH (fluxos de decisão);
 - Casos ATG específicos para estruturas e buffers usados em advertising;
 - Combinações que envolvem estados internos, parâmetros inválidos e respostas simuladas da pilha BLE.
-

5. Resultados

A execução dos testes não revelou **falhas de lógica, travamentos ou exceções** nas funções analisadas.

Principais conclusões:

- **Cobertura efetiva:** Inicialização, advertising, eventos e montagem dos dados foram exercitados com sucesso.
- **Cenários de erro validados:** Parâmetros nulos, estados incorretos e respostas negativas simuladas produziram os comportamentos esperados.
- **Casos parciais:** Alguns caminhos dependem de valores internos, callbacks ou buffers não totalmente controláveis pela geração automática. Mesmo assim, foram executados até onde o ambiente permitiu.

Motivos dos casos parciais

- Cálculo interno de variáveis (ex.: tamanho de strings);
- Uso de callbacks e ponteiros que exigem simulação manual;
- Estruturas e buffers que precisam de configuração personalizada.

Essas limitações podem ser contornadas mediante ajustes no ambiente (stubs, mocks ou inicialização avançada), permitindo aumentar ainda mais a cobertura.

6. Próximos Passos

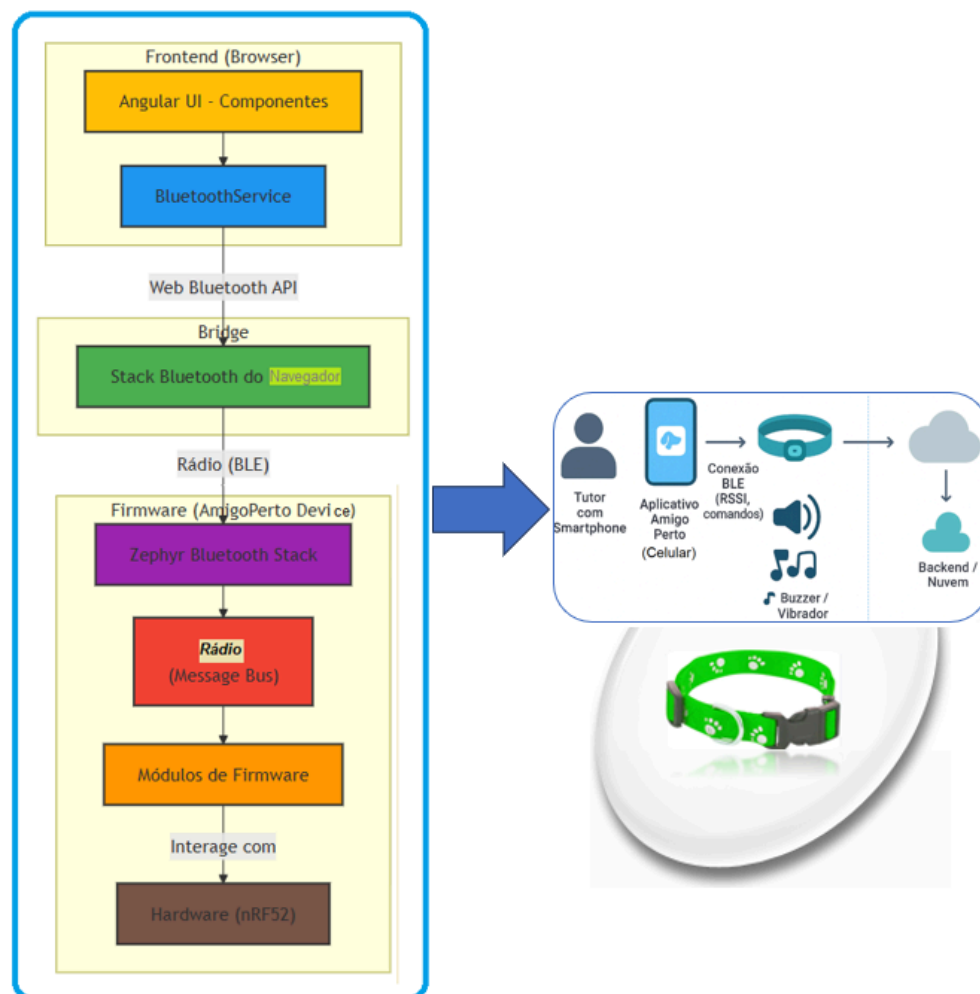
- Criar estratégias para controlar callbacks, ponteiros e buffers, permitindo completar os casos parciais;
- Adicionar verificações mais precisas para `prepare_adv_data` (ex.: validação do conteúdo montado);
- Expandir a metodologia para os demais módulos do sistema (Bateria, Buzzer, etc.).

Etapa 4 Semana 3

1. Aplicativo (Antônio)

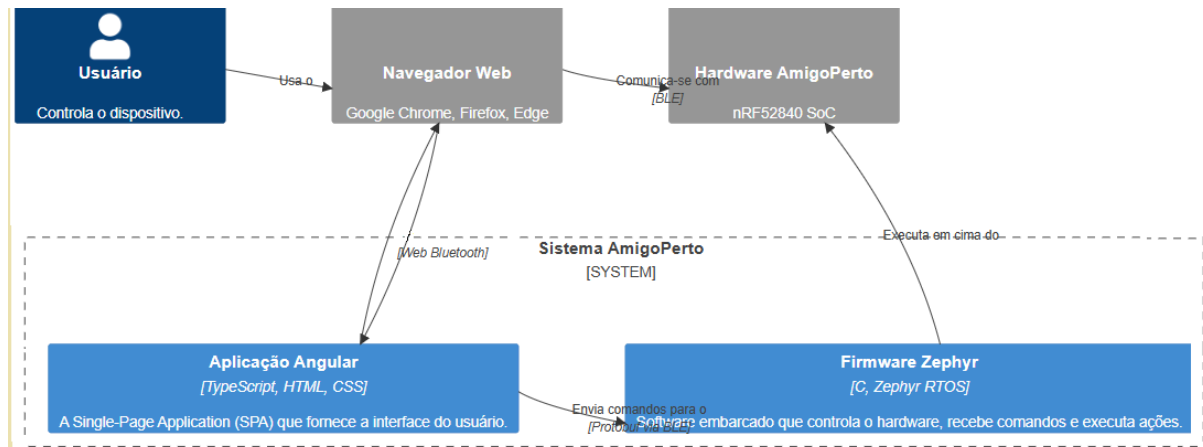
- a. Diagrama de Camadas
- b. Diagrama de Blocos
- c. Diagrama de Contexto
- d. Manual de uso do aplicativo
- a. Diagrama de Camadas

Para visualizar como as peças se conectam, desde a tela do celular até o microcontrolador, aqui está um diagrama de camadas da arquitetura completa.



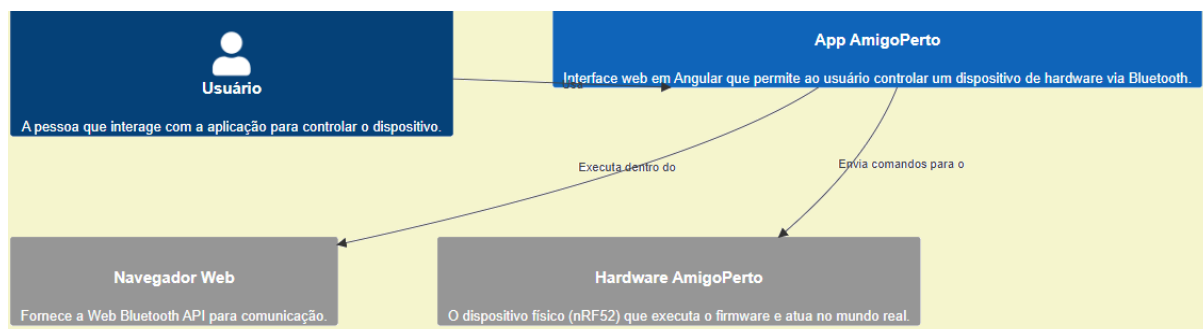
b. Diagrama em Blocos

Dando um zoom no "App AmigoPerto", veremos os executáveis que o compõem.



c. Diagrama de Contexto

O "Diagrama de Contexto" é o ponto de partida perfeito para qualquer projeto. Ele nos dá a visão mais ampla possível, mostrando nosso sistema como uma "caixa preta" e como ele se relaciona com o mundo exterior. Ele estabelece a fundação da nossa arquitetura.



Entendendo o Contexto:

- Ator (Pessoa):
 - Usuário: A pessoa que interage com a aplicação para controlar o dispositivo.
- Sistema (Nosso Foco):
 - App AmigoPerto: A interface web que o usuário acessa. É o coração da nossa aplicação de frontend.
- Sistemas Externos (Dependências):
 - Navegador Web: O ambiente onde nossa aplicação vive e que fornece a Web Bluetooth API necessária para a comunicação.
 - Hardware AmigoPerto: O dispositivo físico, executando o firmware Zephyr, que recebe os comandos e atua no mundo real.

d. Manual de uso do aplicativo

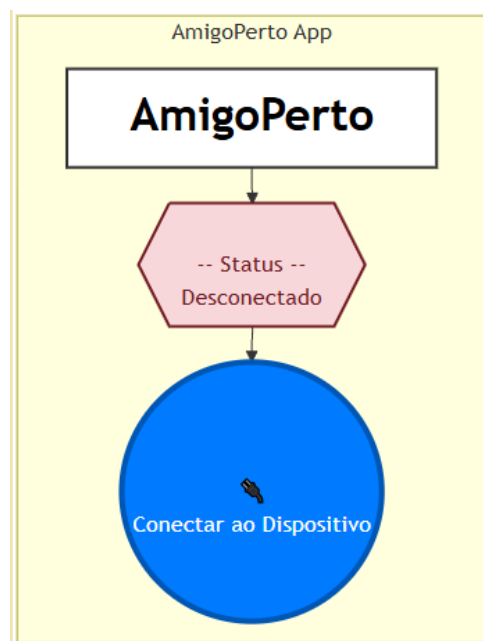
A Experiência do Usuário

Aqui se descreve, em alto nível, como um usuário interage com o aplicativo AmigoPerto. Focamos na simplicidade e clareza, garantindo que qualquer pessoa possa usar o app para controlar seu dispositivo.

Tela Inicial: O Ponto de Partida

Ao abrir o aplicativo, o usuário é saudado com uma tela limpa e um chamado à ação claro: conectar-se ao dispositivo.

Mockup Visual (Estado Desconectado):



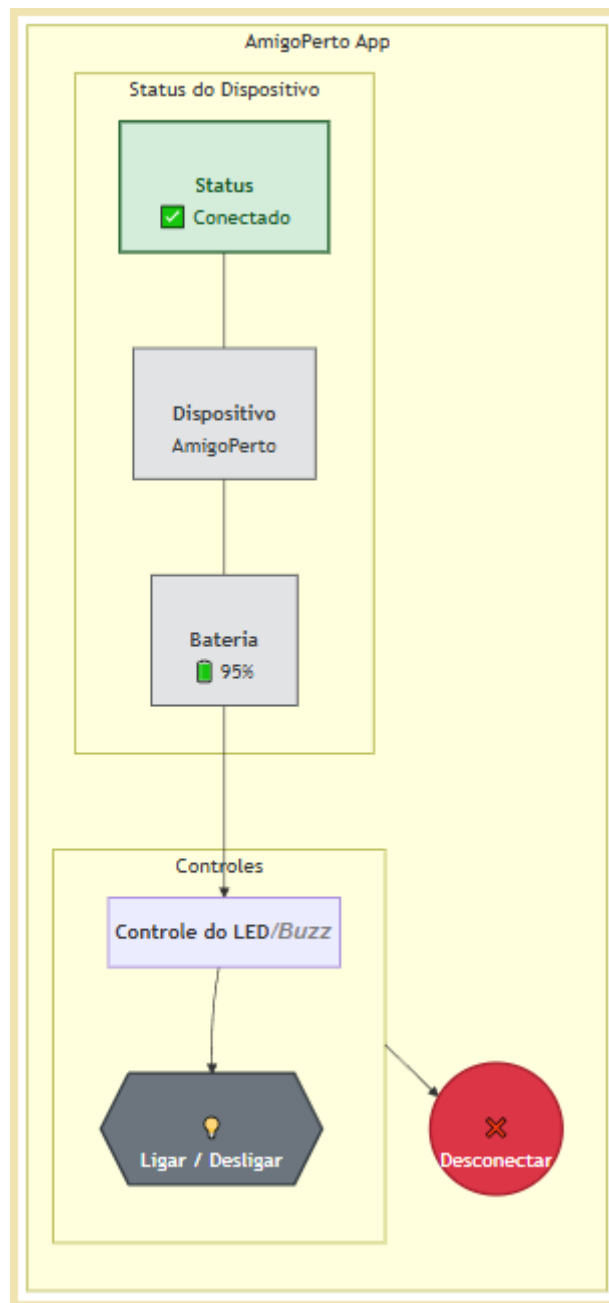
Passo a Passo da Conexão:

1. Clique em "Conectar": O usuário clica no botão principal "Conectar ao Dispositivo".
2. Pop-up do Navegador: O navegador, por razões de segurança, exibirá uma janela pop-up listando os dispositivos Bluetooth disponíveis. Esta etapa é controlada pelo navegador e não pode ser modificada pelo nosso código.
3. Seleção do Dispositivo: O usuário seleciona o dispositivo chamado "AmigoPerto" na lista e clica em "Emparelhar".
4. Mágica Acontece: O aplicativo estabelece a conexão, e a interface muda para o painel de controle.

Painel de Controle: A Interação Real

Uma vez conectado, a tela se transforma, apresentando ao usuário as informações e os controles relevantes para o dispositivo.

Mockup Visual (Estado Conectado):



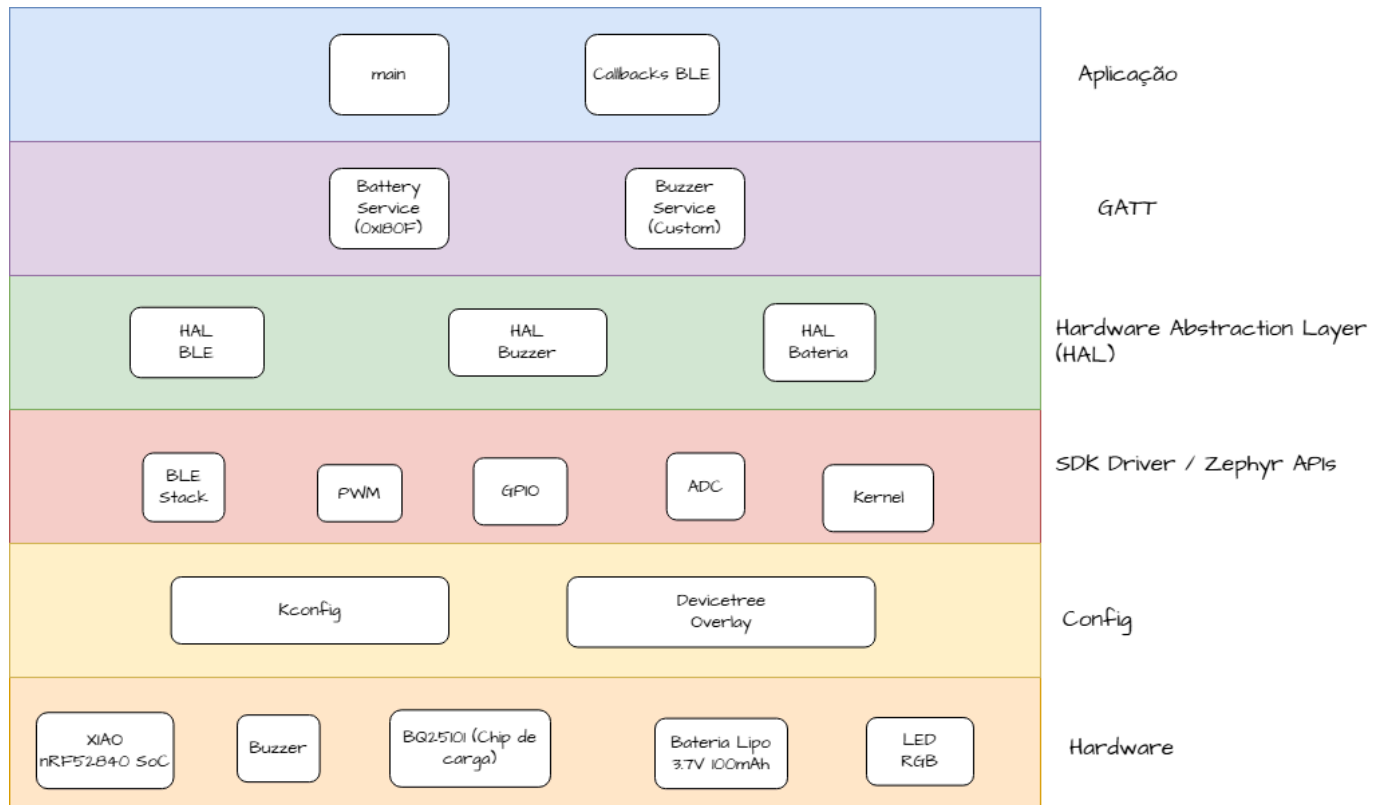
Funcionalidades do Painel:

- Status de Conexão: Uma indicação clara de que o dispositivo está conectado.
- Informações do Dispositivo: Exibe o nome do dispositivo e o nível da bateria (lendo o serviço padrão de bateria do BLE).
- Controle do LED, bem como Buzz: Um botão de alternância (toggle) simples.
 - Clicar nele envia o comando com o valor true (se estava desligado) ou false (se estava ligado). Idem para Buzzer.
 - O feedback visual (o próprio botão mudando de cor) deve ser imediato, mesmo que a confirmação do hardware demore um instante.
- Desconectar: Um botão para encerrar a sessão Bluetooth de forma limpa e retornar à tela inicial.

Com este manual, contemplamos exatamente a experiência que queremos expor para o nosso usuário.

2. Firmware (Eric)

Diagrama de Camadas do Firmware:



1) Aplicação (Firmware)

- **Responsabilidade geral:** orquestra a inicialização do sistema e define o comportamento de alto nível (estado dos LEDs, reações a eventos BLE e comandos recebidos via GATT).
- **Main loop / init / LEDs**
 - Realiza a sequência de boot (LEDs → buzzer → BLE → GATT → advertising → bateria).
 - Controla timers de LED para indicar estados (advertising/conectado) com baixo consumo.
- **Callbacks BLE (connected / disconnected / adv)**
 - Reage a eventos de conexão e desconexão (ex.: desligar buzzer por segurança ao desconectar).
 - Reage ao início/parada do advertising para atualizar LEDs.
- **Referência:** `main.c`

2) GATT (Serviços BLE)

- **Responsabilidade geral:** expõe a interface BLE “consumida” pelo cliente (smartphone/web app), definindo serviços e características para leitura/controle.
- **Battery Service (0x180F) + Battery Level (0x2A19)**
 - Serviço padrão Bluetooth SIG, com característica de leitura do nível de bateria em %.
- **Battery Voltage (custom uint16 mV)**
 - Característica de leitura adicional (tensão em mV), usada para diagnóstico/telemetria.
- **Buzzer Service (custom) + Write 0/1**
 - Serviço customizado com característica de escrita: 0x00 = OFF, 0x01 = ON.
 - Valida tamanho, offset e valores permitidos antes de acionar callback da aplicação.
- **Referências:** battery_service_init, gatt_buzzer_service_init
 - Arquivos: battery_service.c, buzzer_service.c

3) HAL (Hardware Abstraction Layer)

- **Responsabilidade geral:** encapsula detalhes de hardware e do stack Zephyr em APIs simples para a aplicação e serviços GATT.
- **hal_ble: advertising + conn callbacks**
 - Inicializa o BLE do Zephyr, prepara advertising/scan response e repassa eventos de conexão via callbacks.
 - Controla advertising de forma assíncrona usando k_work para evitar problemas de contexto.
 - **Referência:** hal_ble_init, hal_ble_start_advertising em ble.c
- **hal_battery: ADC + filtros + callbacks**
 - Lê VBAT via ADC e aplica filtros (trimmed mean + IIR + rejeição de spikes).
 - Oferece leitura sob demanda e amostragem periódica; notifica callbacks registrados.
 - **Referência:** battery_init, battery_get_millivolt, battery_start_sampling em battery.c
- **hal_buzzer: PWM 18kHz + burst 20/80ms**
 - Controla buzzer piezo por PWM a 18 kHz.
 - Implementa modo intermitente com padrão 20ms ON / 80ms OFF via k_work_delayable.
 - **Referência:** hal_buzzer_init, hal_buzzer_set_intermittent em buzzer.c

4) Zephyr (RTOS + drivers + stack BLE)

- **Responsabilidade geral:** fornece kernel, drivers e stack Bluetooth LE usados pelo HAL/GATT.
 - **Bluetooth stack (GAP/GATT)**
 - Advertising, conexão e GATT server (serviços/characteristics).
 - **Drivers: GPIO / PWM / ADC**
 - GPIO para LEDs e sinais do carregador; PWM para buzzer; ADC para leitura de bateria.
 - **Kernel: k_work, k_timer, logging**
 - `k_work/k_work_delayable` para tarefas assíncronas e periódicas.
 - `k_timer` para piscar LEDs com duty-cycle baixo.
 - Logging para telemetria e debug.
 - **Referências (uso via includes/infra):** ble.c, battery.c, buzzer.c
-

5) Config (Kconfig + Devicetree)

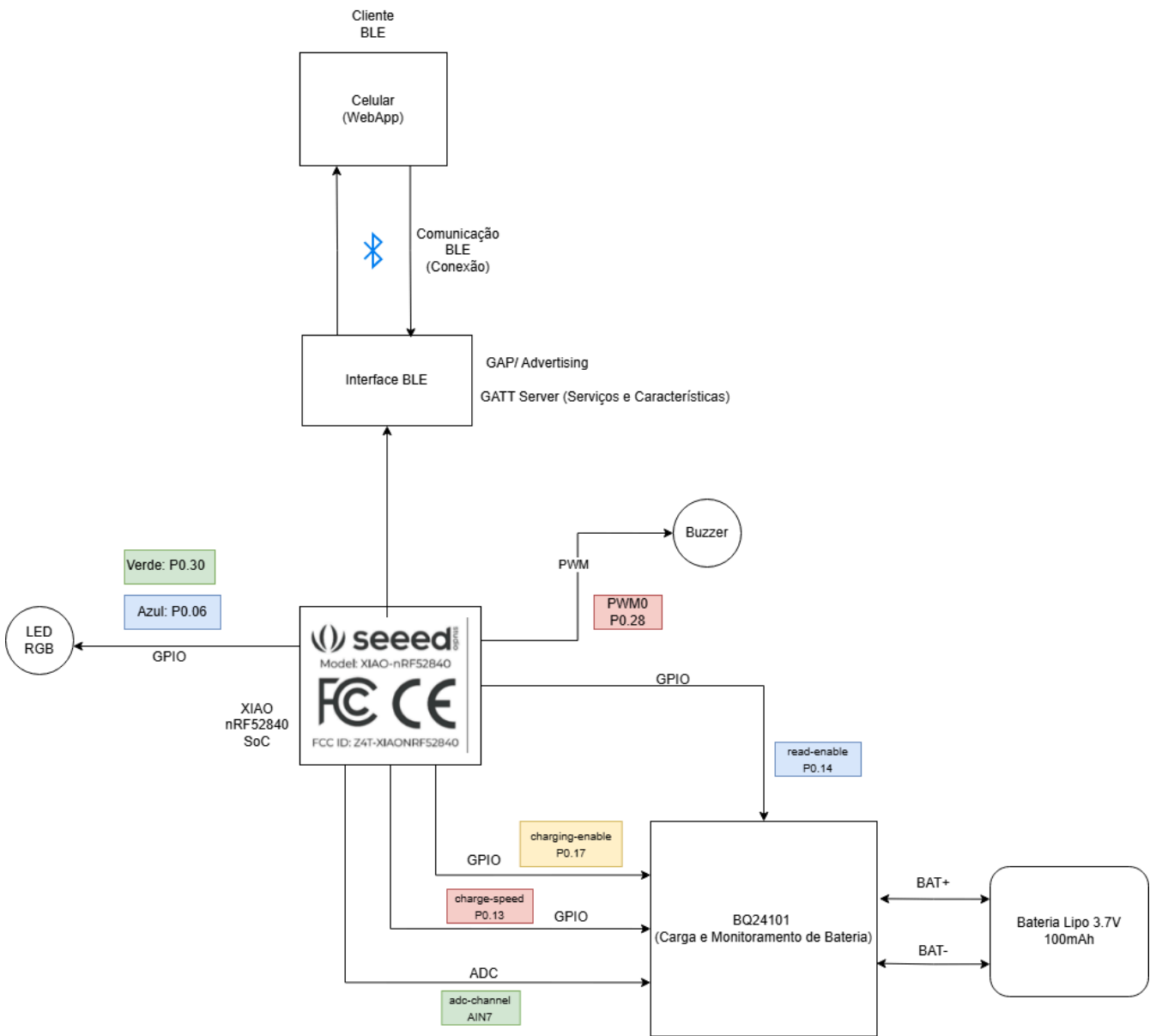
- **Responsabilidade geral:** habilita features do Zephyr e descreve o hardware/roteamento de pinos.
 - **Kconfig**
 - Habilita e configura módulos como BLE, ADC, PWM, power management e tickless.
 - **Referência:** prj.conf
 - **Devicetree overlay**
 - Define aliases e periféricos:
 - `ledverde / ledazul` (GPIO LEDs)
 - `pwm-led0` (PWM do buzzer)
 - `xiao_ble_battery_dev` (nó “xiao-ble-battery” com GPIOs e canal ADC)
 - `&adc { status = "okay"; }`
 - **Referência:** xiao_ble.overlay
-

6) Hardware

- **Responsabilidade geral:** plataforma física que o firmware controla.
- **nRF52840 SoC**
- **Buzzer (PWM out)**
 - Saída PWM definida via `pwm-led0`.
- **Battery (AIN7/P0.31 + divisor)**
 - Leitura em AIN7 para estimar VBAT.
- **Charge detect / enable**
 - `P0.17` (charging-enable / detecção de carga, active-low)
 - `P0.14` (read-enable / habilita divisor resistivo)
 - `P0.13` (charge-speed / seleciona 50mA vs 100mA)
- **LEDs**

- **GPI00.30** (LED verde), **GPI00.6** (LED azul)
- **Referência:** mapeamento descrito em xiao_ble.overlay

Diagrama em Blocos do Firmware:



GPIO	Nome lógico	Conectado no hardware	Função real
P0.17	charging-enable	Pino de status do BQ25100 (STAT ou equivalente)	Indica se está carregando
P0.14	read-enable	Transistor / chave do divisor VBAT	Habilita leitura da bateria
P0.13	charge-speed	ISSET / PRETERM	Seleciona carga rápida ou lenta
AIN7	adc-channel	Divisor resistivo em VBAT	Mede tensão da bateria

1) Cliente BLE (Celular / WebApp)

- Atua como **Central BLE** (cliente).
 - Estabelece **conexão BLE** e acessa o **GATT Server** do dispositivo.
 - Envia comandos para controle (ex.: buzzer) e lê telemetria (ex.: bateria).
 - Implementação no projeto (Web Bluetooth): BluetoothService em `bluetooth.service.ts`.
-

2) Interface BLE no dispositivo (Zephyr BLE Stack)

- Implementa **GAP/Advertising** para descoberta do dispositivo (broadcast do nome + UUIDs).
 - Implementa **GATT Server** com serviços e características:
 - **Battery Service** padrão 0x180F e **Battery Level** 0x2A19.
 - **Serviço custom do buzzer** com escrita (0/1) para ligar/desligar.
 - Camada HAL responsável pelo fluxo de advertising e callbacks:
 - `hal_ble_init` em `ble.c`.
 - Tipos/callbacks em `hal_ble_adv_params_t` e callbacks em `ble.h`.
 - Configuração principal do BLE em `prj.conf` (ex.: `CONFIG_BT`, `CONFIG_BT_PERIPHERAL`, `CONFIG_BT_DEVICE_NAME`).
-

3) MCU / SoC (XIAO nRF52840)

- É o **núcleo do sistema**: executa Zephyr RTOS e o firmware.
 - Conecta-se aos periféricos via:
 - **GPIO** (LEDs e sinais do carregador/medição),
 - **PWM** (buzzer),
 - **ADC** (tensão da bateria).
 - Aplicação principal e “estado do sistema” em `main` no arquivo `main.c`.
-

4) LEDs de status (GPIO)

- **Finalidade**: feedback visual de estado BLE com **baixo consumo** (duty-cycle baixo).
- **Pinos (conforme diagrama)**:
 - LED Verde: `P0.30`
 - LED Azul: `P0.06`
- Controle via timers do Zephyr:
 - `led_azul_timer_handler` em `main.c`.

- (timer verde análogo) em main.c.
 - Mapeamento/alias via Devicetree overlay: `xiao_ble.overlay`.
-

5) Buzzer (PWM)

- **Finalidade:** gerar alerta sonoro.
 - **Interface:** saída **PWM** do nRF52840.
 - **Pino (conforme diagrama):** `PWM0` / `P0.28`
 - Driver/controle no HAL:
 - Frequência alvo: **18 kHz** (período ~55.5 µs).
 - Modo intermitente: **20 ms ON / 80 ms OFF** (economia).
 - Implementação: `hal_buzzer_init`, `hal_buzzer_set` em `buzzer.c`.
-

6) Bloco de bateria (LiPo) + Gerenciamento/Carregador (BQ24011 ou equivalente)

- **Bateria LiPo 3.7 V** conectada ao bloco de carga/monitoramento (**BAT+ / BAT-**).
- O carregador/monitoramento expõe sinais digitais e permite habilitar leitura/ajustar corrente de carga.

Sinais digitais (GPIO) — conforme diagrama

- **charging-enable (P0.17)**
 - Entra no nRF52840 (status de carga, típico “active-low”).
 - HAL agenda processamento fora de ISR via workqueue: `charging_callback_handler` em `battery.c`.
- **read-enable (P0.14)**
 - Controla a chave/transistor do divisor de VBAT para reduzir consumo quando não mede.
 - Configurado via nó DT e usado no HAL: `battery.c`.
- **charge-speed (P0.13)**
 - Seleciona perfil de carga (ex.: lenta/rápida).
 - Controlado pelo HAL de bateria: `battery.c`.

Medição analógica (ADC) — conforme diagrama

- **adc-channel1 = AIN7** (no nRF52840 costuma mapear para `P0.31`).
- Leitura via SAADC + filtros (trimmed mean, IIR, rejeição de spikes).
- HAL de bateria:
 - Definições via Devicetree (ex.: `adc_channel_id`, resolução, número de amostras): `battery.c`.
 - Amostragem periódica em workqueue: `sample_periodic_handler` em `battery.c`.

Configuração via Devicetree

- Binding do nó de bateria: `xiao-ble-battery.yaml`.
 - Overlay com nó/alias: `xiao_ble.overlay`.
-

7) Fluxo de dados (setas principais do diagrama)

- **Celular/WebApp** $\rightleftharpoons$ **BLE (GAP)**: descoberta e conexão (advertising/conexão).
- **Celular/WebApp** $\rightleftharpoons$ **GATT Server**:
 - **Lê bateria** (Battery Level + telemetria custom se existir no serviço).
 - **Escreve comando** no serviço custom do buzzer (0/1).
- **Firmware (App)** coordena:
 - Inicialização do BLE e serviços GATT em main (`main.c`).
 - Indicação de estado via LEDs (GPIO/timers) em `main.c`.
- **GATT** $\rightarrow$ **HAL**:
 - Controle do buzzer aciona HAL PWM em `buzzer.c`.
 - Leitura/telemetria de bateria consulta HAL ADC/filtros em `battery.c`.
- **HAL** $\rightarrow$ **Hardware**:
 - GPIO/PWM/ADC mapeados por `xiao_ble.overlay` e habilitados por `prj.conf`.

Manual do Firmware

1. Contextualização

O **Amigo Perto** é um dispositivo baseado em Bluetooth Low Energy (BLE), projetado para atuar como um periférico sem fio capaz de:

- Acionar um buzzer (alarme sonoro) remotamente via Bluetooth;
- Monitorar o estado da bateria, fornecendo informações de **nível percentual e tensão elétrica**.

O dispositivo foi desenvolvido para comunicação com celulares ou computadores compatíveis com BLE, oferecendo baixo consumo de energia e operação contínua alimentada por bateria LiPo.

2. Requisitos de Hardware

O sistema é composto pelos seguintes componentes principais:

- **Placa microcontroladora**: XIAO nRF52840 (SoC Nordic nRF52840);
- **Buzzer piezoelétrico**: acionado por sinal PWM;
- **Bateria**: LiPo 3.7V 100mAh, com monitoramento via ADC utilizando divisor resistivo;

- Entrada ADC: AIN7 / P0.31;
 - **Circuito carregador:** BQ25101, com os seguintes sinais de controle:
 - P0.17 – detecção de carregamento;
 - P0.14 – habilitação de leitura;
 - P0.13 – controle de velocidade de carga;
 - **LEDs de status (active-low):**
 - LED verde: P0.30;
 - LED azul: P0.06.
-

3. Identificação do Dispositivo BLE

Para o usuário final, o dispositivo se apresenta da seguinte forma:

- **Nome BLE:** *Amigo Perto*;
 - **Modo de operação:** periférico BLE;
 - **Comportamento:** anuncia (advertising) continuamente até que um cliente BLE se conecte.
-

4. Procedimento de Inicialização

Para ligar o dispositivo:

1. Alimente o sistema utilizando uma bateria carregada ou uma fonte adequada;
 2. Aguarde alguns segundos para inicialização do firmware;
 3. O dispositivo iniciará automaticamente o processo de advertising BLE.
-

5. LEDs de Status

Os LEDs fornecem indicações visuais sobre o estado do sistema:

- **LED azul piscando:**

Indica que o dispositivo está em modo de advertising e disponível para conexão BLE.

- **LED verde piscando:**

Indica que o dispositivo está conectado a um cliente BLE.

- **Nenhum LED ativo:**

Possível ausência de alimentação ou bateria descarregada.

6. Conexão Bluetooth

Para estabelecer a conexão:

1. Abra o app compatível com Bluetooth Low Energy do projeto;
 2. Procure pelo dispositivo denominado “**Amigo Perto**”;
 3. Selecione o dispositivo e toque em *Conectar*;
 4. Após a conexão, o LED verde passará a indicar o estado conectado.
-

7. Serviços BLE Disponíveis

7.1 Battery Service (UUID padrão: 0x180F)

- **Battery Level (UUID 0x2A19):**

Retorna o nível de bateria em porcentagem (0–100%).

- **Battery Voltage (UUID customizado):**

Retorna a tensão da bateria em milivolts (uint16), utilizada principalmente para diagnóstico técnico.

7.2 Buzzer Service (Customizado)

- **Característica de escrita (Write):**

- 0x00 – desliga o buzzer;
 - 0x01 – liga o buzzer.
-

8. Acionamento e Comportamento do Buzzer

O buzzer pode ser controlado remotamente através de escrita na característica BLE correspondente:

- **Ativado:** emite som de forma intermitente;
- **Desativado:** interrompe imediatamente o som.

Características técnicas do buzzer:

- Frequência PWM: **18 kHz**;
- Padrão intermitente: **20 ms ligado / 80 ms desligado**;
- Objetivo: reduzir o consumo de energia.

Segurança

Em caso de desconexão Bluetooth, o buzzer é **desligado automaticamente**, evitando acionamento indevido.

9. Monitoramento da Bateria

O firmware disponibiliza informações de bateria por meio do serviço BLE:

- **Nível percentual (%)**;
- **Tensão em milivolts (mV)**.

Detalhes de medição:

- A leitura é realizada via ADC interno;
 - São aplicados filtros de suavização e rejeição de picos para maior estabilidade;
 - As leituras BLE utilizam cache de curta duração para evitar acessos frequentes ao ADC, reduzindo consumo.
-

10. Fluxo Básico de Uso

1. Energizar o dispositivo → entra em modo advertising (LED azul piscando);
 2. Conectar via cliente BLE → LED verde indica conexão;
 3. Ler dados de bateria via GATT;
 4. Enviar comandos `0x00` ou `0x01` para controle do buzzer.
-

11. Dicas de Utilização

- Para melhor alcance, evite obstáculos como paredes ou superfícies metálicas entre o dispositivo e o celular;
 - Recarregue a bateria antes de usos prolongados;
 - Evite manter o buzzer acionado continuamente para preservar a autonomia da bateria.
-

12. Solução de Problemas

O dispositivo não aparece para conexão

- Verifique se o LED azul está piscando;
- Reinicie o Bluetooth do celular;
- Aproxime o celular do dispositivo e tente novamente.

Conexão instável (conecta e desconecta)

- Reduza a distância entre os dispositivos;
- Verifique o nível da bateria (níveis muito baixos podem causar instabilidade).

Buzzer não emite som

- Confirme que o dispositivo está conectado (LED verde);
- Tente desligar e ligar o buzzer novamente pelo aplicativo.

13. Configurações Técnicas Importantes

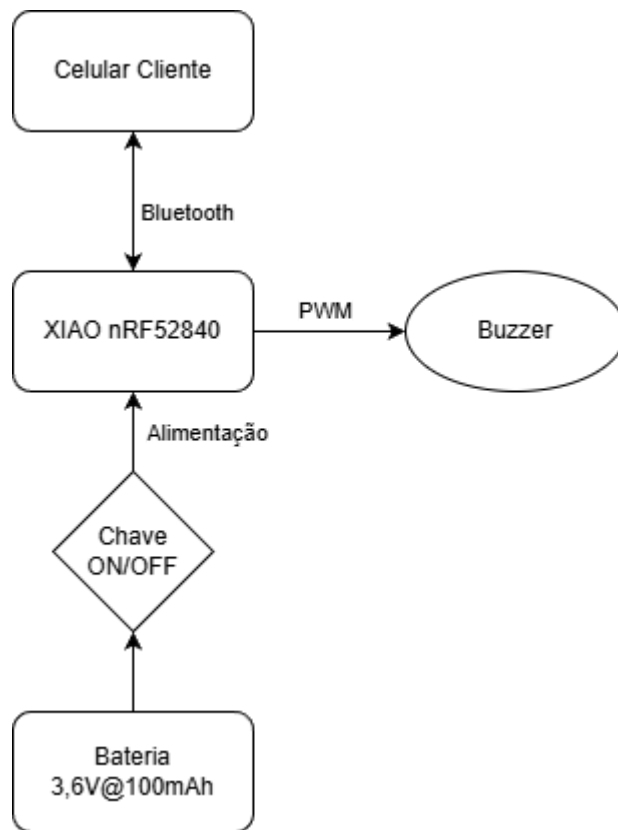
Para manutenção e desenvolvimento do firmware:

3. As habilitações de sistema (BLE, ADC, PWM, gerenciamento de energia, tickless e logs) estão definidas no arquivo **prj.conf**;
4. O mapeamento de pinos (LEDs, PWM, ADC e nó da bateria) está definido no arquivo **xiao_ble.overlay**.

5. Hardware (Vitor)

a. Diagrama de Blocos

Para o hardware, o diagrama de blocos mostra, de forma simplificada, como estão conectados os componentes presentes no nosso sistema:



Como parte inicial do sistema, temos a alimentação, vinda de uma bateria de lítio com 3,6V e 100mAh de capacidade. Essa parte só é liberada para o sistema quando acionada por uma chave, que conecta a bateria ao pino BAT+ da placa XIAO BLE nRF52840. Após esse acionamento, a placa é capaz de se conectar via bluetooth com o celular do cliente e o sistema funciona normalmente com a possibilidade de acionar o buzzer caso seja necessário.

b. Limitações

Após alguns testes foi constatado a limitação relacionada a antena presente na placa da XIAO, uma vez que não consegue entregar toda a potência que o microcontrolador pode oferecer, assim afetando nosso sistema no quesito de medição da distância entre o celular do cliente e o cachorro

c. Instalação e uso do protótipo

Para o protótipo atual, temos a seguinte ordem de montagem:

1. Após a soldagem dos componentes na placa projetada, encaixar a bateria no lugar destinado a ela.
2. Parafusar a placa projetada nos dois furos indicados.
3. Fechar a frente com a conexão dos imãs.

E para usar, é necessário ativar a chave exposta e verificar se o led visto do furo da frente está piscando em azul, se sim, pode utilizar o aplicativo e conectar ao dispositivo. Se não, uma das possibilidades pode ser o descarregamento da bateria, então, conecte um carregador de 5V ao conector USB-C do dispositivo e verifique se um led verde acenderá, se sim, seu dispositivo está carregando. Se isso não funcionar terá de ser feita uma análise mais cuidadosa.

Etapa 4 Semana 4

Manual do Protótipo

1. Introdução

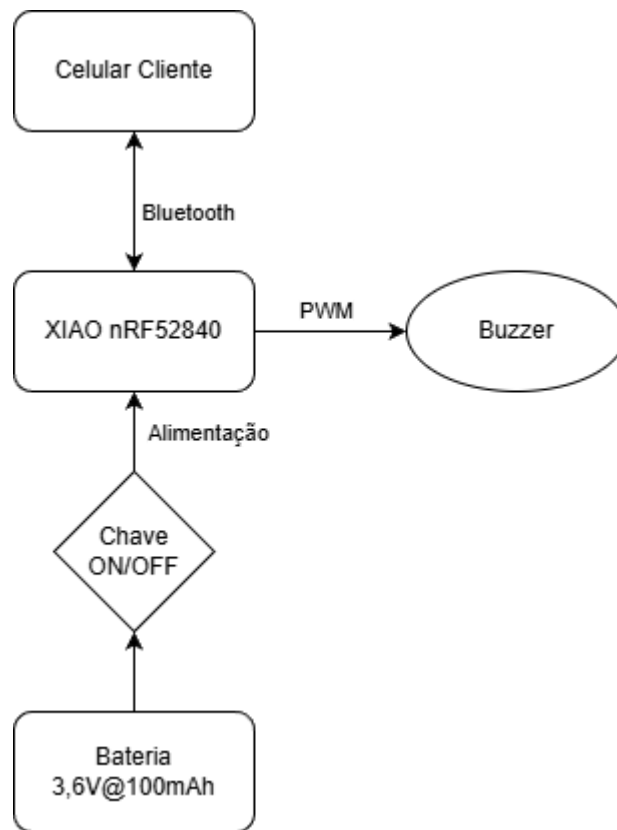
Este manual descreve de forma completa o funcionamento, a montagem, a instalação e o uso do protótipo desenvolvido, cujo objetivo é permitir a comunicação via Bluetooth entre um dispositivo eletrônico acoplado ao cachorro e o celular do cliente, incluindo a possibilidade de acionamento de um buzzer em situações específicas. O documento é destinado a usuários técnicos e não técnicos, servindo tanto como guia de operação quanto como referência básica de funcionamento do sistema.

2. Visão Geral do Sistema

O protótipo é composto por uma placa eletrônica baseada no microcontrolador XIAO BLE nRF52840, alimentada por uma bateria de lítio e equipada com comunicação Bluetooth Low Energy (BLE). O sistema foi projetado para operar de forma portátil, com baixo consumo de energia, permitindo sua utilização contínua enquanto acoplado ao animal.

3. Diagrama de Blocos e Descrição do Hardware

O diagrama de blocos do sistema representa, de forma simplificada, a interligação entre os principais componentes do hardware. A estrutura básica do sistema pode ser descrita da seguinte forma:



- **Fonte de Alimentação:** bateria de lítio de 3,6 V e 100 mAh.
- **Chave de Acionamento:** responsável por liberar ou interromper a alimentação do sistema.
- **Placa de Controle:** XIAO BLE nRF52840, que integra o microcontrolador, o módulo Bluetooth e a antena.
- **Atuador:** buzzer, utilizado para sinalização sonora.
- **Interface de Comunicação:** Bluetooth, para conexão com o aplicativo no celular do cliente.

A alimentação do sistema tem início na bateria de lítio. A tensão só é aplicada à placa quando a chave é acionada, conectando a bateria ao pino BAT+ da XIAO BLE nRF52840. Uma vez energizada, a placa inicializa o microcontrolador e habilita a comunicação Bluetooth, permitindo a conexão com o aplicativo móvel e o funcionamento completo do sistema, incluindo o acionamento do buzzer quando necessário.

4. Funcionamento do Sistema

Após o acionamento da chave de alimentação, o microcontrolador inicia sua rotina de operação. O módulo Bluetooth entra em modo de pareamento e aguarda a conexão com o celular do cliente. Uma vez estabelecida a comunicação, o sistema passa a operar

normalmente, podendo trocar dados com o aplicativo e responder a comandos, como o acionamento do buzzer.

O funcionamento foi projetado para ser simples do ponto de vista do usuário final, exigindo apenas o acionamento da chave e a conexão via aplicativo para que o dispositivo esteja pronto para uso.

5. Limitações do Protótipo

Durante os testes realizados, foi identificada uma limitação relacionada à antena integrada na placa XIAO BLE nRF52840. Apesar de o microcontrolador ser capaz de operar com maior potência de transmissão, a antena embarcada não consegue explorar todo esse potencial, o que impacta diretamente o alcance do sinal Bluetooth.

Essa limitação afeta especialmente a estimativa de distância entre o celular do cliente e o cachorro, baseada na intensidade do sinal recebido (RSSI). Em cenários com obstáculos ou maior distância, a precisão dessa medição pode ser reduzida.

6. Instalação e Montagem do Protótipo

Para a versão atual do protótipo, a montagem deve seguir a seguinte ordem:

1. Após a soldagem de todos os componentes na placa eletrônica projetada, encaixar a bateria no local destinado a ela.
2. Parafusar a placa projetada nos dois furos indicados, garantindo fixação mecânica adequada.
3. Fechar a parte frontal do dispositivo, realizando a conexão por meio dos ímãs previstos no projeto mecânico.

É importante garantir que todos os componentes estejam corretamente posicionados e fixados antes do fechamento final do dispositivo.

7. Uso do Protótipo

Para utilizar o protótipo, o usuário deve:

- Acionar a chave externa de alimentação.
- Verificar se o LED visível pelo furo frontal está piscando na cor azul, o que indica que o dispositivo está ligado e com o Bluetooth ativo.
- Abrir o aplicativo no celular e realizar a conexão com o dispositivo.

Caso o LED azul não esteja piscando, uma das possibilidades é que a bateria esteja descarregada. Nesse caso, deve-se conectar um carregador de 5 V ao conector USB-C

do dispositivo. Se um LED verde acender, isso indica que a bateria está sendo carregada.

Se, mesmo após o procedimento de recarga, o dispositivo não apresentar os sinais esperados de funcionamento, será necessária uma análise mais detalhada do hardware e da alimentação.

Manual do Usuário – Amigo Perto (PDF anexado)

Como usar?

1. Ligue o dispositivo pela chave lateral
2. **LED azul piscando** → dispositivo ligado e procurando conexão
3. Abra o aplicativo no celular
4. Conecte ao dispositivo “**Amigo Perto**”
5. **LED verde piscando** → conectado
6. Use o aplicativo para acionar o alarme (buzzer) ou ver a bateria

1. O que é o Amigo Perto?

O **Amigo Perto** é um dispositivo eletrônico que se conecta ao celular via **Bluetooth**, podendo ser acoplado ao cachorro. Ele permite:

- Localizar o animal por proximidade;
- Acionar um **alarme sonoro (buzzer)** remotamente;
- Acompanhar o **nível de bateria** do dispositivo.

O sistema foi projetado para ser portátil, leve, de baixo consumo e simples de usar.

2. O que vem no dispositivo?

O Amigo Perto possui:

- Bateria recarregável interna (Bateria LiPo 3.7V 100mAh);
- Chave liga/desliga;
- Comunicação Bluetooth;
- Alarme sonoro (buzzer);
- LEDs indicadores de status;
- Entrada USB-C para carregamento.

Tudo já vem integrado em um único módulo.

3. Significado dos LEDs

Os LEDs informam visualmente o estado do dispositivo:

LED azul piscando

- Dispositivo ligado
- Bluetooth ativo
- Aguardando conexão com o celular

LED verde piscando

- Dispositivo conectado ao celular

Nenhum LED aceso

- Dispositivo desligado **ou**
 - Bateria descarregada
-

4. Como ligar e conectar

1. Acione a chave de alimentação;
2. Aguarde alguns segundos;
3. Verifique se o **LED azul está piscando**;
4. Abra o aplicativo no celular;
5. Procure pelo dispositivo **“Amigo Perto”**;
6. Toque em conectar.

Quando a conexão for realizada, o LED verde indicará o estado conectado.

5. Uso do alarme (buzzer)

O alarme pode ser acionado diretamente pelo aplicativo:

- **Alarme ligado:** o dispositivo emite som intermitente;
- **Alarme desligado:** o som para imediatamente.

Segurança:

Se a conexão Bluetooth for perdida, o buzzer é desligado automaticamente, evitando acionamento indevido.

 **Dica:** use o alarme apenas quando necessário para preservar a bateria.

6. Monitoramento da bateria

O aplicativo permite acompanhar o estado da bateria, exibindo:

- **Nível de carga em porcentagem (%):** informação disponível no aplicativo oficial do protótipo, utilizada para acompanhamento do uso e prevenção de desligamentos inesperados.
- **Tensão da bateria (informação de diagnóstico):** não é exibida no aplicativo oficial do protótipo. Essa informação pode ser visualizada por meio do aplicativo de Bluetooth Low Energy da Nordic (**nRF Connect**) ao se conectar ao dispositivo

“Amigo Perto” e acessar as características do **Serviço de Bateria**.

Essas informações ajudam a evitar que o dispositivo descarregue durante o uso.

7. Carregamento da bateria

Se o dispositivo não ligar ou não apresentar LEDs:

1. Conecte um carregador de **5 V** ao conector **USB-C**;
 2. O **LED verde aceso** indica que a bateria está carregando;
 3. Aguarde a recarga antes de utilizar o dispositivo novamente.
-

8. Limitações do dispositivo

O alcance do sinal Bluetooth do Amigo Perto pode variar conforme o ambiente de uso. Fatores como paredes, pessoas, móveis ou estruturas metálicas podem reduzir o alcance e a qualidade da conexão.

A estimativa de distância entre o celular e o dispositivo é baseada na intensidade do sinal Bluetooth e deve ser considerada **apenas como uma referência aproximada**. Em ambientes com obstáculos ou a maiores distâncias, essa estimativa pode apresentar menor precisão. Essas limitações são normais em dispositivos Bluetooth de baixo consumo.

9. Dicas importantes

- Mantenha o celular relativamente próximo ao dispositivo;
 - Evite obstáculos entre o celular e o animal;
 - Recarregue a bateria antes de longos períodos de uso;
 - Não utilize o alarme continuamente;
 - Certifique-se de que o dispositivo esteja bem fixado ao cachorro.
-

10. Solução de problemas

O dispositivo não aparece no aplicativo

- Verifique se o LED azul está piscando;
- Reinicie o Bluetooth do celular;
- Aproxime o celular do dispositivo.

Conexão cai com frequência

- Reduza a distância entre celular e dispositivo;
- Verifique se a bateria está fraca.

O alarme não emite som

- Confirme que o dispositivo está conectado (LED verde);
- Tente desligar e ligar o alarme novamente pelo aplicativo.

11. Considerações finais

O Amigo Perto foi desenvolvido para ser simples, seguro e eficiente. Caso o dispositivo não funcione conforme esperado após todas as verificações, recomenda-se uma análise técnica do hardware ou da alimentação.

Link do Repositório atualizado:

<https://github.com/EmbarcaTech-2025/projeto-amigo-perto/tree/main>